REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD VALLE DEL MOMBOY
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA EN COMPUTACIÓN

DOCUMENTACIÓN TÉCNICA DEL PRODUCTO ECOTRUEQUE VENEZUELA

Anexo técnico del trabajo de grado

DESARROLLO DE UNA APLICACIÓN MÓVIL DE TRUEQUE DE BIENES Y SERVICIOS CON GEOLOCALIZACIÓN Y MÉTRICAS DE CONSUMO RESPONSABLE EN TRUJILLO

Versión del producto: 1.0.0 (MVP) · Etiqueta v1.0.0 · Commit 5224639

Autores:
Br. Rhonny A. Jaimes G.
Br. Norlys A. Castañeda H.

Tutora:
Prof. Caryuly Rosales

Valera, noviembre de 2026

Índice general

1. Introducción 4
1.1. Método de verificación 4
2. Objetivo de la documentación técnica 5
2.1. Objetivo general 5
2.2. Objetivos específicos 5
3. Descripción general de EcoTrueque Venezuela 6
3.1. Identificación del sistema 6
3.2. Objetivo y alcance funcional 6
3.3. Módulos principales 6
4. Arquitectura tecnológica 8
4.1. Descripción de la arquitectura 8
4.2. Tecnologías y herramientas utilizadas 8
4.3. Comunicación entre componentes 9
5. Diagrama entidad-relación 11
5.1. Verificación con la exportación del esquema de Supabase 16
5.2. Relaciones y cardinalidades 17
5.3. Descripción de las entidades 19
6. Inventario SQL de la base de datos 20
6.1. Resumen del inventario de tablas 20
6.2. Diccionario de datos por tabla 21
6.3. Inventario de migraciones y scripts SQL 27
6.4. Script de creación de tablas (extracto real) 29
6.5. Índices 29
6.6. Funciones, triggers y procedimientos 30
6.7. Funciones RPC verificadas en producción 31
7. Seguridad y políticas de acceso 33
7.1. Autenticación 33
7.2. Seguridad a nivel de fila (RLS) 33
7.3. Almacenamiento de archivos 34
7.4. Refuerzos de seguridad 34
8. Evidencias visuales de la aplicación 36
8.1. Logros registrados de la cuenta de prueba 46
8.2. Panel de administración 49
8.3. Representación en dispositivos Android e iOS 54
9. Identificación de versión 57

9.1. Estado de las funcionalidades 57
9.2. Corte de datos de los indicadores M1–M7 58
10. Repositorio y estructura del código fuente 60
10.1. Historial de commits 60
10.2. Estructura del proyecto 61
10.3. Organización del código fuente 62
10.4. Instalación y ejecución 62
10.5. Estrategia y entorno de pruebas 63
10.6. Pruebas unitarias 64
10.7. Pruebas funcionales 64
10.8. Pruebas no funcionales 68
10.9. Registro de defectos 72
10.10. Resumen de la ejecución de pruebas 72
11. Matriz de trazabilidad de evidencias 73
12. Limitaciones técnicas y mejoras aplicadas 75
12.1. Mejoras aplicadas 75
12.2. Limitaciones vigentes 75
13. Conclusiones 77
14. Anexos técnicos 78
Anexo A. Código Mermaid del diagrama entidad-relación 78
Anexo B. Función complete_trade (código real) 78
Anexo C. Función get_nearby_listings (consulta geoespacial) 79
Anexo D. Archivos que acompañan este documento 80
Anexo E. Exportación del esquema desde Supabase 80
Anexo F. Instrumento de usabilidad (System Usability Scale) 83
Anexo G. Evidencias de las pruebas funcionales 84
15. Evidencias pendientes de incorporar o verificar 97

1. Introducción

El presente documento constituye el anexo técnico del trabajo de grado **«Desarrollo de una aplicación móvil de trueque de bienes y servicios con geolocalización y métricas de consumo responsable en Trujillo»**, en el que se desarrolla **EcoTrueque Venezuela**.

Su elaboración responde a la observación formulada por los evaluadores: **«No hay evidencia técnica verificable del producto: diagrama entidad-relación, inventario SQL, capturas reales, versión o repositorio»**. Para atenderla, se analizó el código fuente del repositorio, los scripts de migración SQL, la base de datos en producción en Supabase (mediante consultas de solo lectura) y la aplicación publicada en la web.

Cada afirmación del documento se apoya en una fuente identificable: un archivo del repositorio, un commit, una consulta a la base de datos o una captura de la aplicación en ejecución. Cuando una información no pudo comprobarse, se declara expresamente como pendiente o parcial y se recoge en la sección «Evidencias pendientes de incorporar o verificar».

1.1. Método de verificación

Tabla 1. *Fuentes y procedimientos de verificación utilizados*

Fuente	Procedimiento	Fecha
Repositorio Git	Lectura del historial con git log, rama main sincronizada con origin/main (https://github.com/Norlysc/ecotrueque-venezuela)	4 de noviembre de 2026
Código fuente	Revisión de package.json, app.config.ts, src/ (81 archivos TypeScript) y supabase/	4 de noviembre de 2026
Scripts SQL	Lectura de supabase/migrations/, supabase/all_migrations.sql y scripts complementarios de la raíz	4 de noviembre de 2026
Base de datos en producción	Consultas REST de solo lectura con la clave pública y una cuenta de prueba: columnas reales de cada tabla, conteo de filas visibles y existencia de funciones RPC	4 de noviembre de 2026
Aplicación publicada	Ejecución de https://ecotrueque-venezuela-ten.vercel.app en Google Chrome (vista móvil de 412 × 892 px) y captura automatizada de pantallas con Playwright; las escrituras se bloquearon para no alterar datos	4 de noviembre de 2026
Pruebas y tipos	Ejecución de npx jest --coverage (49 pruebas) y npx tsc --noEmit sobre el commit 14df813	4 de noviembre de 2026
Pruebas funcionales	35 casos ejecutados con Playwright sobre la aplicación publicada (sección 10.7)	4 de noviembre de 2026

Fuente: elaboración propia a partir del análisis del proyecto.

2. Objetivo de la documentación técnica

2.1. Objetivo general

Demostrar, mediante evidencias técnicas verificables, la arquitectura, la estructura de datos, la implementación y el estado actual del producto mínimo viable (MVP) EcoTrueque Venezuela.

2.2. Objetivos específicos

1. Describir la arquitectura tecnológica y las herramientas realmente utilizadas, con sus versiones declaradas en el proyecto.
2. Presentar el diagrama entidad-relación correspondiente a la base de datos implementada.
3. Inventariar las tablas, campos, restricciones, índices, funciones, triggers, políticas de seguridad y migraciones SQL.
4. Aportar capturas reales de la aplicación en ejecución.
5. Identificar la versión del producto y su trazabilidad en el repositorio Git.
6. Señalar con objetividad las funcionalidades implementadas, parciales y pendientes, así como las limitaciones técnicas.

3. Descripción general de EcoTrueque Venezuela

3.1. Identificación del sistema

Nombre del sistema	EcoTrueque Venezuela (nombre técnico de la aplicación: «EcoTrueque», slug «ecotrueque»)
Tipo de sistema	Aplicación móvil multiplataforma (Android, iOS y web) con backend como servicio
Ámbito geográfico	Valera y otras ciudades del estado Trujillo, Venezuela (ubicación por defecto de la exploración: Valera)
Identificador de paquete	com.ecotrueque.app (Android e iOS, según app.config.ts)
Versión declarada	1.0.0 (ver sección 9)
Repositorio	https://github.com/Norlysc/ecotrueque-venezuela
Versión web publicada	https://ecotrueque-venezuela-ten.vercel.app

3.2. Objetivo y alcance funcional

EcoTrueque Venezuela permite que las personas publiquen bienes o servicios que ya no utilizan o que pueden ofrecer, encuentren ofertas cercanas mediante geolocalización, propongan intercambios sin uso de dinero, coordinen el encuentro por chat y valoren la experiencia. Cada trueque completado registra una estimación de CO_2 y residuos evitados, y otorga EcoPoints como mecanismo de gamificación.

El alcance del MVP comprende los módulos descritos en la Tabla 2. Las estimaciones ambientales son valores orientativos configurados por categoría y no constituyen una medición certificada.

3.3. Módulos principales

Tabla 2. *Módulos funcionales del sistema y su implementación en el código*

Módulo	Funciones	Archivos principales
Autenticación	Registro en dos pasos, inicio de sesión, enlace mágico, recuperación y restablecimiento de contraseña	src/app/(auth)/*, reset-password.tsx, services/auth.service.ts
Publicaciones	Explorar, buscar, filtrar, ver detalle, «Me gusta», publicar (asistente de 4 pasos), editar y eliminar (borrado lógico)	(tabs)/index.tsx, publish.tsx, listing/[id].tsx, edit-listing/[id].tsx, services/listings.service.ts
Geolocalización	Mapa con radios de 5, 10, 20 y 50 km; ubicación y distancia aproximadas	(tabs)/map.tsx (móvil), map.web.tsx (web), hooks/useLocation.ts, utils/location.ts
Intercambios	Propuesta, solicitudes recibidas, aceptación o rechazo, confirmación doble y cierre	trade/request.tsx, trade-requests.tsx, services/trades.service.ts
Comunicación	Chat en tiempo real, envío de fotos (galería o cámara), menú de opciones, reporte y bloqueo de usuarios	(tabs)/chat.tsx, chat/[conversationId].tsx, components/chat/ChatOptionsMenu.tsx, services/chat.service.ts, reports.service.ts, blocks.service.ts

Módulo	Funciones	Archivos principales
Reputación	Reseñas de 1 a 5 estrellas y promedio de reputación	review-modal.tsx, user/[userId].tsx
Métricas e impacto	EcoPoints, niveles, metas, distribución, analogías, impacto de la comunidad y logros registrados	eco/dashboard.tsx, services/eco.service.ts, utils/eco.ts
Perfil y soporte	Perfil con pestañas, edición, notificaciones, tema claro u oscuro, ayuda y acerca de	(tabs)/profile.tsx, edit-profile.tsx, notifications.tsx, help.tsx, about.tsx
Administración	Revisión de reportes con filtros y notas, pausa de publicaciones reportadas y avisos a los usuarios	admin/index.tsx, services/admin.service.ts, hooks/useAdmin.ts, utils/reports.ts

Fuente: análisis del directorio src/ del repositorio.

4. Arquitectura tecnológica

4.1. Descripción de la arquitectura

El sistema sigue una arquitectura **cliente – backend como servicio (BaaS)**. El cliente es una única base de código en React Native con Expo, que se ejecuta en Android, iOS y web (mediante react-native-web). El backend es Supabase, que ofrece autenticación, una API REST generada automáticamente sobre PostgreSQL (PostgREST), suscripciones en tiempo real, almacenamiento de archivos y la base de datos PostgreSQL con la extensión PostGIS. No existe un servidor de aplicación propio: la lógica de negocio crítica reside en funciones y triggers de PostgreSQL, y el control de acceso en políticas de seguridad a nivel de fila (RLS).

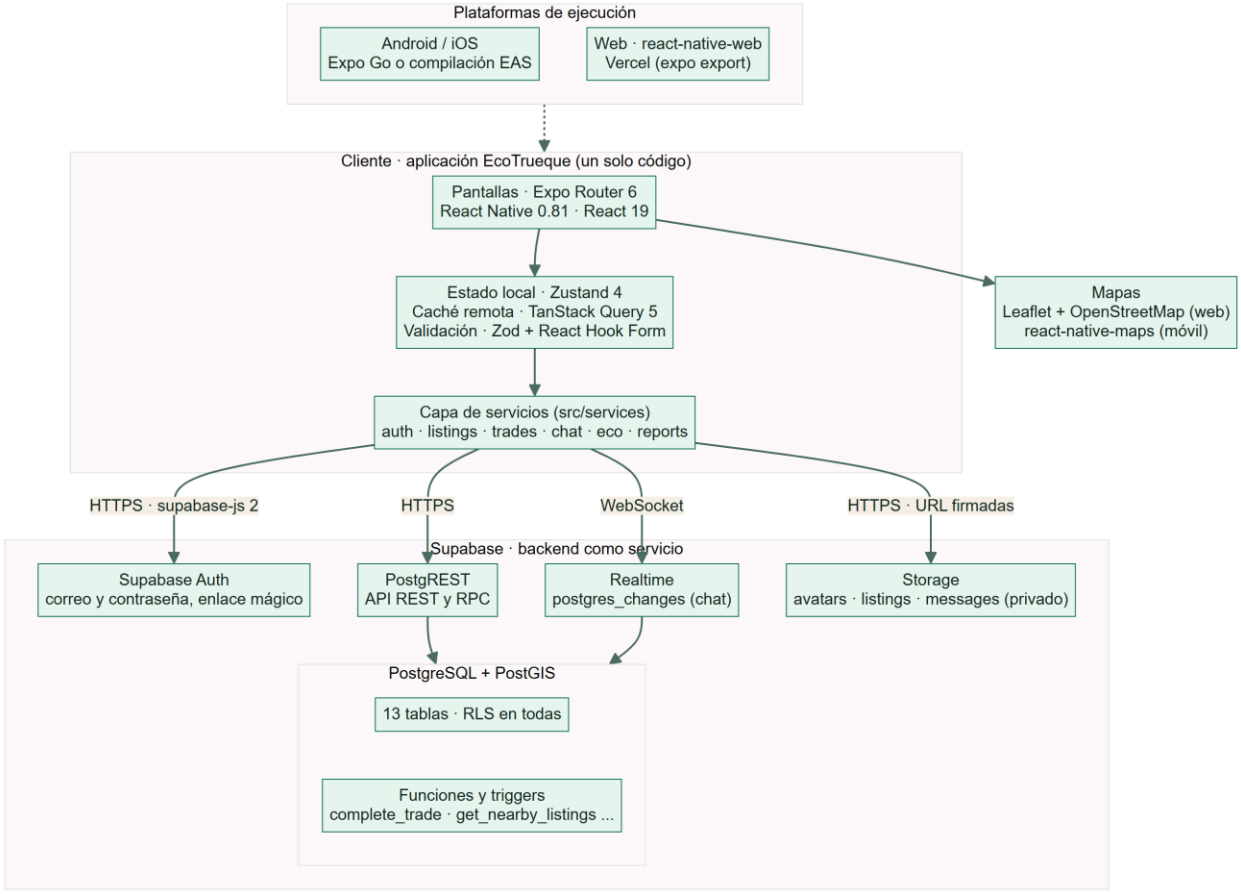


Figura 1. *Arquitectura tecnológica de EcoTrueque Venezuela*

Fuente: elaboración propia a partir del código fuente (src/lib/supabase.ts, src/services/*) y de la configuración de despliegue (vercel.json, eas.json).

4.2. Tecnologías y herramientas utilizadas

Las versiones de la Tabla 3 son las declaradas en package.json del repositorio; los símbolos «~» y «^» indican el rango de versiones compatible que admite npm.

Tabla 3. *Tecnologías del proyecto y versiones declaradas*

Capa	Tecnología	Versión declarada	Uso en el proyecto
Lenguaje	TypeScript	~5.6.0	Tipado estático de todo el código fuente
Framework móvil	React Native	0.81.5	Interfaz nativa multiplataforma
Librería de UI	React / React DOM	19.1.0	Componentes
Plataforma	Expo SDK	~54.0.0	Compilación, módulos nativos y ejecución en Expo Go
Navegación	Expo Router	~6.0.24	Rutas basadas en archivos (src/app)
Web	react-native-web	^0.21.0	Versión web publicada en Vercel
Estado global	Zustand	^4.5.2	Sesión, ubicación, tema y avisos (src/stores)
Datos remotos	TanStack Query	^5.40.0	Caché y sincronización de consultas
Formularios	React Hook Form + Zod	^7.51.5 / ^3.23.8	Captura y validación de datos
Cliente backend	@supabase/supabase-js	^2.43.4	Auth, REST/RPC, Realtime y Storage
Persistencia local	AsyncStorage	2.2.0	Sesión y preferencia de tema
Mapas	react-native-maps / Leaflet + OpenStreetMap	1.20.1 / —	Mapa en móvil / mapa en web
Multimedia	expo-image-picker, expo-image, expo-location	~17.0 / ~3.0 / ~19.0	Fotos, imágenes y ubicación del dispositivo
Gráficos e íconos	victory-native, react-native-svg, lucide-react-native	^36.9.1 / 15.12.1 / ^1.21.0	Panel de impacto e iconografía
Base de datos	PostgreSQL + PostGIS, pg_trgm, unaccent	Gestionado por Supabase	Datos, consultas espaciales y búsqueda
Calidad	Jest + jest-expo, TypeScript (tsc), ESLint	^29.7 / ~54.0 / ~5.6 / ^8.57	49 pruebas automatizadas en src/tests (ver sección 10.6)
Despliegue	Vercel (web) · EAS Build (móvil)	—	vercel.json y eas.json
Control de versiones	Git + GitHub	—	https://github.com/Norlysc/ecotrueque-venezuela

Fuente: package.json, app.config.ts, vercel.json y eas.json del repositorio.

4.3. Comunicación entre componentes

Tabla 4. *Canales de comunicación entre el cliente y Supabase*

Origen → destino	Protocolo	Uso	Ejemplo en el código
Cliente → Supabase Auth	HTTPS (JWT)	Registro, inicio de sesión, enlace mágico, recuperación	auth.service.ts: signUp, signInWithOtp, resetPasswordForEmail
Cliente → PostgREST	HTTPS REST	Lectura y escritura de tablas protegidas por RLS	supabase.from('trade_requests').insert(...)
Cliente → PostgREST (RPC)	HTTPS	Lógica en la base de datos	rpc('get_nearby_listings'), rpc('complete_trade')

Origen → destino	Protocolo	Uso	Ejemplo en el código
Cliente ↔ Realtime	WebSocket	Mensajes nuevos del chat	chat.service.ts: channel().on('postgres_changes', ...)
Cliente → Storage	HTTPS	Avatares, fotos de publicaciones y fotos de chat (URL firmadas de 1 hora)	chat.service.ts: createSignedUrls
Cliente → OpenStreetMap	HTTPS	Teselas del mapa en la versión web	(tabs)/map.web.tsx

Fuente: src/services/*.ts y src/app/(tabs)/map.web.tsx.

5. Diagrama entidad-relación

La base de datos en producción consta de **13 tablas** en el esquema public de PostgreSQL, más la tabla auth.users administrada por Supabase Auth. La estructura se definió en supabase/migrations/001_initial_schema.sql y se amplió con el script 016 (columna conversations.listing_id). La migración 021 añade una decimocuarta tabla, **user_blocks**, para el bloqueo de usuarios (estado: aplicada · verificada en producción). El resultado de la verificación contra la base de datos en producción se resume en la Tabla 5.

Tabla 5. *Verificación de la estructura en la base de datos en producción*

Tabla	Columnas verificadas	Filas visibles para la cuenta de prueba*
profiles	20	10
listings	24	99 visibles (96 activas, 3 intercambiadas)
favorites	4	3 (propias)
trade_requests	12	3 (en las que participa la cuenta)
trades	9	2 (propios)
conversations	7	3 (propias)
messages	9	19 (propias)
reviews	7	4
achievements	9	16
user_achievements	4	0
safe_spots	12	20 (Caracas, Maracaibo, Valencia, Barquisimeto y Mérida); la migración 022 agrega 8 de Valera (aplicada · verificada en producción)
notifications	8	5 (propias)
reports	11	0 visibles
user_blocks	4 (definidas en 021)	Aplicada · verificada en producción

Fuente: consulta REST de solo lectura a la base de datos de Supabase, 4 de noviembre de 2026. *Las filas visibles dependen de las políticas RLS; no representan necesariamente el total almacenado.

La Figura 2 muestra el modelo completo con sus relaciones; las Figuras 3 a 6 detallan los atributos por dominio funcional para asegurar su legibilidad. La notación es de «pata de gallo» (crow's foot): || uno y solo uno, |o cero o uno, o{ cero o muchos. PK indica clave primaria, FK clave foránea y UK clave única.

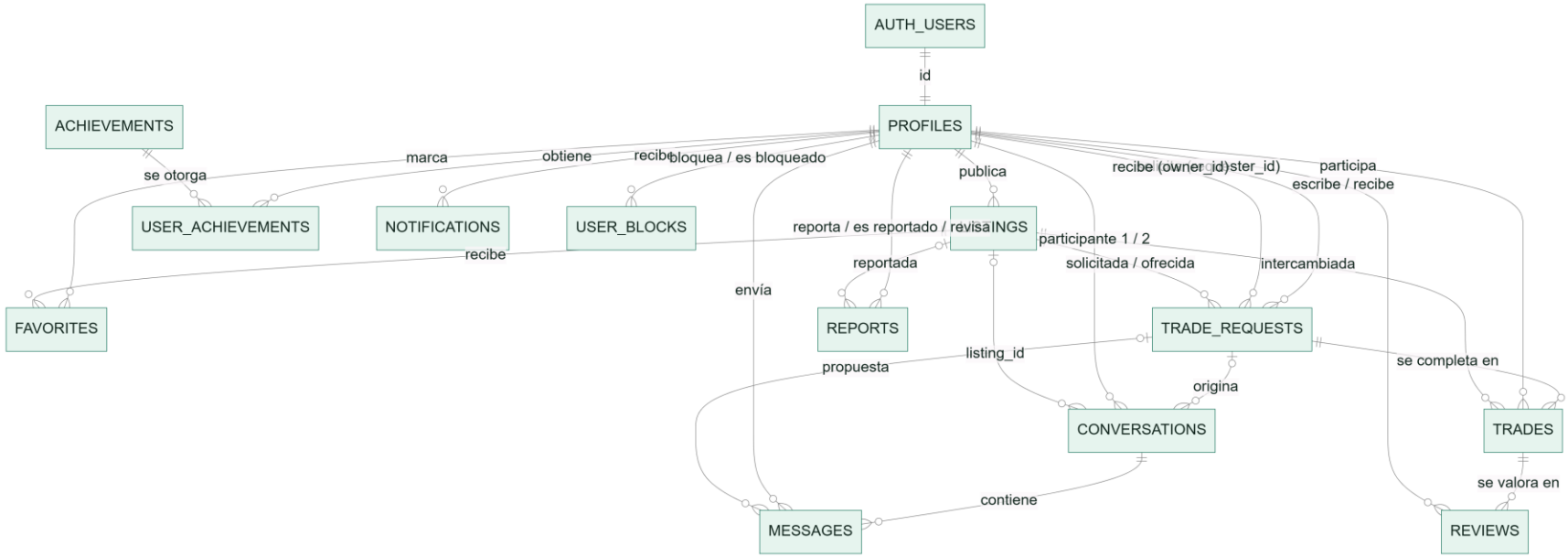


Figura 2. *Diagrama entidad-relación general de EcoTrueque Venezuela (14 tablas y auth.users)*

Fuente: elaboración propia a partir de supabase/migrations/001_initial_schema.sql, script 016 y verificación en la base de datos. Archivo editable: diagramas/der-general.mmd (Mermaid).

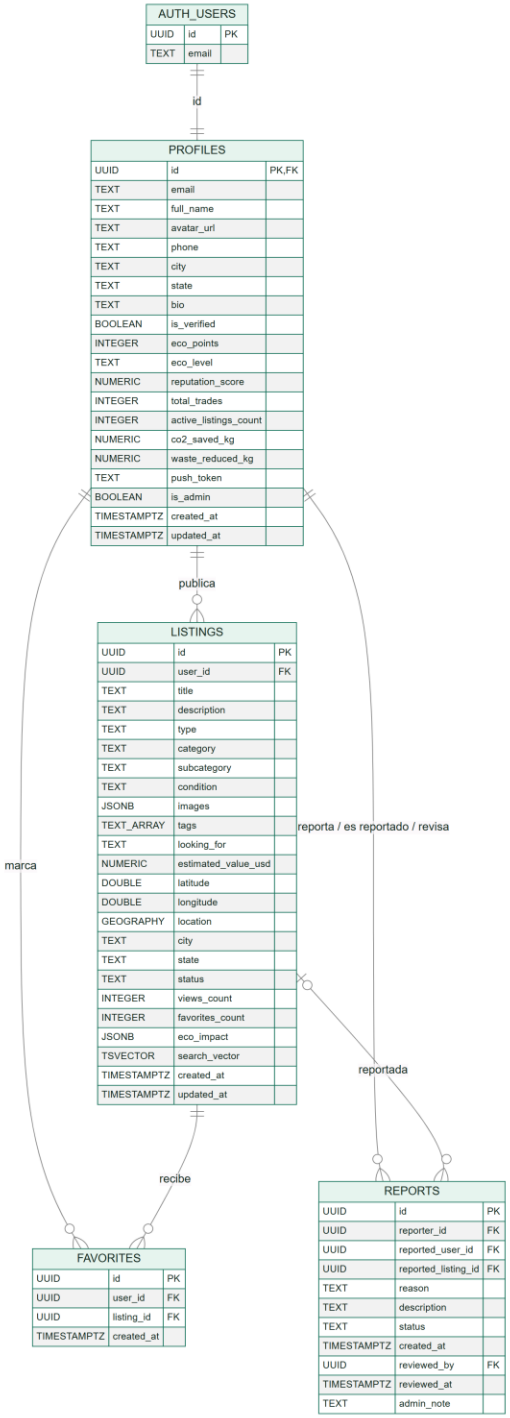


Figura 3. *DER del dominio usuarios y publicaciones*

Fuente: elaboración propia (Mermaid). Archivo editable: diagramas/der-1-usuarios-publicaciones.mmd.

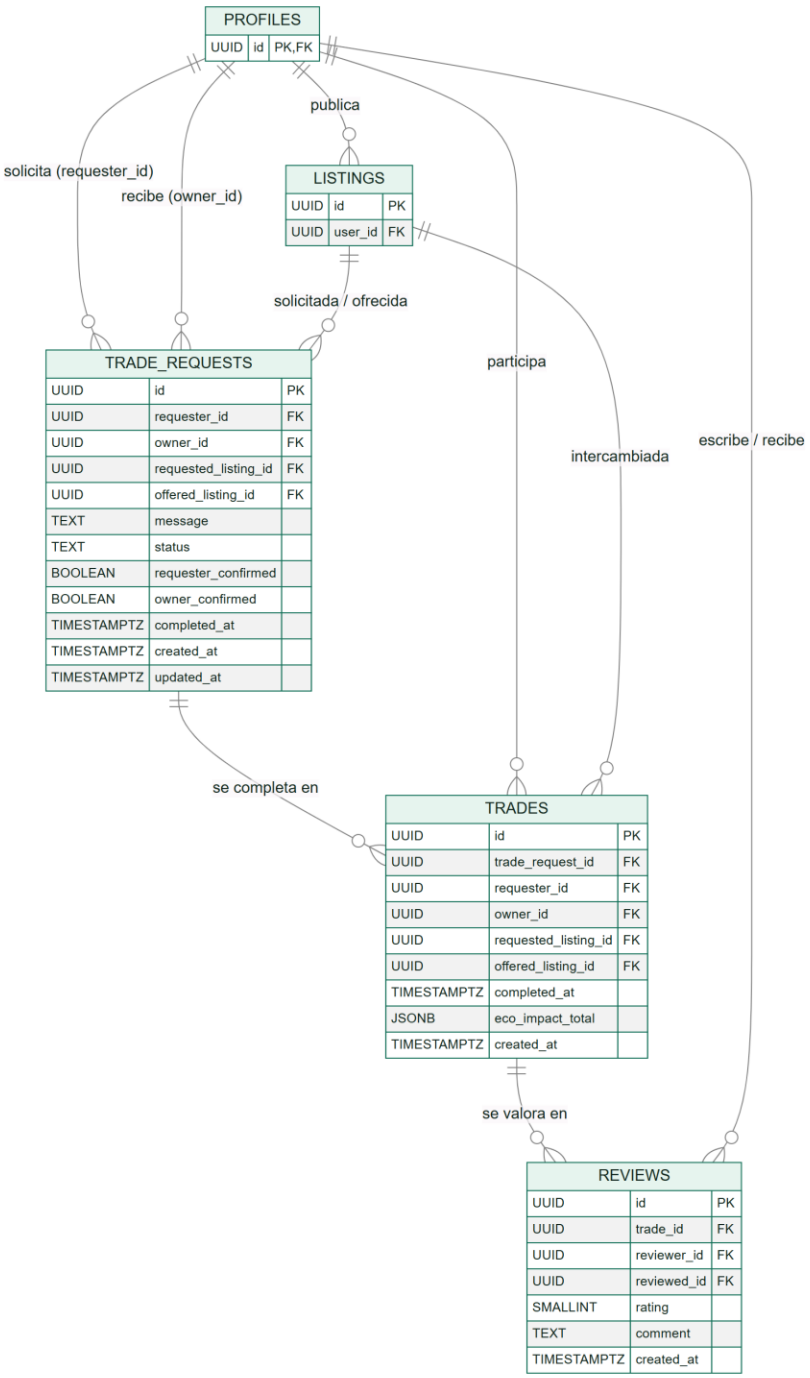


Figura 4. *DER del dominio intercambios y reputación*

Fuente: elaboración propia (Mermaid). Archivo editable: diagramas/der-2-intercambio.mmd.

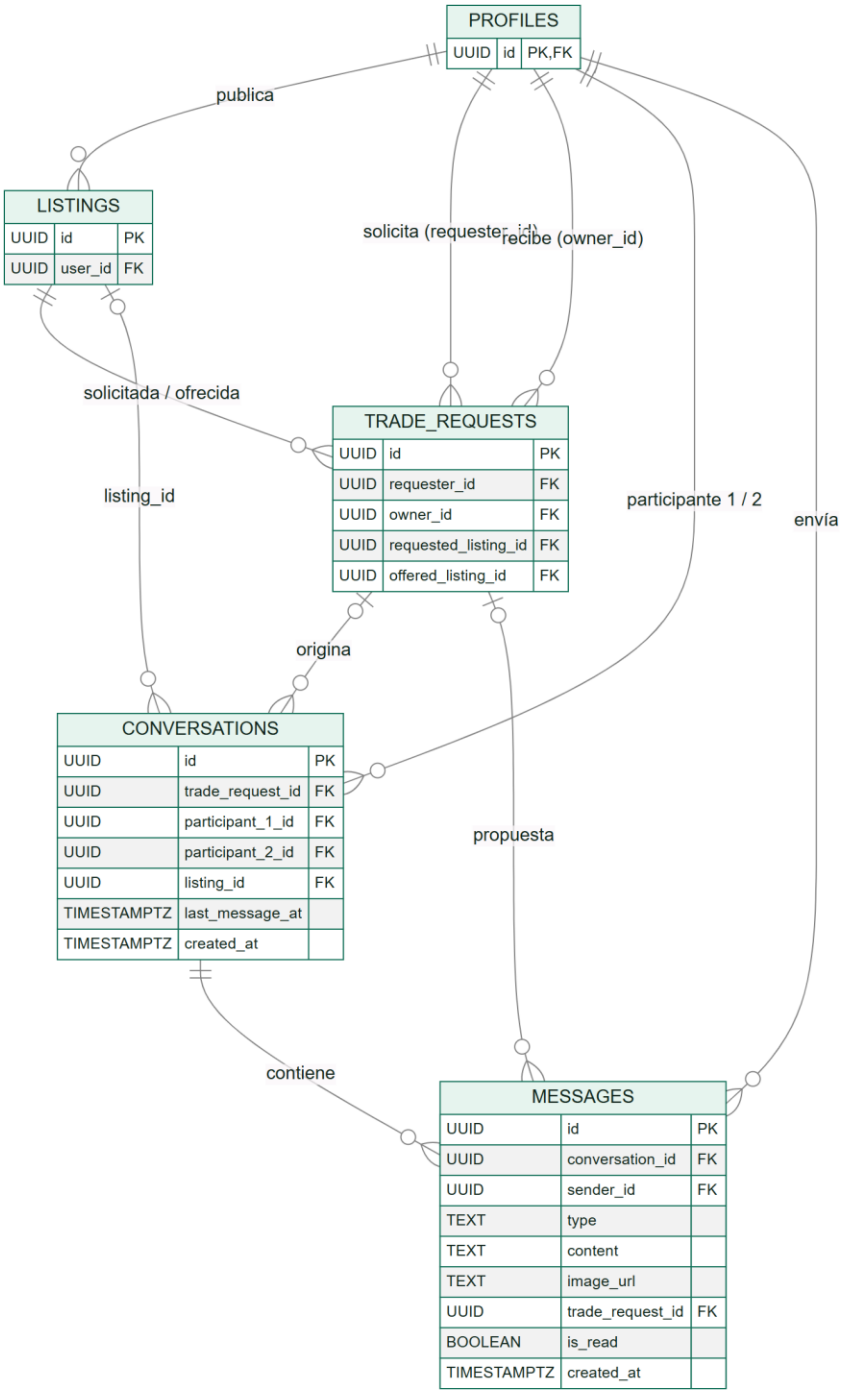


Figura 5. *DER del dominio comunicación (chat)*

Fuente: elaboración propia (Mermaid). Archivo editable: diagramas/der-3-comunicacion.mmd.

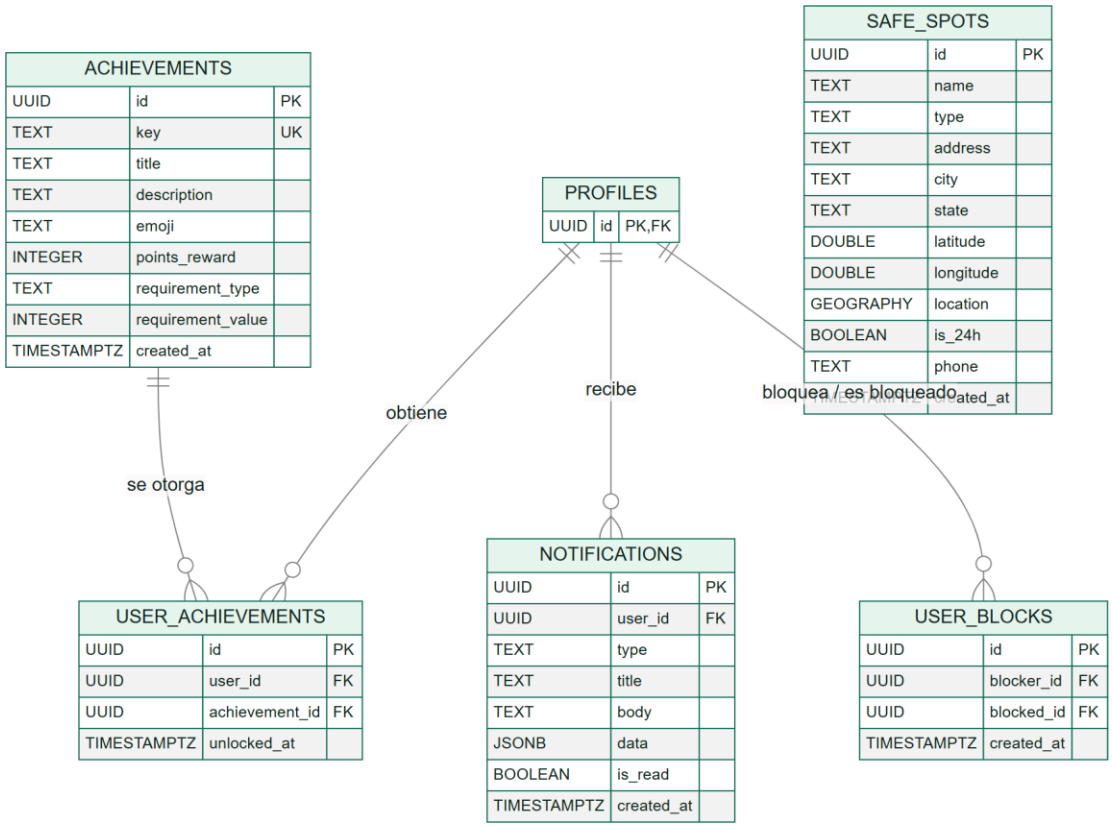


Figura 6. *DER del dominio gamificación, seguridad y soporte*

Fuente: elaboración propia (Mermaid). Archivo editable: diagramas/der-4-gamificacion-soporte.mmd.

5.1. Verificación con la exportación del esquema de Supabase

Los autores exportaron el esquema directamente desde el panel de Supabase el 4 de noviembre de 2026 (definición de tablas en Anexo E y visualizador de esquema en la Figura 7). La Tabla 6 contrasta esa exportación con lo documentado.

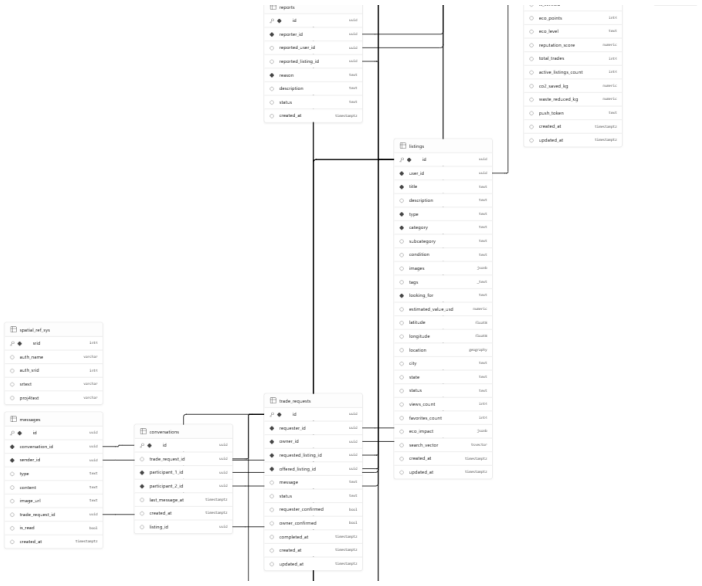


Figura 7. *Visualizador de esquema del panel de Supabase (fragmento)*

Fuente: panel de Supabase del proyecto EcoTrueque Venezuela, captura de los autores, 4 de noviembre de 2026.

Tabla 6. *Contraste entre la exportación del esquema y la documentación*

Aspecto	Resultado de la exportación	Conclusión
Tablas de la aplicación	14: las 13 de la migración 001 más user_blocks (021)	Coincide; confirma que la migración 021 está aplicada
Columnas y tipos	Mismas columnas y tipos de la sección 6.2 (incluida conversations.listing_id de la 016)	Coincide
Claves primarias y foráneas	Todas las PK y FK de la Tabla 7	Coincide
Restricciones CHECK y UNIQUE de columna	eco_level, type, condition, status, rating y achievements.key	Coincide
spatial_ref_sys	Tabla del sistema instalada por la extensión PostGIS	Esperado; no forma parte del modelo de la aplicación
v_user_id	Tabla de una sola columna (id uuid), sin filas	No estaba en ninguna migración: se creó al ejecutar fuera de su bloque la línea «SELECT id INTO v_user_id …» del script 007. Se eliminó con la migración 023 (verificado: la API responde 404)
Formato de la exportación	Omite las restricciones UNIQUE compuestas y las acciones ON DELETE	Esos detalles se verifican con supabase/diagnostics/05_inventario_esquema.sql

Fuente: evidencias/esquema/esquema_supabase_2026-11-04.sql y migraciones del repositorio.

5.2. Relaciones y cardinalidades

Tabla 7. *Relaciones entre entidades*

Entidad (1)	Entidad (N)	Cardinalidad	Clave foránea	Descripción
auth.users	profiles	1 : 1	profiles.id	Cada cuenta de autenticación tiene un perfil (trigger on_auth_user_created)
profiles	listings	1 : N	listings.user_id	Un usuario publica muchas ofertas

Entidad (1)	Entidad (N)	Cardinalidad	Clave foránea	Descripción
profiles	favorites	1 : N	favorites.user_id	Un usuario marca muchas publicaciones
listings	favorites	1 : N	favorites.listing_id	Una publicación recibe muchos «Me gusta»
profiles	trade_requests	1 : N (×2)	requester_id, owner_id	Un usuario envía y recibe muchas propuestas
listings	trade_requests	1 : N (×2)	requested_listing_id, offered_listing_id	Una publicación participa en muchas propuestas
trade_requests	trades	1 : N*	trades.trade_request_id	Una propuesta completada genera su registro de trueque (*sin UNIQUE; en la lógica es 1:1)
profiles	trades	1 : N (×2)	requester_id, owner_id	Partes del trueque completado
listings	trades	1 : N (×2)	requested_listing_id, offered_listing_id	Artículos intercambiados
trade_requests	conversations	0..1 : N	conversations.trade_request_id	La propuesta origina la conversación (opcional)
profiles	conversations	1 : N (×2)	participant_1_id, participant_2_id	Participantes del chat
listings	conversations	0..1 : N	conversations.listing_id	Publicación de referencia (opcional)
conversations	messages	1 : N	messages.conversation_id	Una conversación contiene muchos mensajes
profiles	messages	1 : N	messages.sender_id	Un usuario envía muchos mensajes
trade_requests	messages	0..1 : N	messages.trade_request_id	Mensajes de tipo propuesta
trades	reviews	1 : N	reviews.trade_id	Hasta una reseña por participante (UNIQUE trade_id, reviewer_id)
profiles	reviews	1 : N (×2)	reviewer_id, reviewed_id	Autor y persona valorada
profiles	user_achievements	1 : N	user_achievements.user_id	Logros de un usuario
achievements	user_achievements	1 : N	user_achievements.achievement_id	Un logro se otorga a muchos usuarios
profiles	notifications	1 : N	notifications.user_id	Notificaciones de un usuario
profiles	reports	1 : N (×2)	reporter_id, reported_user_id	Quien reporta y quien es reportado
listings	reports	0..1 : N	reports.reported_listing_id	Publicación reportada (opcional)
profiles	reports	0..1 : N	reports.reviewed_by	Administrador que revisó el reporte (migración 024)
profiles	user_blocks	1 : N (×2)	blocker_id, blocked_id	Quien bloquea y quien es bloqueado (migración 021)

Fuente: restricciones REFERENCES de supabase/migrations/001_initial_schema.sql, 016 y 021.

5.3. Descripción de las entidades

Tabla 8. *Propósito de cada entidad*

Entidad	Propósito
profiles	Perfil público y métricas acumuladas de cada usuario registrado; se crea automáticamente al registrarse en Supabase Auth.
listings	Publicaciones de bienes o servicios ofrecidos para trueque, con ubicación geográfica, imágenes e impacto ecológico estimado.
favorites	Registro de las publicaciones marcadas con «Me gusta» por cada usuario.
trade_requests	Propuestas de intercambio entre un solicitante y el dueño de una publicación; registran el estado y la confirmación doble del encuentro.
trades	Historial de trueques completados con el impacto ecológico total estimado; lo genera la función complete_trade.
conversations	Conversaciones de chat entre dos usuarios, normalmente originadas por una propuesta de trueque.
messages	Mensajes del chat: texto, imagen, propuesta de trueque o mensaje de sistema.
reviews	Reseñas de 1 a 5 estrellas entre las partes de un trueque completado; alimentan la reputación.
achievements	Catálogo de logros de gamificación (16 registros semilla).
user_achievements	Logros otorgados a cada usuario.
safe_spots	Sitios públicos sugeridos para realizar los intercambios de forma segura.
notifications	Notificaciones internas de la aplicación (propuestas, aceptaciones, reseñas, logros).
reports	Reportes de usuarios o publicaciones por comportamiento o contenido inadecuado.
user_blocks	Bloqueos entre usuarios: quien bloquea deja de ver las publicaciones de la otra persona y ninguno puede enviar mensajes ni propuestas al otro (migración 021).

Fuente: supabase/migrations/001_initial_schema.sql y uso en src/services/.

6. Inventario SQL de la base de datos

6.1. Resumen del inventario de tablas

Tabla 9. *Inventario de tablas: claves, restricciones y seguridad*

Tabla	PK	Claves foráneas	Restricciones destacadas	RLS
profiles	id	id → auth.users(id) ON DELETE CASCADE	eco_level: CHECK (seedling, sprout, guardian, protector, hero, legend)	Activa · SELECT: todos (TRUE) · INSERT/UPDATE: auth.uid() = id
listings	id	user_id → profiles(id) ON DELETE CASCADE	type: CHECK (good, service) · condition: CHECK (new, like_new, good, fair, poor) · status: CHECK (active, paused, traded, expired, deleted)	Activa · SELECT: status <> deleted · INSERT/UPDATE/DELETE: auth.uid() = user_id
favorites	id	user_id → profiles(id); listing_id → listings(id) (ambas ON DELETE CASCADE)	UNIQUE (user_id, listing_id)	Activa · SELECT/INSERT/DELETE: auth.uid() = user_id
trade_requests	id	requester_id, owner_id → profiles(id); requested_listing_id, offered_listing_id → listings(id)	status: CHECK (pending, accepted, rejected, cancelled, completed)	Activa · SELECT/UPDATE: solicitante o dueño · INSERT: auth.uid() = requester_id
trades	id	trade_request_id → trade_requests(id); requester_id, owner_id → profiles(id); requested_listing_id, offered_listing_id → listings(id)	NOT NULL en campos obligatorios	Activa · SELECT: solicitante o dueño (las escrituras las hace complete_trade)
conversations	id	trade_request_id → trade_requests(id); participant_1_id, participant_2_id → profiles(id); listing_id → listings(id) ON DELETE SET NULL	NOT NULL en campos obligatorios	Activa · SELECT/INSERT/UPDATE: participante 1 o 2
messages	id	conversation_id → conversations(id); sender_id → profiles(id); trade_request_id → trade_requests(id)	type: CHECK (text, image, trade_proposal, system)	Activa · SELECT/UPDATE: participante de la conversación · INSERT: remitente = auth.uid() y participante
reviews	id	trade_id → trades(id); reviewer_id, reviewed_id → profiles(id)	rating: CHECK (rating BETWEEN 1 AND 5) · UNIQUE (trade_id, reviewer_id)	Activa · SELECT: todos · INSERT: auth.uid() = reviewer_id
achievements	id	—	key: UNIQUE, NOT NULL	Activa · SELECT: todos
user_achievements	id	user_id → profiles(id) ON DELETE CASCADE; achievement_id → achievements(id)	UNIQUE (user_id, achievement_id)	Activa · SELECT: auth.uid() = user_id · INSERT: WITH CHECK (TRUE)
safe_spots	id	—	type: CHECK (police, mall, metro, bank, hospital, public)	Activa · SELECT: todos
notifications	id	user_id → profiles(id) ON DELETE CASCADE	NOT NULL en campos obligatorios	Activa · SELECT/UPDATE: auth.uid() = user_id · INSERT: WITH CHECK (TRUE)

Tabla	PK	Claves foráneas	Restricciones destacadas	RLS
reports	id	reporter_id, reported_user_id → profiles(id); reported_listing_id → listings(id)	status: CHECK (pending, reviewed, resolved)	Activa · INSERT/SELECT: auth.uid() = reporter_id
user_blocks	id	blocker_id, blocked_id → profiles(id) ON DELETE CASCADE	UNIQUE (blocker_id, blocked_id); CHECK (blocker_id <> blocked_id)	Activa · solo bloqueos propios (021)

Fuente: supabase/migrations/001_initial_schema.sql y 004_rls_policies.sql.

6.2. Diccionario de datos por tabla

Las tablas siguientes recogen, para cada entidad, las columnas existentes, su tipo SQL real, las restricciones y su descripción. Las columnas coinciden con las obtenidas de la base de datos en producción (Tabla 5).

6.2.1. Tabla profiles

Perfil público y métricas acumuladas de cada usuario registrado; se crea automáticamente al registrarse en Supabase Auth.

Tabla 10. *Diccionario de datos de la tabla profiles*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK, FK → auth.users(id)	Identificador; coincide con el usuario de Supabase Auth
email	TEXT	NOT NULL	Correo electrónico
full_name	TEXT	NOT NULL	Nombre completo
avatar_url	TEXT	—	URL de la foto de perfil (bucket avatars)
phone	TEXT	—	Teléfono opcional
city	TEXT	—	Ciudad
state	TEXT	—	Estado de Venezuela
bio	TEXT	—	Biografía breve
is_verified	BOOLEAN	DEFAULT FALSE	Indicador de verificación
eco_points	INTEGER	DEFAULT 0	EcoPoints acumulados
eco_level	TEXT	DEFAULT 'seedling'; CHECK (seedling, sprout, guardian, protector, hero, legend)	Nivel ecológico
reputation_score	NUMERIC(3,2)	DEFAULT 0.00	Promedio de calificaciones recibidas
total_trades	INTEGER	DEFAULT 0	Trueques completados
active_listings_count	INTEGER	DEFAULT 0	Publicaciones activas
co2_saved_kg	NUMERIC(10,2)	DEFAULT 0	CO_2 estimado acumulado (kg)
waste_reduced_kg	NUMERIC(10,2)	DEFAULT 0	Residuos estimados acumulados (kg)
push_token	TEXT	—	Token para notificaciones push (no utilizado)

Campo	Tipo de dato	Clave / restricciones	Descripción
is_admin	BOOLEAN	NOT NULL DEFAULT FALSE; protegido por trigger (024)	Rol de administrador; solo se asigna desde Supabase
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de actualización

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.2. Tabla listings

Publicaciones de bienes o servicios ofrecidos para trueque, con ubicación geográfica, imágenes e impacto ecológico estimado.

Tabla 11. *Diccionario de datos de la tabla listings*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK, DEFAULT uuid_generate_v4()	Identificador
user_id	UUID	NOT NULL, FK → profiles(id)	Dueño de la publicación
title	TEXT	NOT NULL	Título
description	TEXT	DEFAULT ''	Descripción
type	TEXT	NOT NULL; CHECK (good, service)	Bien o servicio
category	TEXT	NOT NULL	Categoría (15 definidas en la aplicación)
subcategory	TEXT	—	Subcategoría
condition	TEXT	CHECK (new, like_new, good, fair, poor)	Estado del artículo
images	JSONB	DEFAULT '[]'	Lista de URL de imágenes
tags	TEXT[]	DEFAULT '{}'	Etiquetas
looking_for	TEXT	NOT NULL	Lo que se busca a cambio
estimated_value_usd	NUMERIC(10,2)	—	Valor referencial en USD
latitude	DOUBLE PRECISION	—	Latitud
longitude	DOUBLE PRECISION	—	Longitud
location	GEOGRAPHY(POINT,4326)	Calculado por trigger	Punto geográfico para consultas PostGIS
city	TEXT	—	Ciudad
state	TEXT	—	Estado
status	TEXT	DEFAULT 'active'; CHECK (active, paused, traded, expired, deleted)	Estado de la publicación
views_count	INTEGER	DEFAULT 0	Visualizaciones
favorites_count	INTEGER	DEFAULT 0	«Me gusta» recibidos (trigger)
eco_impact	JSONB	DEFAULT {co2_saved_kg, waste_reduced_kg, estimated_value_usd}	Impacto estimado por categoría
search_vector	TSVECTOR	Calculado por trigger	Vector de búsqueda de texto completo en español
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de actualización

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.3. Tabla favorites

Registro de las publicaciones marcadas con «Me gusta» por cada usuario.

Tabla 12. *Diccionario de datos de la tabla favorites*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
user_id	UUID	NOT NULL, FK → profiles(id)	Usuario que marca
listing_id	UUID	NOT NULL, FK → listings(id)	Publicación marcada
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha

Fuente: supabase/migrations/001_initial_schema.sql; restricción de tabla UNIQUE (user_id, listing_id).

6.2.4. Tabla trade_requests

Propuestas de intercambio entre un solicitante y el dueño de una publicación; registran el estado y la confirmación doble del encuentro.

Tabla 13. *Diccionario de datos de la tabla trade_requests*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
requester_id	UUID	NOT NULL, FK → profiles(id)	Quien propone
owner_id	UUID	NOT NULL, FK → profiles(id)	Dueño de la publicación solicitada
requested_listing_id	UUID	NOT NULL, FK → listings(id)	Publicación que se desea
offered_listing_id	UUID	NOT NULL, FK → listings(id)	Publicación que se ofrece
message	TEXT	—	Mensaje de la propuesta
status	TEXT	DEFAULT 'pending'; CHECK (pending, accepted, rejected, cancelled, completed)	Estado
requester_confirmed	BOOLEAN	DEFAULT FALSE	Confirmación del solicitante
owner_confirmed	BOOLEAN	DEFAULT FALSE	Confirmación del dueño
completed_at	TIMESTAMPTZ	—	Fecha de cierre
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de actualización

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.5. Tabla trades

Historial de trueques completados con el impacto ecológico total estimado; lo genera la función complete_trade.

Tabla 14. *Diccionario de datos de la tabla trades*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
trade_request_id	UUID	NOT NULL, FK → trade_requests(id)	Solicitud de origen

Campo	Tipo de dato	Clave / restricciones	Descripción
requester_id	UUID	NOT NULL, FK → profiles(id)	Solicitante
owner_id	UUID	NOT NULL, FK → profiles(id)	Dueño
requested_listing_id	UUID	NOT NULL, FK → listings(id)	Publicación solicitada
offered_listing_id	UUID	NOT NULL, FK → listings(id)	Publicación ofrecida
completed_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de cierre
eco_impact_total	JSONB	DEFAULT '{}'	CO_2 y residuos estimados del intercambio
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.6. Tabla conversations

Conversaciones de chat entre dos usuarios, normalmente originadas por una propuesta de trueque.

Tabla 15. *Diccionario de datos de la tabla conversations*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
trade_request_id	UUID	FK → trade_requests(id)	Propuesta asociada
participant_1_id	UUID	NOT NULL, FK → profiles(id)	Primer participante
participant_2_id	UUID	NOT NULL, FK → profiles(id)	Segundo participante
last_message_at	TIMESTAMPTZ	—	Fecha del último mensaje
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación
listing_id	UUID	FK → listings(id) ON DELETE SET NULL (agregada en 016)	Publicación relacionada

Fuente: supabase/migrations/001_initial_schema.sql y 016_conversations_listing_id.sql.

6.2.7. Tabla messages

Mensajes del chat: texto, imagen, propuesta de trueque o mensaje de sistema.

Tabla 16. *Diccionario de datos de la tabla messages*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
conversation_id	UUID	NOT NULL, FK → conversations(id) ON DELETE CASCADE	Conversación
sender_id	UUID	NOT NULL, FK → profiles(id)	Remitente
type	TEXT	DEFAULT 'text'; CHECK (text, image, trade_proposal, system)	Tipo de mensaje
content	TEXT	—	Contenido textual
image_url	TEXT	—	Ruta de la imagen en el bucket privado messages
trade_request_id	UUID	FK → trade_requests(id)	Propuesta referida
is_read	BOOLEAN	DEFAULT FALSE	Lectura

Campo	Tipo de dato	Clave / restricciones	Descripción
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de envío

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.8. Tabla reviews

Reseñas de 1 a 5 estrellas entre las partes de un trueque completado; alimentan la reputación.

Tabla 17. *Diccionario de datos de la tabla reviews*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
trade_id	UUID	NOT NULL, FK → trades(id)	Trueque valorado
reviewer_id	UUID	NOT NULL, FK → profiles(id)	Autor
reviewed_id	UUID	NOT NULL, FK → profiles(id)	Persona valorada
rating	SMALLINT	NOT NULL; CHECK (rating BETWEEN 1 AND 5)	Calificación
comment	TEXT	—	Comentario opcional
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha

Fuente: supabase/migrations/001_initial_schema.sql; restricción de tabla UNIQUE (trade_id, reviewer_id).

6.2.9. Tabla achievements

Catálogo de logros de gamificación (16 registros semilla).

Tabla 18. *Diccionario de datos de la tabla achievements*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
key	TEXT	UNIQUE, NOT NULL	Clave interna
title	TEXT	NOT NULL	Título
description	TEXT	NOT NULL	Descripción
emoji	TEXT	NOT NULL	Ícono
points_reward	INTEGER	DEFAULT 0	EcoPoints otorgados
requirement_type	TEXT	NOT NULL	Tipo de requisito
requirement_value	INTEGER	NOT NULL	Valor requerido
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.10. Tabla user_achievements

Logros otorgados a cada usuario.

Tabla 19. *Diccionario de datos de la tabla user_achievements*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
user_id	UUID	NOT NULL, FK → profiles(id)	Usuario

Campo	Tipo de dato	Clave / restricciones	Descripción
achievement_id	UUID	NOT NULL, FK → achievements(id)	Logro
unlocked_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de obtención

Fuente: supabase/migrations/001_initial_schema.sql; restricción de tabla UNIQUE (user_id, achievement_id).

6.2.11. Tabla safe_spots

Sitios públicos sugeridos para realizar los intercambios de forma segura.

Tabla 20. *Diccionario de datos de la tabla safe_spots*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
name	TEXT	NOT NULL	Nombre
type	TEXT	NOT NULL; CHECK (police, mall, metro, bank, hospital, public)	Tipo de sitio
address	TEXT	NOT NULL	Dirección
city	TEXT	NOT NULL	Ciudad
state	TEXT	NOT NULL	Estado
latitude	DOUBLE PRECISION	NOT NULL	Latitud
longitude	DOUBLE PRECISION	NOT NULL	Longitud
location	GEOGRAPHY(POINT,4326)	Calculado por trigger	Punto geográfico
is_24h	BOOLEAN	DEFAULT FALSE	Atención 24 horas
phone	TEXT	—	Teléfono
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.12. Tabla notifications

Notificaciones internas de la aplicación (propuestas, aceptaciones, reseñas, logros).

Tabla 21. *Diccionario de datos de la tabla notifications*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
user_id	UUID	NOT NULL, FK → profiles(id)	Destinatario
type	TEXT	NOT NULL	Tipo (trade_request, trade_accepted, new_review…)
title	TEXT	NOT NULL	Título
body	TEXT	NOT NULL	Texto
data	JSONB	DEFAULT '{}'	Datos de navegación
is_read	BOOLEAN	DEFAULT FALSE	Lectura
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.13. Tabla reports

Reportes de usuarios o publicaciones por comportamiento o contenido inadecuado.

Tabla 22. *Diccionario de datos de la tabla reports*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK	Identificador
reporter_id	UUID	NOT NULL, FK → profiles(id)	Quien reporta
reported_user_id	UUID	FK → profiles(id)	Usuario reportado
reported_listing_id	UUID	FK → listings(id)	Publicación reportada
reason	TEXT	NOT NULL	Motivo
description	TEXT	—	Detalle
status	TEXT	DEFAULT 'pending'; CHECK (pending, reviewed, resolved)	Estado del reporte
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha
reviewed_by	UUID	FK → profiles(id) ON DELETE SET NULL (024)	Administrador que lo revisó
reviewed_at	TIMESTAMPTZ	— (024)	Fecha de la revisión
admin_note	TEXT	— (024)	Nota de la revisión

Fuente: supabase/migrations/001_initial_schema.sql.

6.2.14. Tabla user_blocks

Bloqueos entre usuarios: quien bloquea deja de ver las publicaciones de la otra persona y ninguno puede enviar mensajes ni propuestas al otro (migración 021).

Tabla 23. *Diccionario de datos de la tabla user_blocks*

Campo	Tipo de dato	Clave / restricciones	Descripción
id	UUID	PK, DEFAULT uuid_generate_v4()	Identificador
blocker_id	UUID	NOT NULL, FK → profiles(id)	Quien bloquea
blocked_id	UUID	NOT NULL, FK → profiles(id)	Persona bloqueada
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha del bloqueo

Fuente: supabase/migrations/021_bloqueo_usuarios.sql; restricción de tabla UNIQUE (blocker_id, blocked_id); CHECK (blocker_id <> blocked_id).

6.3. Inventario de migraciones y scripts SQL

Todos los scripts SQL están en la carpeta supabase/migrations/, con numeración correlativa de 001 a 021. Hasta el 4 de noviembre de 2026, los scripts 008 a 017 estaban en la raíz del proyecto como archivos .txt; se trasladaron a esa carpeta, se eliminaron las copias duplicadas de 008 y 010 y el consolidado ya aplicado quedó en supabase/historico/ como referencia. El estado de la Tabla 24 indica lo que pudo comprobarse en la base de datos.

Tabla 24. *Inventario de migraciones y scripts SQL*

Archivo	Contenido	Estado verificado
001_initial_schema.sql	Extensiones (uuid-ossp, postgis, pg_trgm, unaccent) y 13 tablas	Aplicada · estructura verificada
002_indexes.sql	27 índices y triggers de ubicación	Aplicada · no consultable con clave pública
003_functions.sql	Funciones, triggers y RPC del negocio	Aplicada · RPC de lectura verificadas
004_rls_policies.sql	RLS en 13 tablas y 31 políticas	Aplicada · efecto observado
005_storage.sql	Buckets avatars, listings y messages, y 10 políticas de Storage	Aplicada · fotos firmadas del chat
006_seed_data.sql	16 logros y 20 sitios seguros	Aplicada · 16 y 20 registros
007, 008, 009, 012, 013, 018, 019	Publicaciones e imágenes de demostración	Datos de demostración
010_soft_delete_listing.sql	Función delete_own_listing	Aplicada · usada por la app
011_migrar_trujillo.sql	Reubica usuarios y publicaciones en el estado Trujillo	Aplicada · datos en Valera
014_fix_favorites_is_favorited.sql	get_nearby_listings con datos del autor e is_favorited	Aplicada · columnas presentes
015_unread_message_count.sql	RPC get_unread_message_count (corregida)	Aplicada · verificada en producción
016_conversations_listing_id.sql	Columna conversations.listing_id	Aplicada · columna presente
017_fix_reputation_score.sql	Corrige el trigger de reputación	Aplicada · reputación 5.0 observada
020_politicas_seguridad.sql	Correo privado y políticas de notificaciones, logros y fotos del chat	Aplicada · verificada en producción
021_bloqueo_usuarios.sql	Tabla user_blocks, función is_blocked_between y políticas	Aplicada · verificada en producción
022_sitios_seguros_valera.sql	8 lugares públicos de Valera tomados de OpenStreetMap	aplicada · verificada en producción
024_panel_administracion.sql	Rol de administrador, política de lectura de reportes y funciones del panel	Aplicada · Norlys Castañeda registrada como administradora
023_limpieza_v_user_id.sql	Elimina la tabla accidental v_user_id	Aplicada · verificada en producción (404)
supabase/all_migrations.sql	Consolidado de 001 a 006 para instalaciones nuevas	Script de instalación
supabase/aplicar_mejoras_2026-11.sql	Reúne 015, 020, 021 y 022 en un solo archivo	Script de actualización
supabase/historico/todas_las_migraciones_010-016.sql	Consolidado aplicado anteriormente	Solo referencia
supabase/diagnostics/01–04 *.sql	Diagnóstico de mapa, trigger de ubicación y permisos RPC	Scripts de soporte
supabase/diagnostics/05_inventario_esquema.sql	Inventario del esquema desde el servidor (tablas, restricciones, índices, triggers, funciones y políticas)	Consulta de solo lectura
supabase/diagnostics/06_indicadores_m1_m7.sql	Corte de los indicadores M1–M7 para un período	Consulta de solo lectura

Fuente: estructura del repositorio y consultas a la base de datos, 4 de noviembre de 2026.

Para una instalación nueva se ejecuta supabase/all_migrations.sql y después las migraciones 010, 014, 015, 016, 017, 020, 021 y 022. Para actualizar la base de datos existente basta con supabase/aplicar_mejoras_2026-11.sql, que reúne 015, 020 y 021 y puede ejecutarse más de una vez.

6.4. Script de creación de tablas (extracto real)

Se reproduce, sin modificaciones, la definición de la tabla listings, núcleo del sistema. El script completo de las 13 tablas está en supabase/migrations/001_initial_schema.sql.

```
CREATE TABLE IF NOT EXISTS public.listings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT DEFAULT '',
  type TEXT NOT NULL CHECK (type IN ('good','service')),
  category TEXT NOT NULL,
  subcategory TEXT,
  condition TEXT CHECK (condition IN ('new','like_new','good','fair','poor')),
  images JSONB DEFAULT '[]'::JSONB,
  tags TEXT[] DEFAULT '{}',
  looking_for TEXT NOT NULL,
  estimated_value_usd NUMERIC(10,2),
  latitude DOUBLE PRECISION,
  longitude DOUBLE PRECISION,
  location GEOGRAPHY(POINT,4326),
  city TEXT,
  state TEXT,
  status TEXT DEFAULT 'active' CHECK (status IN ('active','paused','traded','expired','deleted')),
  views_count INTEGER DEFAULT 0,
  favorites_count INTEGER DEFAULT 0,
  eco_impact JSONB DEFAULT '{"co2_saved_kg":0,"waste_reduced_kg":0,"estimated_value_usd":0}'::JSONB,
  search_vector TSVECTOR,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

Fuente: supabase/migrations/001_initial_schema.sql, líneas 39 a 64.

6.5. Índices

Tabla 25. *Índices definidos en la base de datos*

Índice	Tabla	Tipo	Finalidad
idx_listings_location	listings	GIST (location)	Consultas geoespaciales por radio
idx_listings_search_vector	listings	GIN (search_vector)	Búsqueda de texto completo
idx_listings_title_trgm	listings	GIN (title gin_trgm_ops)	Búsqueda aproximada por título
idx_listings_description_trgm	listings	GIN (description gin_trgm_ops)	Búsqueda aproximada por descripción
idx_listings_user_id / status / category / type	listings	B-tree	Filtros frecuentes
idx_listings_created_at	listings	B-tree (created_at DESC)	Orden por recientes
idx_trade_requests_requester_id / owner_id / status	trade_requests	B-tree	Solicitudes por usuario y estado
idx_messages_conversation_id / sender_id	messages	B-tree	Mensajes por conversación

Índice	Tabla	Tipo	Finalidad
idx_messages_created_at	messages	B-tree (created_at ASC)	Orden cronológico
idx_messages_is_read	messages	B-tree parcial (WHERE is_read = FALSE)	Conteo de no leídos
idx_conversations_participant_1 / participant_2	conversations	B-tree	Conversaciones por usuario
idx_conversations_last_message_at	conversations	B-tree (DESC NULLS LAST)	Orden de la bandeja
idx_conversations_listing_id	conversations	B-tree parcial (WHERE listing_id IS NOT NULL)	Script 016
idx_favorites_user_id / listing_id	favorites	B-tree	Favoritos por usuario y publicación
idx_notifications_user_id	notifications	B-tree	Notificaciones por usuario
idx_notifications_is_read	notifications	B-tree parcial (WHERE is_read = FALSE)	No leídas
idx_notifications_created_at	notifications	B-tree (DESC)	Orden por fecha
idx_safe_spots_location	safe_spots	GIST (location)	Sitios seguros cercanos
idx_profiles_eco_points	profiles	B-tree (eco_points DESC)	Clasificación por EcoPoints
idx_profiles_city	profiles	B-tree	Filtro por ciudad
idx_user_blocks_blocker / blocked	user_blocks	B-tree	Consulta de bloqueos (migración 021)

Fuente: supabase/migrations/002_indexes.sql (27 índices), 016 (1 índice) y 021 (2 índices).

6.6. Funciones, triggers y procedimientos

Tabla 26. *Funciones y triggers de PostgreSQL*

Función	Tipo / disparador	Seguridad	Propósito	Script
handle_new_user()	Trigger AFTER INSERT en auth.users (on_auth_user_created)	SECURITY DEFINER	Crea el perfil del usuario recién registrado; desde 020 no copia el correo al perfil público	003, 020
update_listing_location()	Trigger BEFORE INSERT/UPDATE en listings	—	Calcula location (PostGIS) a partir de latitud y longitud	002
update_safe_spot_location()	Trigger BEFORE INSERT/UPDATE en safe_spots	—	Calcula location de los sitios seguros	002
update_listing_search_vector()	Trigger BEFORE INSERT/UPDATE en listings	—	Construye el vector de búsqueda en español (unaccent)	003
handle_favorite_change()	Trigger AFTER INSERT/DELETE en favorites	—	Actualiza favorites_count	003
update_active_listings_count()	Trigger AFTER INSERT/UPDATE/DELETE en listings	—	Recalcula las publicaciones activas del perfil	003
update_user_reputation() / update_reputation_score()	Trigger AFTER INSERT en reviews (trg_update_reputation)	SECURITY DEFINER (017)	Recalcula reputation_score como promedio de calificaciones	003, 017

Función	Tipo / disparador	Seguridad	Propósito	Script
complete_trade(p_request_id)	RPC	SECURITY DEFINER	Cierra el trueque, registra trades, suma EcoPoints, CO_2 y residuos y actualiza niveles	003
update_eco_level(p_user_id)	Función interna	—	Asigna el nivel según EcoPoints (0, 100, 300, 700, 1500, 3000)	003
get_nearby_listings(...)	RPC (STABLE)	—	Publicaciones activas dentro de un radio con filtros, búsqueda y distancia	003, 014
get_trade_matches(...)	RPC (STABLE)	—	Sugerencias de intercambio con puntaje ponderado (categoría, búsqueda, tipo, estado)	003
get_nearby_safe_spots(...)	RPC (STABLE)	—	Sitios seguros cercanos; la usa el mapa web y móvil	003
delete_own_listing(p_listing_id)	RPC	SECURITY DEFINER	Borrado lógico (status = deleted) solo por el dueño	010
get_unread_message_count(p_user_id)	RPC	SECURITY DEFINER	Conteo de mensajes no leídos del propio usuario; corregida para usar participant_1_id y participant_2_id	015
protect_is_admin()	Trigger BEFORE INSERT/UPDATE en profiles	—	Impide que un usuario se asigne el rol de administrador desde la app	024
is_admin()	Función (STABLE)	SECURITY DEFINER	Indica si el usuario autenticado es administrador	024
admin_update_report(p_report_id, p_status, p_note)	RPC	SECURITY DEFINER	Cambia el estado de un reporte con nota, registra quién y cuándo y avisa a quien reportó	024
admin_pause_listing(p_listing_id)	RPC	SECURITY DEFINER	Pausa una publicación reportada y avisa a su dueño	024
is_blocked_between(p_a, p_b)	Función (STABLE)	SECURITY DEFINER	Indica si existe un bloqueo entre dos personas; la usan las políticas de mensajes y propuestas	021

Fuente: supabase/migrations/002_indexes.sql, 003_functions.sql y migraciones 010, 014, 015, 017, 020 y 021.

La función **complete_trade** concentra la regla de negocio del cierre de un trueque y el cálculo de EcoPoints: EcoPoints = ⌊CO_2 estimado × 2 + residuos estimados × 3 + 20⌋, aplicado a ambas partes. Su código real se incluye en el Anexo B. Además, cuando una persona alcanza un logro del catálogo, la aplicación lo registra en user_achievements y suma su recompensa (points_reward) a los EcoPoints (src/services/eco.service.ts).

6.7. Funciones RPC verificadas en producción

Tabla 27. *Comprobación de funciones RPC en la base de datos*

Función	Resultado de la consulta	Uso en la aplicación
get_nearby_listings	Responde; devuelve distance_km, user_full_name, user_avatar_url, user_eco_level e is_favorited (versión del script 014)	Explorar y mapa
get_nearby_safe_spots	Responde; 8 sitios en un radio de 10 km de Valera	Sitios seguros del mapa

Función	Resultado de la consulta	Uso en la aplicación
get_trade_matches	Responde	Sugerencias en el detalle de publicación
get_unread_message_count	Responde (0 mensajes sin leer)	Contador de mensajes no leídos
is_blocked_between	Responde	Políticas de mensajes y propuestas
complete_trade, delete_own_listing	No se invocaron para no modificar datos	Cierre de trueque y borrado lógico

Fuente: consultas RPC de solo lectura, 4/11/2026, 6:04:17 p. m..

7. Seguridad y políticas de acceso

7.1. Autenticación

La autenticación se delega en Supabase Auth: registro con correo y contraseña (signUp), inicio de sesión, enlace mágico (signInWithOtp) y recuperación de contraseña (resetPasswordForEmail). La sesión se conserva con AsyncStorage y se renueva automáticamente (src/lib/supabase.ts). El trigger on_auth_user_created crea el perfil público al registrarse. Las credenciales del proyecto se leen de variables de entorno (EXPO_PUBLIC_SUPABASE_URL y EXPO_PUBLIC_SUPABASE_ANON_KEY); el archivo .env está excluido del repositorio y se ofrece una plantilla .env.example.

7.2. Seguridad a nivel de fila (RLS)

Las 13 tablas tienen activada la seguridad a nivel de fila (ALTER TABLE … ENABLE ROW LEVEL SECURITY) y suman 31 políticas; la tabla user_blocks de la migración 021 también la activa. La Tabla 28 resume las reglas vigentes y los cambios que introducen las migraciones 020 y 021. Su efecto se observó en las consultas de verificación, en las que la cuenta de prueba solo obtuvo sus propias conversaciones, mensajes, favoritos y notificaciones.

Tabla 28. *Políticas RLS por tabla*

Tabla	Regla vigente (004)	Cambio en 020 / 021
profiles	SELECT: todos (TRUE) · INSERT/UPDATE: auth.uid() = id	El correo deja de guardarse en el perfil público (020)
listings	SELECT: status <> deleted · INSERT/UPDATE/DELETE: auth.uid() = user_id	—
favorites	SELECT/INSERT/DELETE: auth.uid() = user_id	—
trade_requests	SELECT/UPDATE: solicitante o dueño · INSERT: auth.uid() = requester_id	INSERT exige además que no exista un bloqueo entre ambos (021)
trades	SELECT: solicitante o dueño (las escrituras las hace complete_trade)	—
conversations	SELECT/INSERT/UPDATE: participante 1 o 2	—
messages	SELECT/UPDATE: participante de la conversación · INSERT: remitente = auth.uid() y participante	INSERT exige además que no exista un bloqueo entre los participantes (021)
reviews	SELECT: todos · INSERT: auth.uid() = reviewer_id	—
achievements	SELECT: todos	—
user_achievements	SELECT: auth.uid() = user_id · INSERT: WITH CHECK (TRUE)	INSERT: solo auth.uid() = user_id (020)
safe_spots	SELECT: todos	—
notifications	SELECT/UPDATE: auth.uid() = user_id · INSERT: WITH CHECK (TRUE)	INSERT: para uno mismo o para alguien con quien se tiene una propuesta de trueque (020)
reports	INSERT/SELECT: auth.uid() = reporter_id	SELECT adicional para administradores: is_admin() (024); las actualizaciones solo mediante admin_update_report

Tabla	Regla vigente (004)	Cambio en 020 / 021
user_blocks	— (tabla nueva)	SELECT/INSERT/DELETE: solo auth.uid() = blocker_id (021)

Fuente: supabase/migrations/004_rls_policies.sql, 020_politicas_seguridad.sql y 021_bloqueo_usuarios.sql. Estado de 020: aplicada · verificada en producción; de 021: aplicada · verificada en producción.

Extracto real de la política de lectura de mensajes, que restringe el acceso a los participantes de la conversación:

```
CREATE POLICY "messages_select_participant"
  ON public.messages FOR SELECT
  USING (
    EXISTS (
      SELECT 1 FROM public.conversations c
      WHERE c.id = conversation_id
      AND (c.participant_1_id = auth.uid() OR c.participant_2_id = auth.uid())
    )
  );
```

Fuente: supabase/migrations/004_rls_policies.sql.

7.3. Almacenamiento de archivos

Tabla 29. *Buckets de Supabase Storage*

Bucket	Acceso	Tamaño máximo	Tipos permitidos	Políticas
avatars	Público	5 MB	JPEG, PNG, WebP, GIF	Lectura pública; subir, actualizar y borrar en la carpeta propia
listings	Público	10 MB	JPEG, PNG, WebP	Lectura pública; subir, actualizar y borrar en la carpeta propia
messages	Privado	10 MB	JPEG, PNG, WebP	Lectura: quien la subió y quienes conversan con esa persona (020; antes, cualquier usuario autenticado). Subida en la carpeta propia. Se muestran con URL firmadas de 1 hora

Fuente: supabase/migrations/005_storage.sql, 020_politicas_seguridad.sql y src/services/chat.service.ts.

7.4. Refuerzos de seguridad

El análisis de la primera versión de este anexo identificó tres aspectos de seguridad mejorables. La migración 020 los corrige y la 021 añade el bloqueo de usuarios:

Tabla 30. *Refuerzos de seguridad aplicados en las migraciones 020 y 021*

Aspecto	Situación anterior	Corrección	Estado
Correo de los perfiles	profiles_select_all permitía leer la columna email con la clave pública	El correo ya no se guarda en profiles; se vacía en los perfiles existentes y la app lo toma de Supabase Auth	Aplicada · verificada en producción
Inserción de notificaciones	WITH CHECK (TRUE): cualquiera podía crear avisos a nombre de otro	Solo para uno mismo o para alguien con quien se tiene una propuesta de trueque	Aplicada · verificada en producción
Inserción de logros	WITH CHECK (TRUE)	Solo auth.uid() = user_id	Aplicada · verificada en producción

Aspecto	Situación anterior	Corrección	Estado
Fotos del chat	Cualquier usuario autenticado podía leer el bucket messages	Solo quien subió la foto y quienes conversan con esa persona	Aplicada · verificada en producción
Contacto no deseado	Solo existía el reporte	Bloqueo: sin mensajes ni propuestas entre personas bloqueadas (políticas con is_blocked_between)	Aplicada · verificada en producción

Fuente: supabase/migrations/020_politicas_seguridad.sql y 021_bloqueo_usuarios.sql; estado verificado en producción el 4/11/2026.

8. Evidencias visuales de la aplicación

Las 30 capturas de esta sección se obtuvieron el 4 de noviembre de 2026 de la aplicación publicada en https://ecotrueque-venezuela-ten.vercel.app, ejecutada en Google Chrome con una vista móvil de 412 × 892 píxeles e iniciando sesión con la cuenta de prueba de la autora Norlys Castañeda, después de publicar las mejoras del 4 de noviembre de 2026 (commit 14df813). Las capturas no fueron retocadas ni se ocultó información; durante la sesión se bloquearon las operaciones de escritura para no alterar los datos. Todas corresponden a la versión web; no se incluyen capturas de un dispositivo Android o iOS físico (ver sección 15).

Tabla 31. *Relación de evidencias visuales*

N.º	Pantalla	Qué demuestra	Funcionalidad	Ruta
Figura 8	Inicio de sesión	Formulario de correo y contraseña con validación, acceso a recuperación de contraseña, identificación institucional (UVM, autores y tutora) y CO_2 estimado de la comunidad calculado con los datos reales (39 kg).	Autenticación · Supabase Auth	/login
Figura 9	Registro de usuario (paso 1 de 2)	Captura de nombre, correo, teléfono opcional, estado y ciudad; el selector de estado ya se muestra correctamente en la web.	Registro de usuarios	/register
Figura 10	Recuperación de contraseña	Envío de enlace de restablecimiento al correo del usuario.	Autenticación · resetPasswordForEmail	/forgot-password
Figura 11	Pantalla principal (Explorar)	Ofertas cercanas con foto, distancia aproximada, lo que se busca a cambio y CO_2 estimado; categorías y saludo personalizado. El banner muestra el impacto real de la comunidad (39 kg de CO_2 estimados).	Listado de publicaciones · get_nearby_listings	/
Figura 12	Búsqueda por texto	Resultados de la búsqueda «guitarra» (texto completo en español y coincidencia por título).	Búsqueda · search_vector / ILIKE	/
Figura 13	Filtro por categoría	Listado filtrado por la categoría Electrónica.	Filtros · p_category	/
Figura 14	Detalle de publicación	Fotografías, tipo y categoría, estado del artículo, lo que se busca a cambio, impacto estimado (25 kg CO_2, 10 kg residuos, USD 60), descripción, etiquetas, ubicación y acciones Chat y Proponer trueque.	Publicaciones · eco_impact	/listing/[id]
Figura 15	Mapa de trueques	Ofertas alrededor de Valera con radios de 5, 10, 20 y 50 km; 30 trueques en 20 km. Los marcadores se ubican en el centro de una cuadrícula de unos 550 m y la lista muestra distancias aproximadas. Mapa Leaflet con datos de OpenStreetMap.	Geolocalización · PostGIS ST_DWithin	/map
Figura 16	Mapa con sitios seguros sugeridos	Botón «Sitios seguros (8)» activo: escudos azules con los lugares públicos de Valera cargados por la migración 022, junto a las ofertas.	Geolocalización · safe_spots, get_nearby_safe_spots	/map?seguros=1
Figura 17	Propuesta de trueque	Selección de una publicación propia para ofrecer, mensaje opcional, impacto estimado del intercambio y acceso a los lugares públicos sugeridos para el encuentro.	Gestión de solicitudes · trade_requests	/trade/request

N.º	Pantalla	Qué demuestra	Funcionalidad	Ruta
Figura 18	Bandeja de mensajes	Conversaciones del usuario con el estado de cada propuesta (Aceptado, Completado) y la publicación asociada.	Comunicación · conversations	/chat
Figura 19	Conversación (mensajes de texto)	Propuesta registrada dentro del chat, mensajes con fecha, hora y estado de lectura.	Comunicación · messages, Realtime	/chat/[id]
Figura 20	Conversación (fotografías)	Fotografías enviadas en el chat, almacenadas en el bucket privado messages y mostradas mediante URL firmadas temporales.	Comunicación · Supabase Storage	/chat/[id]
Figura 21	Menú de opciones del chat	Opciones Ver perfil, Ver publicación, Reportar usuario y Bloquear usuario.	Comunicación y seguridad · reports	/chat/[id]
Figura 22	Confirmación de bloqueo	Explica los efectos del bloqueo antes de confirmarlo; puede revertirse desde el mismo menú.	Seguridad · user_blocks	/chat/[id]
Figura 23	Solicitudes recibidas	Solicitudes con filtros Todas, Pendientes y Aceptadas; lo que se ofrece y lo que se quiere, mensaje y estado Completado con acción Dejar reseña.	Gestión de solicitudes · trade_requests	/trade-requests
Figura 24	Notificaciones	Avisos de trueque aceptado, nuevas propuestas y nuevas reseñas.	Notificaciones internas · notifications	/notifications
Figura 25	Perfil de usuario	Nivel Brote, 2 trueques, reputación 5.0, 63 publicaciones activas y 286 EcoPoints; pestaña Publicaciones con Editar y Eliminar.	Perfil · profiles, listings	/profile
Figura 26	Perfil: historial de intercambios	Trueques completados con el CO_2 estimado de cada intercambio.	Historial · trades	/profile?tab=Historial
Figura 27	Perfil: reseñas recibidas	Calificaciones de 5 estrellas con comentario de otros usuarios.	Reputación · reviews	/profile?tab=Reseñas
Figura 28	Editar perfil	Foto, nombre, biografía, teléfono, ciudad y estado.	Perfil · profiles, bucket avatars	/edit-profile
Figura 29	Publicar oferta (paso 1 de 4)	Elección entre bien o servicio y las 15 categorías disponibles.	Publicación de bienes o servicios	/publish
Figura 30	Panel de impacto ecológico	Nivel Brote con 286 EcoPoints (136 por trueques y 150 por logros), 39 kg de CO_2 y 6 kg de residuos estimados, USD 30 de ahorro aproximado y 2 trueques completados.	Métricas e indicadores · eco.service	/eco/dashboard
Figura 31	Panel: distribución, metas y huella	Distribución del impacto, metas ecológicas con progreso y analogías («Tu huella en el planeta»).	Métricas e indicadores	/eco/dashboard
Figura 32	Panel: logros	Los 16 logros del catálogo (tabla achievements) con su recompensa y progreso; Primera Reseña, Primera Semilla y Nivel Brote aparecen desbloqueados.	Gamificación · achievements, user_achievements	/eco/dashboard
Figura 33	Perfil público de otro usuario	Reputación, EcoPoints, publicaciones y reseñas de Rhonny Jaimes.	Perfil y reputación	/user/[id]
Figura 34	Ayuda y soporte	Preguntas frecuentes por tema con buscador.	Soporte al usuario	/help
Figura 35	Acerca de EcoTrueque	Propósito y funcionamiento; muestra «Versión 1.0.0».	Identificación de versión	/about
Figura 36	Pantalla principal en modo oscuro	Interfaz adaptada al tema oscuro del dispositivo.	Adaptación visual	/
Figura 37	Panel de impacto en modo oscuro	Panel de métricas con el tema oscuro.	Adaptación visual	/eco/dashboard

Fuente: capturas del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.



Figura 8. *Inicio de sesión*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 9. *Registro de usuario (paso 1 de 2)*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 10. *Recuperación de contraseña*
Fuente: captura del sistema EcoTrueque Venezuela.

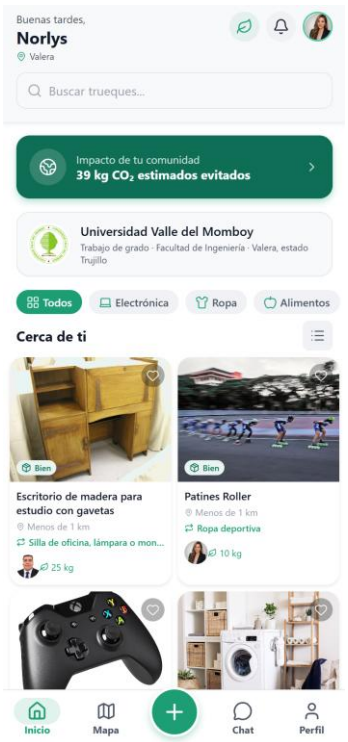


Figura 11. *Pantalla principal (Explorar)*
Fuente: captura del sistema EcoTrueque Venezuela.

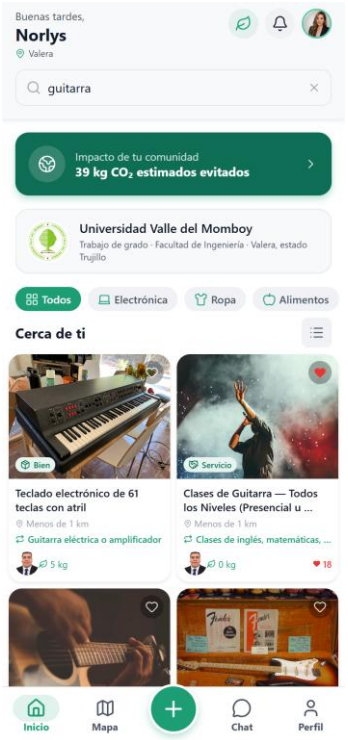


Figura 12. *Búsqueda por texto*
Fuente: captura del sistema EcoTrueque Venezuela.

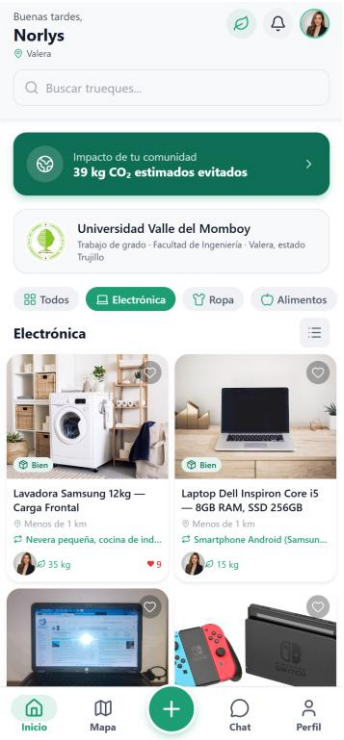


Figura 13. *Filtro por categoría*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 14. *Detalle de publicación*
Fuente: captura del sistema EcoTrueque Venezuela.

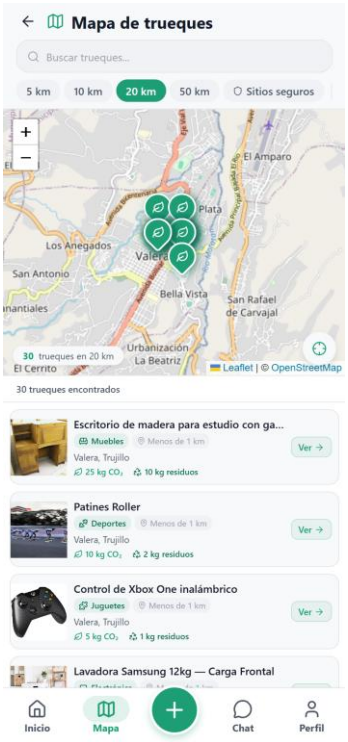


Figura 15. *Mapa de trueques*
Fuente: captura del sistema EcoTrueque Venezuela.

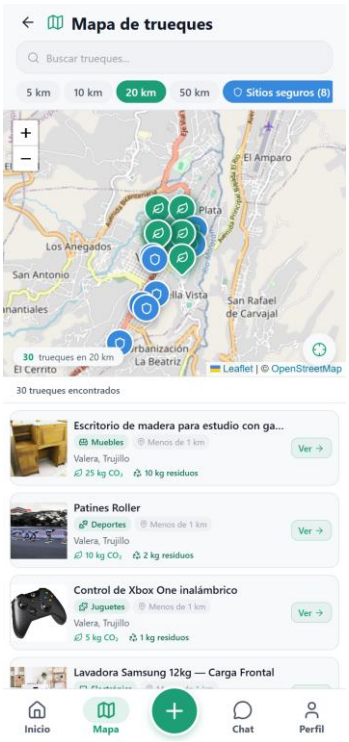


Figura 16. *Mapa con sitios seguros sugeridos*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 17. *Propuesta de trueque*
Fuente: captura del sistema EcoTrueque Venezuela.

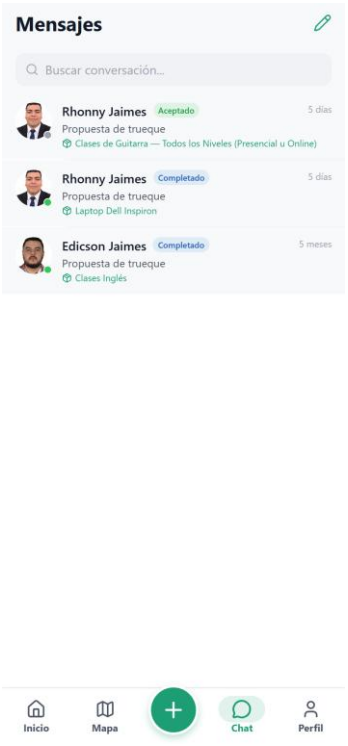


Figura 18. *Bandeja de mensajes*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 19. *Conversación (mensajes de texto)*
Fuente: captura del sistema EcoTrueque Venezuela.

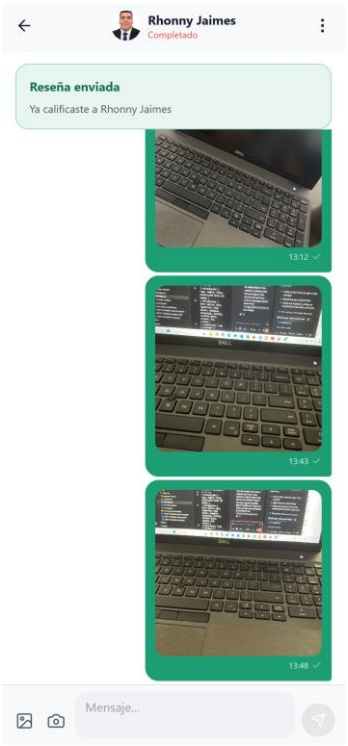


Figura 20. *Conversación (fotografías)*
Fuente: captura del sistema EcoTrueque Venezuela.

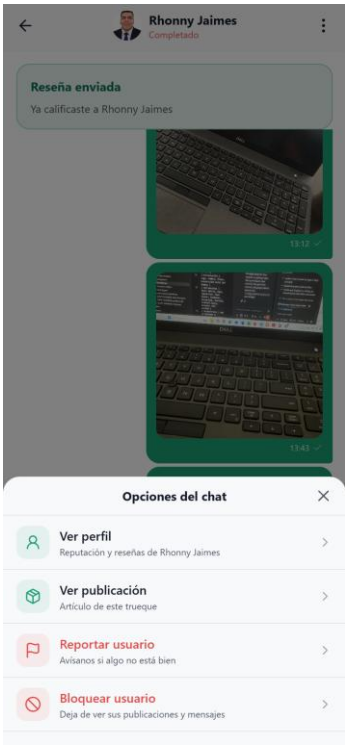


Figura 21. *Menú de opciones del chat*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 22. *Confirmación de bloqueo*
Fuente: captura del sistema EcoTrueque Venezuela.

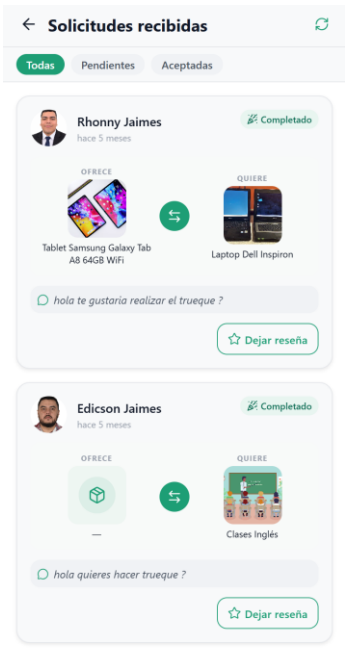


Figura 23. *Solicitudes recibidas*
Fuente: captura del sistema EcoTrueque Venezuela.

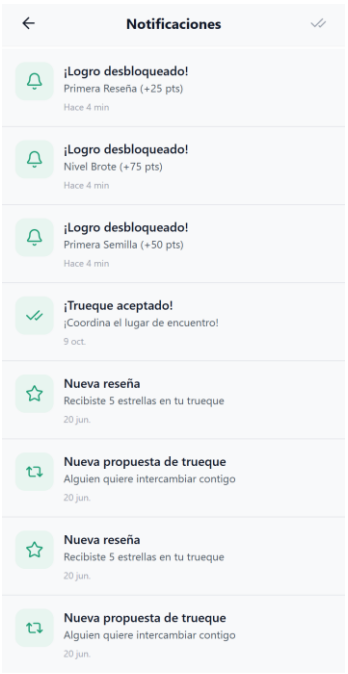


Figura 24. *Notificaciones*
Fuente: captura del sistema EcoTrueque Venezuela.

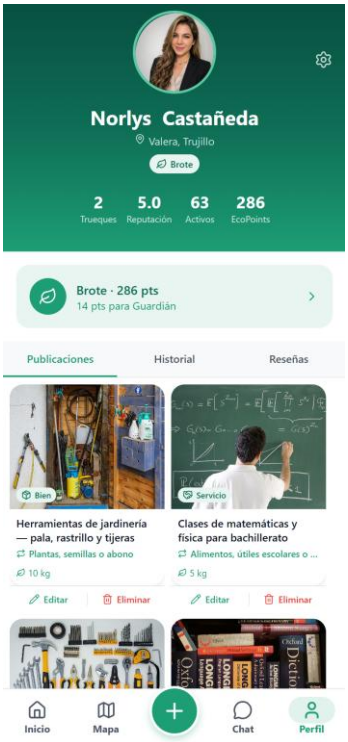


Figura 25. *Perfil de usuario*
Fuente: captura del sistema EcoTrueque Venezuela.

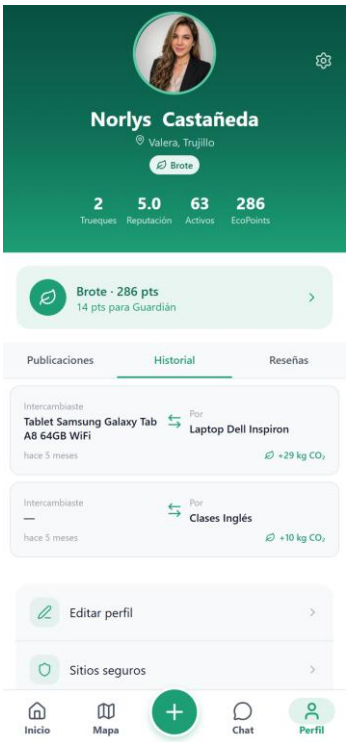


Figura 26. *Perfil: historial de intercambios*
Fuente: captura del sistema EcoTrueque Venezuela.

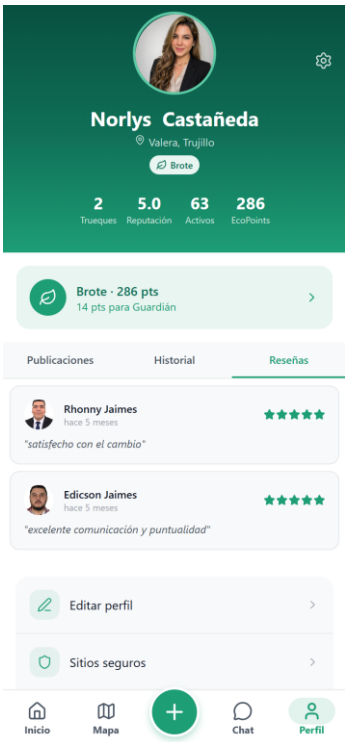


Figura 27. *Perfil: reseñas recibidas*
Fuente: captura del sistema EcoTrueque Venezuela.

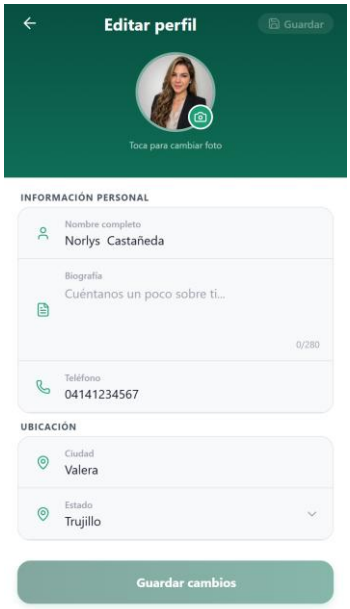


Figura 28. *Editar perfil*
Fuente: captura del sistema EcoTrueque Venezuela.

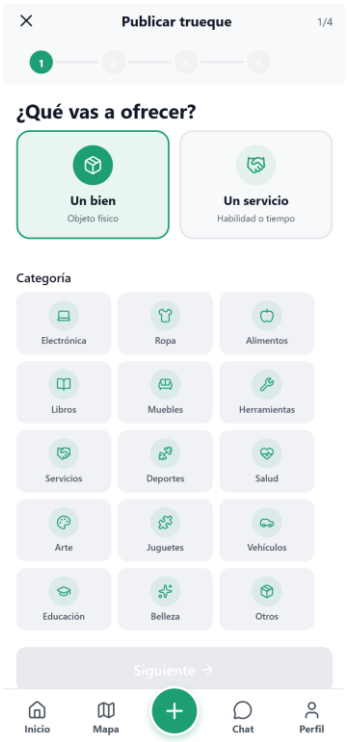


Figura 29. *Publicar oferta (paso 1 de 4)*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 30. *Panel de impacto ecológico*
Fuente: captura del sistema EcoTrueque Venezuela.

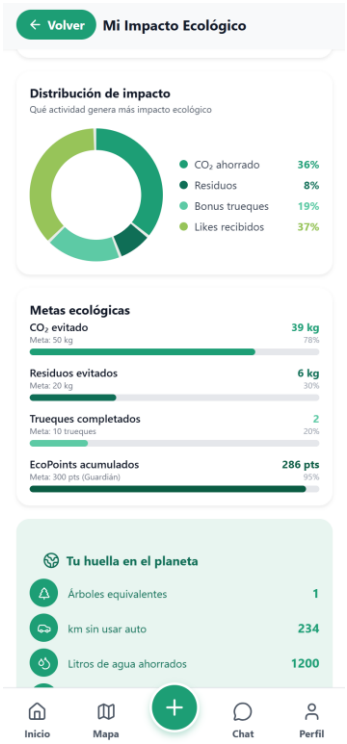


Figura 31. *Panel: distribución, metas y huella*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 32. *Panel: logros*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 33. *Perfil público de otro usuario*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 34. *Ayuda y soporte*
Fuente: captura del sistema EcoTrueque Venezuela.



Figura 35. *Acerca de EcoTrueque*
Fuente: captura del sistema EcoTrueque Venezuela.

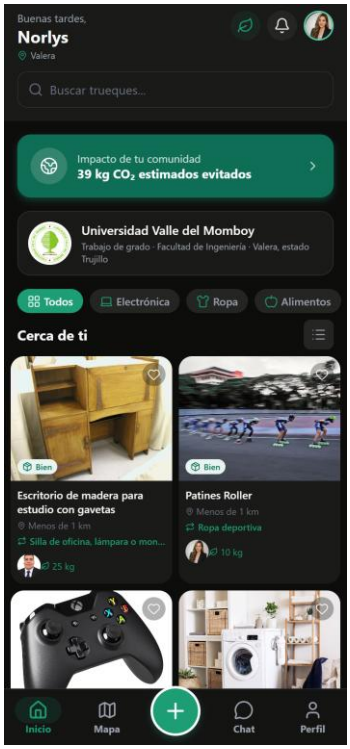


Figura 36. *Pantalla principal en modo oscuro*
Fuente: captura del sistema EcoTrueque Venezuela.

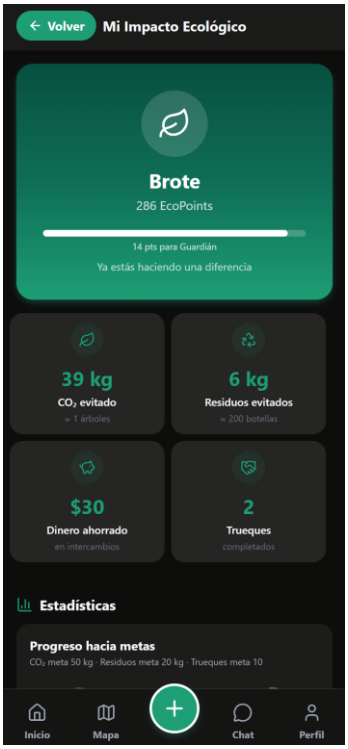


Figura 37. *Panel de impacto en modo oscuro*
Fuente: captura del sistema EcoTrueque Venezuela.

8.1. Logros registrados de la cuenta de prueba

Al abrir el panel de impacto con la cuenta de Norlys Castañeda, sin interceptar escrituras, la aplicación registró en user_achievements los tres logros alcanzados (Primera Semilla, Nivel Brote y Primera Reseña), sumó sus recompensas (50 + 75 + 25 = 150) a los 136 EcoPoints obtenidos por trueques y creó una notificación por cada logro. La consulta posterior a la base de datos confirma 3 logros registrados y 286 EcoPoints.



Figura 38. *Panel de impacto con 286 EcoPoints*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.



Figura 39. *Logros desbloqueados*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

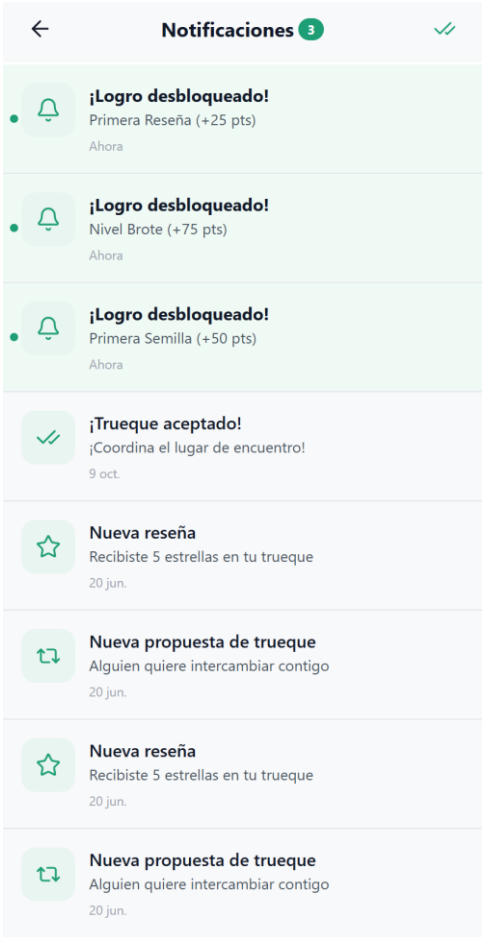


Figura 40. *Notificaciones «¡Logro desbloqueado!»*
Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

8.2. Panel de administración

El panel se probó con un reporte real creado para este fin: Norlys Castañeda reportó a Rhonny Jaimes desde el chat con el motivo «Otro motivo» y la descripción «Reporte de prueba del panel de administración (trabajo de grado). No corresponde a un incidente real.». Luego, como administradora, lo pasó a revisión y lo resolvió con una nota. No se pausó ninguna publicación real (esa acción se verificó con escritura interceptada en la prueba CP-34).



Figura 41. *Envío del reporte de prueba desde el chat*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

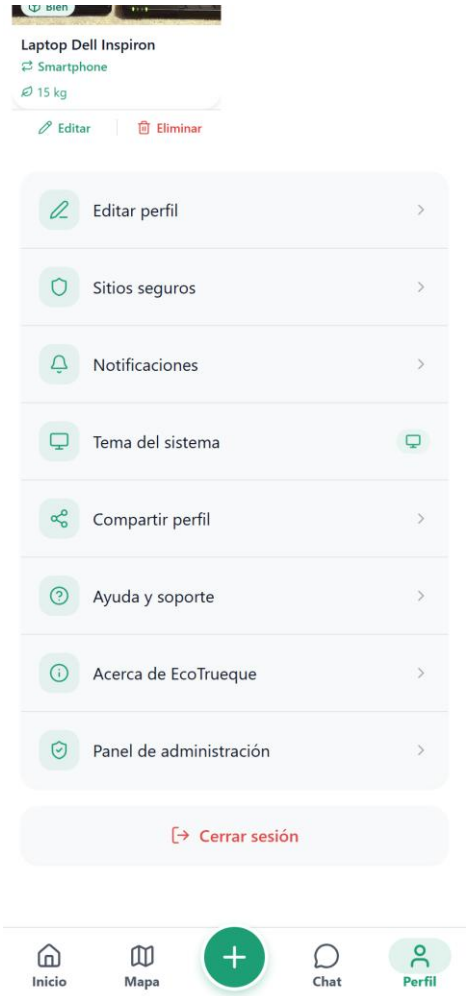


Figura 42. *Opción «Panel de administración» en el perfil*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.



Figura 43. *Panel con el reporte pendiente*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

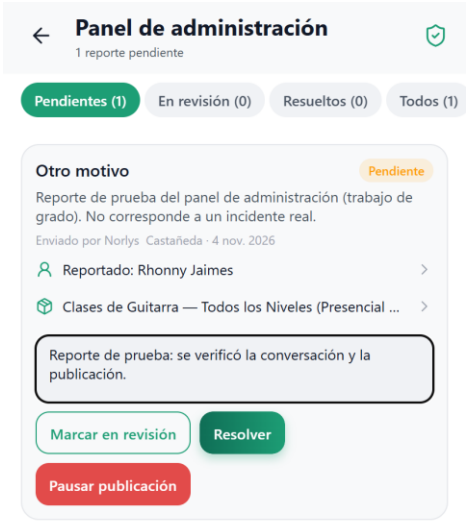


Figura 44. *Detalle del reporte, nota y acciones*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.



Figura 45. *Reporte resuelto con su nota*

Fuente: captura del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

8.3. Representación en dispositivos Android e iOS

Las Figuras 46 y 47 son representaciones ilustrativas, con fines demostrativos y documentales, de la aplicación en teléfonos Android e iOS. Las pantallas son capturas reales de la versión web publicada (las mismas de la sección 8), colocadas dentro de un marco de dispositivo. **No son capturas tomadas en un teléfono físico**: muestran la misma interfaz que se ejecuta en Android e iOS, porque EcoTrueque usa un único código React Native, pero no constituyen evidencia de ejecución nativa.

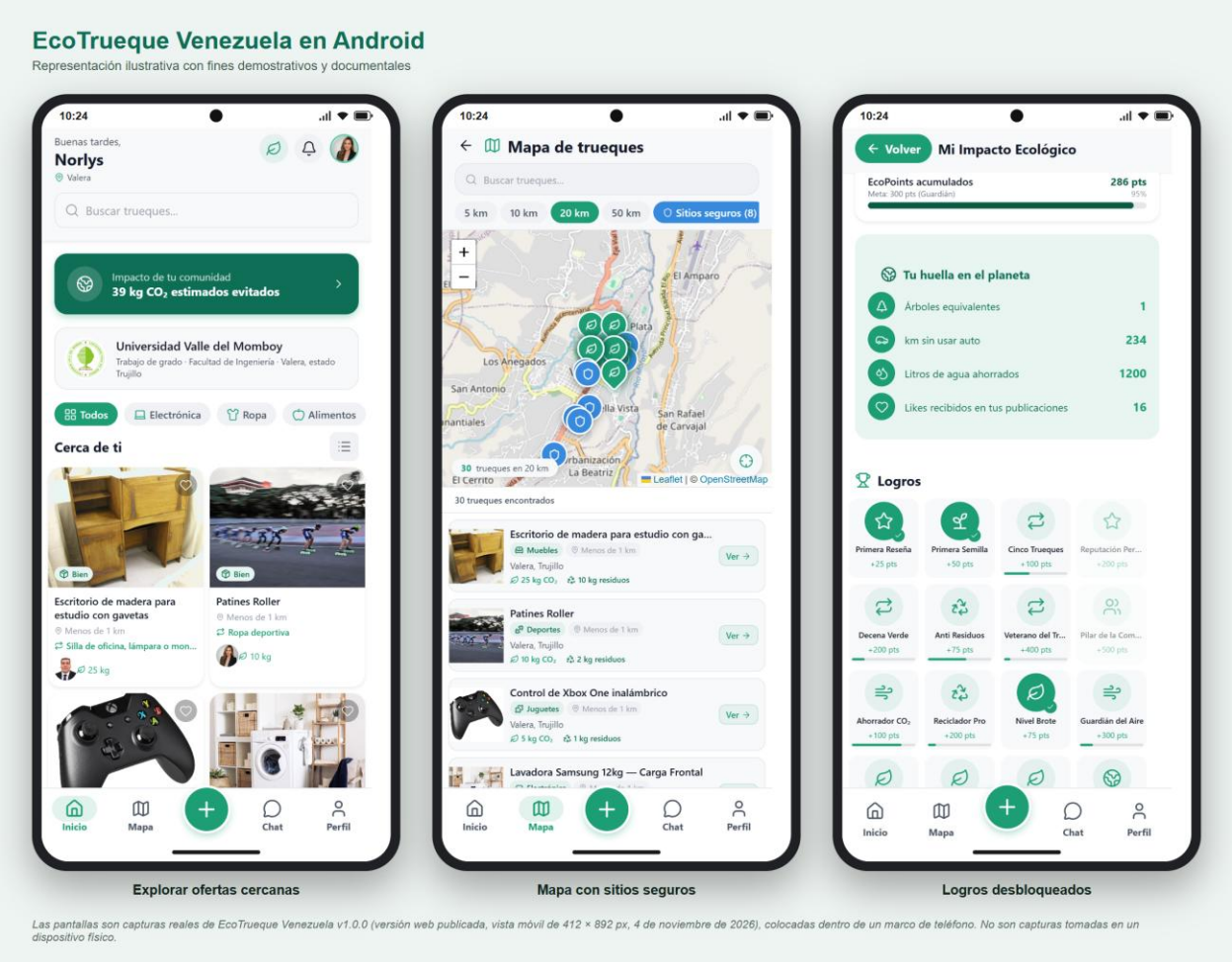


Figura 46. *Representación ilustrativa de EcoTrueque Venezuela en Android*

Fuente: elaboración propia con capturas reales de la versión web del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

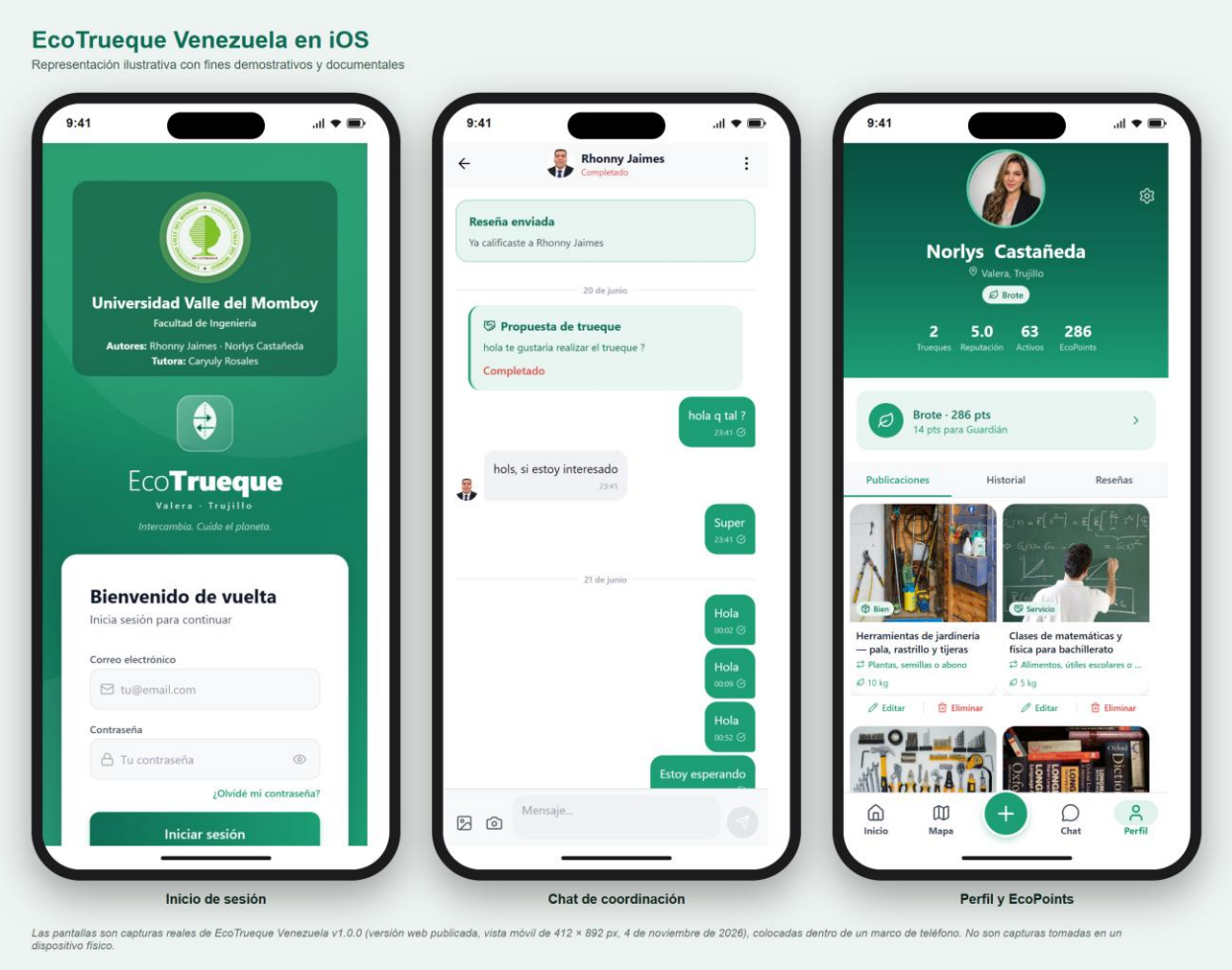


Figura 47. *Representación ilustrativa de EcoTrueque Venezuela en iOS*

Fuente: elaboración propia con capturas reales de la versión web del sistema EcoTrueque Venezuela, 4 de noviembre de 2026.

9. Identificación de versión

El proyecto declara la versión **1.0.0** en tres lugares verificables: package.json ("version": "1.0.0"), app.config.ts (version: '1.0.0', versionCode 1 para Android) y la pantalla «Acerca de EcoTrueque» de la aplicación publicada (Figura 35). Se adopta, por tanto, la nomenclatura **EcoTrueque Venezuela v1.0.0 (MVP)**, asociada al commit 14df813 de la rama main. En el video de demostración la versión se rotula como «V1.001», que corresponde a esta misma entrega.

Tabla 32. *Ficha técnica de la versión*

Nombre del producto	EcoTrueque Venezuela
Versión actual verificable	1.0.0 (MVP) — package.json, app.config.ts, pantalla Acerca de y etiqueta Git v1.0.0 (commit 5224639)
Identificador de código	Commit 14df813e614f1b411917c03822f0424bdb304ab8 (rama main, 2026-11-04 18:11:05 -0500)
Fecha de documentación	4 de noviembre de 2026
Tecnologías principales	React Native 0.81.5, Expo SDK 54, TypeScript 5.6, Expo Router 6, Supabase (PostgreSQL + PostGIS)
Estado de desarrollo	Producto mínimo viable funcional, publicado en la web; en fase de validación académica. Migraciones 015, 020 y 021: aplicadas
Entorno de ejecución	Web: https://ecotrueque-venezuela-ten.vercel.app (respuesta HTTP 200 servida por Vercel, verificada el 4 de noviembre de 2026). Móvil: ejecución con Expo Go mediante npx expo start; no se verificó la publicación en Google Play ni App Store
Base de datos	PostgreSQL gestionado por Supabase, con PostGIS, pg_trgm, unaccent y uuid-ossp; 13 tablas en producción (14 con la migración 021)
Datos en producción (visibles)	10 perfiles, 99 publicaciones (88 bienes y 11 servicios), 16 logros en el catálogo, 3 logros registrados por la cuenta de prueba, 28 sitios seguros (8 en Valera), 4 reseñas

Fuente: package.json, app.config.ts, historial Git y verificación de la aplicación publicada.

9.1. Estado de las funcionalidades

Tabla 33. *Funcionalidades implementadas, parciales y pendientes*

Estado	Funcionalidad	Evidencia
Implementada	Registro, inicio de sesión y recuperación de contraseña	Figuras 8–10; auth.service.ts
Implementada	Exploración de ofertas cercanas, búsqueda por texto y filtro por categoría	Figuras 11–13; get_nearby_listings
Implementada	Detalle de publicación y «Me gusta»	Figura 14; tabla favorites
Implementada	Mapa con radio de búsqueda (PostGIS), ubicación y distancia aproximadas	Figura 15; utils/location.ts
Implementada	Publicación en 4 pasos, edición y borrado lógico	Figuras 29 y 25; delete_own_listing
Implementada	Propuesta de trueque, solicitudes, aceptación o rechazo y confirmación doble	Figuras 17 y 23; trades.service.ts, complete_trade
Implementada	Chat en tiempo real con fotos y menú de opciones (ver perfil, ver publicación, reportar)	Figuras 18–21

Estado	Funcionalidad	Evidencia
Implementada	Reseñas de 1 a 5 estrellas y reputación	Figuras 27 y 33; trigger de reputación
Implementada	Notificaciones internas	Figura 24
Implementada	Perfil, historial, edición de perfil, ayuda y acerca de	Figuras 25–28, 34 y 35
Implementada	Panel de impacto: EcoPoints, niveles, metas, distribución, analogías e impacto real de la comunidad	Figuras 30–32; utils/eco.ts
Implementada	Logros: se registran en user_achievements al abrir el panel o completar un trueque y suman su recompensa	Figura 32; eco.service.ts; pruebas en src/tests/eco.test.ts
Implementada	Modo claro u oscuro, elegido en el perfil o según el dispositivo (también en la web)	Figuras 36 y 37; commit b174f03
Implementada	Sitios seguros sugeridos en el mapa: 8 lugares públicos de Valera tomados de OpenStreetMap; el punto de encuentro lo acuerdan las dos personas	Figura 17; prueba CP-13; migración 022 (aplicada · verificada en producción)
Implementada	Bloqueo y desbloqueo de usuarios desde el chat	Figuras 21 y 22; migración 021 (aplicada · verificada en producción)
Implementada	Contador de mensajes no leídos (RPC corregida)	Migración 015 (aplicada · verificada en producción); la bandeja ya calcula los no leídos en el cliente
Implementada	Panel de administración: reportes, notas, pausa de publicaciones y avisos	Sección 8.2; pruebas CP-31 a CP-35; migración 024
Implementada (corte)	Indicadores M1–M7 del trabajo de grado	Corte global al 4 de noviembre de 2026 (sección 9.2) con la consulta supabase/diagnostics/06_indicadores_m1_m7.sql
Pendiente	Notificaciones push	Existe la columna push_token y la dependencia expo-notifications, pero no hay envío; supabase/functions/send-notification está vacía
Pendiente	Motor de matching como Edge Function y suspensión de cuentas	supabase/functions/matching-engine está vacía (el matching funciona como RPC get_trade_matches); la suspensión de cuentas queda para una etapa posterior
Pendiente	Publicación en tiendas de aplicaciones	Existe eas.json, sin evidencia de compilaciones publicadas

Fuente: análisis del código fuente, de la base de datos y de las capturas.

9.2. Corte de datos de los indicadores M1–M7

La Tabla 34 presenta el corte de los indicadores definidos en la Tabla 32 del trabajo de grado. Los valores provienen de la consulta supabase/diagnostics/06_indicadores_m1_m7.sql, ejecutada por los autores en el SQL Editor de Supabase el 4 de noviembre de 2026 con acceso de administrador, por lo que cubren toda la plataforma (Figura 48). El período va del 1 de junio al 4 de noviembre de 2026.

Los datos corresponden a las cuentas de prueba del MVP (los autores y usuarios de demostración): demuestran que el sistema registra y calcula los indicadores, pero no representan el comportamiento de una comunidad de usuarios de Trujillo. Sus valores operacionales se obtendrán con el uso real de la aplicación o con una prueba piloto.

Tabla 34. *Corte de los indicadores M1–M7 al 4 de noviembre de 2026 (toda la plataforma)*

Indicador	Definición (Tabla 32)	Valor	Tipo y lectura
M1 Intercambios completados	Conteo de solicitudes cerradas	2	Registro directo: trueques cerrados en la tabla trades
M2 Bienes reutilizados	Bienes presentes en trueques cerrados	2	Registro directo: Tablet Samsung Galaxy Tab A8 y Laptop Dell Inspiron
M3 Servicios gestionados	Servicios en acuerdos cerrados	2	Registro directo: «Clases Inglés» y un servicio de San Cristóbal cuya publicación fue eliminada después del trueque
M4 Tasa de conversión	Cierres ÷ solicitudes válidas (por período)	50,0 %	Indicador calculado: 2 trueques cerrados de 4 solicitudes válidas (se excluyen las canceladas)
M5 Compras potencialmente evitadas	Declaración o supuesto de sustitución	2	Estimación condicionada: supone que cada bien recibido sustituye una compra
M6 Ahorro económico aproximado	Valor declarado o referencial	USD 600	Estimación condicionada: suma del valor declarado de los 4 artículos intercambiados
M7 Distribución por categoría y zona	Agrupación por categoría y municipio	Electrónica · Valera: 2; Educación · Valera: 1; Servicios · San Cristóbal: 1	Registro directo
Complementario: impacto estimado	CO_2 y residuos de los trueques cerrados	39 kg CO_2 · 6 kg residuos	Recurso educativo: factores promedio por categoría (no es una medición)

Fuente: resultado de la consulta 06_indicadores_m1_m7.sql en el SQL Editor de Supabase, ejecutada por los autores el 4 de noviembre de 2026.



Figura 48. *Resultado de la consulta de indicadores M1–M7 en Supabase*

Fuente: SQL Editor del panel de Supabase, captura de los autores, 4 de noviembre de 2026.

10. Repositorio y estructura del código fuente

Tabla 35. *Datos del repositorio y control de versiones*

Plataforma	GitHub
URL	https://github.com/Norlysc/ecotrueque-venezuela
Rama principal	main (sincronizada con origin/main)
Total de commits	45 (del 14/07/2026 al 04/11/2026)
Primer commit	3438cc6 · 14/07/2026 · «ecoTrueque»
Último commit verificable	14df813 · 2026-11-04 18:11:05 -0500 · «Corregir títulos de notificaciones recortados»
Hash completo del último commit	14df813e614f1b411917c03822f0424bdb304ab8
Autoría registrada	Norlys (castaneda30@hotmail.com), 45 commits; 38 de ellos registran la línea Co-Authored-By de la herramienta de asistencia de programación utilizada (Claude)
Archivos versionados	140 (81 dentro de src/)
Etiquetas (tags)	v1.0.0 (anotada) → commit 5224639; publicada en GitHub
Visibilidad	Público: se puede consultar y clonar sin iniciar sesión (verificado con la API de GitHub el 4/11/2026)
Cambios locales sin confirmar	Ninguno: todo el código está confirmado y publicado en GitHub
Despliegue continuo	Vercel compila la rama main con npx expo export --platform web (vercel.json)

Fuente: git log, git status y git remote del repositorio local, 4 de noviembre de 2026.

10.1. Historial de commits

Tabla 36. *Historial completo de commits de la rama main*

Commit	Fecha	Mensaje
14df813	04/11/2026	Corregir títulos de notificaciones recortados
bae9565	04/11/2026	Agregar pruebas del panel de administración y documentarlo en el README
fca2a46	04/11/2026	Agregar el panel de administración de reportes
d2deed0	04/11/2026	Agregar el rol de administrador y las funciones del panel de reportes
3c246ed	04/11/2026	Agregar migración para eliminar la tabla accidental v_user_id
5224639	04/11/2026	Agregar consultas de inventario del esquema y de los indicadores M1–M7
a60bb61	04/11/2026	Mostrar en el mapa los sitios seguros sugeridos de Valera
da07680	04/11/2026	Importar las funciones de Jest en las pruebas para que pasen la revisión de tipos
8331437	04/11/2026	Agregar las migraciones 015 corregida, 020 de seguridad y 021 de bloqueo
dcf34c9	04/11/2026	Actualizar el README con las nuevas migraciones, pruebas y limitaciones
c240c5c	04/11/2026	Agregar pruebas automatizadas con Jest para EcoPoints, logros, ubicación y formatos
b06d0d8	04/11/2026	Permitir bloquear y desbloquear usuarios desde el chat
0ad3ec1	04/11/2026	Mostrar la ubicación aproximada de las publicaciones
2e1cec0	04/11/2026	Mostrar el impacto real de la comunidad y guardar los logros alcanzados

Commit	Fecha	Mensaje
fb2ff54	04/11/2026	Corregir el selector de estado del registro en la web y los tipos de BottomSheetProvider
8116f81	04/11/2026	Ordenar las migraciones SQL y agregar seguridad y bloqueo de usuarios
a6481c8	04/11/2026	Reescribir el README como guía técnica de la aplicación
b174f03	04/11/2026	Hacer funcionar el selector de modo claro y oscuro también en la web
63dfca5	30/10/2026	Dar función al menú de tres puntos del chat
bd0718c	30/10/2026	Permitir usar la cámara del chat también en la versión web
da7d735	30/10/2026	Mostrar las fotos del chat con enlaces firmados temporales
5169f47	09/10/2026	Arreglar el cambio de foto de perfil
9fe3f87	09/10/2026	Engranaje del perfil funcional, botón Volver y barra inferior más visible
cb9f726	09/10/2026	Arreglar la navegación de las notificaciones
9a43a19	09/10/2026	Agregar script con 12 artículos de demostración para Rhonny Jaimes en Valera
f1ee178	09/10/2026	Explorar desde Valera cuando el GPS está lejos y logo en Ayuda y soporte
9dbe022	09/10/2026	Ubicar la app en Valera, estado Trujillo y completar 4 productos por categoría en Valera
64d0481	09/10/2026	Quitar las 3 fotos rotas sin importar el dueño de la publicación
e0764a2	09/10/2026	Agregar script con fotos y productos de demostración por categoría
4799a06	04/10/2026	Fondo de marca con degradado y círculos en las pantallas de acceso
69d7e8b	04/10/2026	Destacar a la universidad arriba en las pantallas de acceso, con animación
2dc5373	04/10/2026	Agregar pie con logo de la universidad, facultad y autores en las pantallas de acceso
8fb5d96	04/10/2026	Agregar logo de la Universidad Valle del Momboy en la página principal
0d9cfc1	04/10/2026	Reemplazar emojis por íconos profesionales en verde
c86d4c8	04/10/2026	Agregar página de Ayuda y soporte
99cc7fa	04/10/2026	Mejorar el diseño en escritorio
790a0c0	04/10/2026	cambios y ajustes para recuperacion de contraseña
4797775	04/10/2026	Diseño de escritorio para recuperar y restablecer contraseña
de9dbef	04/10/2026	Arreglar recuperación de contraseña en web y usar el logo como favicon
7824894	16/07/2026	ajustes en formulario para escritorio
a7c48af	14/07/2026	Fix web rendering and platform-specific bottom sheet provider
8d0e8e6	14/07/2026	Fix Expo web favicon for Vercel deployment
cc66446	14/07/2026	Configure web compatibility and deployment
8b82a06	14/07/2026	mvp
3438cc6	14/07/2026	ecoTrueque

Fuente: git log --format="%h|%ad|%s" de la rama main (mensajes transcritos sin modificar).

10.2. Estructura del proyecto

Tabla 37. *Estructura de carpetas del proyecto*

Ruta	Contenido
src/app/	Pantallas y rutas de Expo Router: (auth) acceso, (tabs) navegación inferior, y rutas de detalle, chat, intercambio, perfil, panel eco, ayuda y acerca de

Ruta	Contenido
src/components/	Componentes reutilizables: ui (16 componentes), listing (ListingCard), chat (ChatOptionsMenu)
src/services/	Acceso a Supabase por dominio: auth, listings, trades, chat, eco, reports, blocks, safeSpots
src/hooks/	Hooks de datos: useAuth, useListings, useTrades, useChat, useLocation, useColorScheme, useBlocks, useSafeSpots
src/utils/	Cálculos puros sin acceso a red: EcoPoints, equivalencias, impacto de la comunidad, logros, ubicación aproximada y formatos
src/tests/	Pruebas automatizadas con Jest (49 pruebas)
src/stores/	Estado global con Zustand: sesión, ubicación, avisos y tema
src/lib/	Cliente de Supabase (supabase.ts) y configuración de TanStack Query (queryClient.ts)
src/constants/	Tema visual, colores y las 15 categorías con sus factores de impacto
src/types/	Tipos TypeScript de la aplicación (app.types.ts)
supabase/migrations/	Migraciones SQL numeradas de 001 a 022: esquema, índices, funciones, RLS, almacenamiento, seguridad, bloqueo, sitios seguros y datos semilla
supabase/historico/	Consolidado de migraciones aplicado anteriormente (solo referencia)
supabase/diagnostics/	Diagnóstico del mapa y permisos, inventario del esquema (05) y corte de indicadores M1–M7 (06)
supabase/functions/	Carpetas de Edge Functions (matching-engine, send-notification) sin código
assets/	Íconos, logotipos de la aplicación y de la universidad, pantalla de inicio
scripts/	Scripts de apoyo (datos de demostración)
Raíz	package.json (incluye la configuración de Jest), app.config.ts, eas.json, vercel.json, tsconfig.json, babel.config.js, metro.config.js, .env.example, README.md y GUIA_INSTALACION.md

Fuente: árbol de directorios del repositorio.

10.3. Organización del código fuente

El código se organiza por capas: las pantallas (src/app) no acceden directamente a la base de datos, sino a través de hooks (src/hooks) que encapsulan consultas de TanStack Query, y estos llaman a los servicios (src/services), únicos responsables de comunicarse con Supabase. El estado de sesión, ubicación y tema reside en stores de Zustand. Los alias de importación (@hooks, @services, @stores, @constants) se configuran en tsconfig.json y babel.config.js.

10.4. Instalación y ejecución

1. Clonar el repositorio: `git clone https://github.com/Norlysc/ecotrueque-venezuela.git`.
2. Instalar dependencias con Node.js 20 o superior: `npm install`.
3. Crear el archivo .env a partir de .env.example con EXPO_PUBLIC_SUPABASE_URL y EXPO_PUBLIC_SUPABASE_ANON_KEY.

4. Ejecutar en Supabase los scripts SQL en el orden indicado en la sección 6.3.
5. Iniciar la aplicación: `npx expo start` (teclas w, a o i para web, Android o iOS) o `npx expo start --web`.
6. Ejecutar las pruebas: `npm test`.

10.5. Estrategia y entorno de pruebas

Las pruebas se organizaron en tres niveles: pruebas unitarias de la lógica de negocio, pruebas funcionales de cada funcionalidad sobre la aplicación publicada y pruebas no funcionales de rendimiento, compatibilidad, accesibilidad y seguridad. La Tabla 38 resume el plan; cada nivel tiene su registro en las secciones 10.6 a 10.8, y los defectos encontrados se registran en la sección 10.9.

Tabla 38. *Plan de pruebas por nivel*

Nivel	Objetivo	Herramienta	Entorno	Criterio de aceptación
Unitarias	Verificar los cálculos y reglas de negocio	Jest 29 + jest-expo	Equipo de desarrollo (Node.js)	Todas las pruebas aprobadas y verificación de tipos sin errores
Funcionales	Verificar cada funcionalidad de punta a punta en la interfaz	Playwright con Google Chrome	Aplicación publicada; escrituras interceptadas	Resultado obtenido igual al esperado en cada caso
Rendimiento	Medir tiempos de carga y de respuesta del backend	Playwright (Performance API) y solicitudes HTTP	Aplicación publicada y API de Supabase	LCP ≤ 2,5 s en red normal; p95 de la API ≤ 1 s
Compatibilidad	Verificar la interfaz en navegadores y tamaños de pantalla	Google Chrome y Microsoft Edge	5 resoluciones, de 360 a 1920 px	Contenido visible, sin desborde horizontal ni errores de JavaScript
Accesibilidad	Detectar barreras de acceso (WCAG 2.1 A y AA)	axe-core 4.10	7 pantallas principales	Sin violaciones críticas; las demás se registran como defectos
Seguridad	Comprobar que el acceso indebido se rechaza	Solicitudes HTTP a la API y al sitio	Producción, solo lecturas y accesos sin sesión	Todo acceso no autorizado devuelve vacío o error
Usabilidad	Medir la facilidad de uso percibida	Cuestionario SUS (Anexo F)	Usuarios de la comunidad	Puntaje SUS ≥ 68 (pendiente de aplicación)

Fuente: elaboración propia.

Entorno. Las pruebas se ejecutaron el 4 de noviembre de 2026 sobre la versión publicada en https://ecotrueque-venezuela-ten.vercel.app (commit 14df813) y su base de datos en Supabase, con la cuenta de prueba de la autora Norlys Castañeda. Los autores decidieron no crear una base de datos de pruebas separada; para no alterar los datos reales, las pruebas funcionales y no funcionales interceptaron toda operación de escritura y las pruebas de seguridad se limitaron a lecturas y a accesos sin sesión. Las escrituras reales se limitaron a dos flujos de demostración identificados en las secciones 8.1 y 8.2.

Ejecución. Las pruebas fueron ejecutadas por el equipo de desarrollo con scripts automatizados que dejan un registro en formato JSON y una captura de cada caso (carpeta evidencias/). Las pruebas pueden repetirse con los mismos scripts.

10.6. Pruebas unitarias

El proyecto incluye 49 pruebas unitarias con Jest (preset jest-expo) en src/tests/. Cubren la lógica de negocio extraída a src/utils/: la fórmula de EcoPoints (la misma de complete_trade, incluido el ejemplo laptop ⇄ bicicleta de 69 puntos), los umbrales de nivel, las analogías del panel, el impacto de la comunidad sin doble conteo, las reglas de los logros, la ubicación aproximada, los formatos numéricos y las reglas del panel de administración.

Tabla 39. *Registro de ejecución de las pruebas unitarias*

Archivo de prueba	Grupos	Pruebas	Resultado
src/tests/location-format.test.ts	Ubicación y distancia aproximadas, formato de cantidades	17	Aprobadas
src/tests/reports.test.ts	Panel de administración: conteo, filtro, orden y acciones	9	Aprobadas
src/tests/eco.test.ts	EcoPoints, niveles, analogías, impacto de la comunidad, logros	23	Aprobadas
Total	3 suites	49	Cobertura de src/utils: 95 % instr. · 88.46 % ramas · 100 % funciones

Fuente: salida de npx jest --verbose --coverage, 4 de noviembre de 2026.

La verificación de tipos con `npx tsc --noEmit` termina sin errores.

10.7. Pruebas funcionales

Se diseñaron y ejecutaron 35 casos de prueba funcional que recorren todas las funcionalidades de la aplicación, incluido el panel de administración. Cada caso realiza los pasos indicados en Google Chrome (vista móvil de 412 × 892 píxeles), comprueba el resultado esperado y guarda una captura en la que el elemento verificado aparece señalado con un recuadro rojo y el identificador del caso (Anexo G).

Las operaciones de escritura se interceptaron en el navegador: el caso verifica la solicitud que la aplicación envía a Supabase (método, tabla y datos) y la respuesta se simula sin guardar nada. En los casos CP-16 y CP-17 se mostró como «pendiente», solo en el navegador, una solicitud ya completada, porque la base de datos no tenía solicitudes pendientes; el estado guardado no cambió.

Tabla 40. *Resumen de la ejecución de las pruebas funcionales*

Casos	Aprobados	Bloqueados	Fallidos	Escrituras interceptadas	Errores de JavaScript
35	35	0	0	24	0

Fuente: evidencias/pruebas/resultados.json, 4 de noviembre de 2026.

Tabla 41. *Registro de casos de prueba funcional*

ID	Módulo	Caso y pasos	Resultado esperado	Resultado obtenido	Estado
CP-01 (Fig. 50)	Autenticación	**Inicio de sesión con credenciales válidas** 1. Abrir /login 2. Escribir el correo y la contraseña de la cuenta de prueba 3. Pulsar «Iniciar sesión»	Se abre la pantalla principal con el saludo de la usuaria	Sesión iniciada; URL /; saludo «Norlys»	Aprobado
CP-02 (Fig. 51)	Autenticación	**Validación de correo inválido al iniciar sesión** 1. Abrir /login 2. Escribir «correo-invalido» y una contraseña 3. Pulsar «Iniciar sesión»	Se muestra «Ingresa un email válido» y no se envía la solicitud	Mensaje «Ingresa un email válido» visible; la sesión no se inició	Aprobado
CP-03 (Fig. 52)	Autenticación	**Registro: paso 1 a paso 2** 1. Abrir /register 2. Completar nombre, correo, teléfono, estado (Trujillo) y ciudad 3. Pulsar «Continuar»	Se avanza al paso 2 de 2 (contraseña y términos)	Se muestra «Paso 2 de 2» con los campos de contraseña	Aprobado
CP-04 (Fig. 53)	Autenticación	**Registro: contraseñas que no coinciden** 1. En el paso 2 escribir «Clave1234» y confirmar con «Clave9999» 2. Aceptar términos 3. Pulsar «Crear cuenta»	Se muestra un error de coincidencia y no se crea la cuenta	Mensaje «Las contraseñas no coinciden»; no se envió el registro	Aprobado
CP-05 (Fig. 54)	Autenticación	**Recuperación de contraseña: validación del correo** 1. Abrir /forgot-password 2. Escribir «abc» 3. Pulsar «Enviar enlace de recuperación»	Se muestra «Ingresa un email válido»	Mensaje de validación visible; no se envió el correo	Aprobado
CP-06 (Fig. 55)	Publicaciones	**Listado de ofertas cercanas** 1. Abrir la pantalla principal con ubicación en Valera	Se listan ofertas con foto, distancia aproximada y lo que se busca a cambio	30 tarjetas con distancia «Menos de 1 km»	Aprobado
CP-07 (Fig. 56)	Publicaciones	**Búsqueda por texto** 1. Escribir «guitarra» en el buscador 2. Pulsar Enter	Los resultados contienen «Guitarra»	6 resultados con «Guitarra»	Aprobado
CP-08 (Fig. 57)	Publicaciones	**Filtro por categoría** 1. Volver a la pantalla principal 2. Pulsar la categoría «Electrónica»	El listado muestra el título «Electrónica»	Sección «Electrónica» con sus publicaciones	Aprobado
CP-09 (Fig. 58)	Publicaciones	**Detalle de publicación** 1. Abrir «Escritorio de madera para estudio con gavetas»	Se ven fotos, «Busca a cambio», impacto estimado y «Proponer trueque»	«Busca a cambio», impacto (25 kg CO_2, 10 kg residuos, USD 60) y botón «Proponer trueque» visibles	Aprobado
CP-10 (Fig. 59)	Publicaciones	**Marcar «Me gusta»** 1. En el detalle, pulsar el corazón	La app envía la solicitud a favorites con la publicación	POST favorites (listing_id 1e566dc3-9f20-4310-aa75-16a467704acd) (Escritura interceptada)	Aprobado

ID	Módulo	Caso y pasos	Resultado esperado	Resultado obtenido	Estado
CP-11 (Fig. 60)	Publicaciones	**Asistente para publicar (paso 1 a paso 2)** 1. Abrir «Publicar» 2. Elegir «Un bien» y la categoría «Libros» 3. Pulsar «Siguiente»	Se avanza al paso 2 de 4	Indicador «2/4» visible	Aprobado
CP-12 (Fig. 61)	Geolocalización	**Cambio del radio de búsqueda** 1. Abrir el mapa 2. Pulsar «5 km»	El contador indica «trueques en 5 km»	«30 trueques en 5 km»	Aprobado
CP-13 (Fig. 62)	Geolocalización	**Sitios seguros sugeridos en el mapa** 1. En el mapa, pulsar «Sitios seguros»	Se muestran los lugares públicos sugeridos de Valera	Botón «Sitios seguros (8)»	Aprobado
CP-14 (Fig. 63)	Intercambios	**Enviar una propuesta de trueque** 1. Abrir la propuesta para «Clases de Guitarra» 2. Elegir una publicación propia 3. Escribir un mensaje 4. Pulsar «Enviar propuesta de trueque»	La app crea la solicitud (trade_requests), la conversación y la notificación al dueño	Solicitudes enviadas: trade_requests, conversations, messages, notifications (Escrituras interceptadas)	Aprobado
CP-15 (Fig. 64)	Intercambios	**Solicitudes recibidas y filtros** 1. Abrir «Solicitudes recibidas» 2. Pulsar «Aceptadas» y luego «Todas»	Se listan las solicitudes con su estado	2 solicitudes «Completado» de Rhonny Jaimes y Edicson Jaimes	Aprobado
CP-16 (Fig. 66)	Intercambios	**Aceptar una solicitud recibida** 1. Abrir «Solicitudes recibidas» con una solicitud pendiente 2. Pulsar «Aceptar trueque»	La app cambia el estado a accepted y notifica a quien propuso	PATCH trade_requests {"status":"accepted"} (Estado «pendiente» simulado en el navegador; escrituras interceptadas)	Aprobado
CP-17 (Fig. 67)	Intercambios	**Rechazar una solicitud recibida** 1. Abrir «Solicitudes recibidas» con una solicitud pendiente 2. Pulsar «Rechazar»	La app cambia el estado a rejected y notifica a quien propuso	PATCH trade_requests {"status":"rejected"} (Estado «pendiente» simulado en el navegador; escrituras interceptadas)	Aprobado
CP-18 (Fig. 68)	Intercambios	**Confirmación doble del encuentro** 1. Abrir el chat del trueque aceptado por «Clases de Guitarra» (Norlys ya confirmó)	El chat muestra «¿Realizaron el encuentro?» con el estado de cada parte; el trueque se completa solo cuando ambas confirman	Banner visible: «Tú confirmaste» y «El otro: pendiente»; el botón ya no se ofrece a quien confirmó (requester_confirmed = true, owner_confirmed = false en la base de datos)	Aprobado
CP-19 (Fig. 70)	Intercambios	**Reseña con estrellas** 1. Abrir el formulario de reseña de Rhonny Jaimes 2. Elegir 5 estrellas 3. Escribir un comentario 4. Pulsar «Enviar reseña»	Se muestra «Excelente» y la app envía la reseña (reviews)	POST reviews rating 5 (Escritura interceptada)	Aprobado
CP-20 (Fig. 71)	Comunicación	**Conversación con mensajes** 1. Abrir el chat con Rhonny Jaimes	Se ven la propuesta registrada y los mensajes con hora	Propuesta registrada y mensajes «hola q tal ?», «hols, si estoy interesado», «Super» visibles	Aprobado
CP-21 (Fig. 72)	Comunicación	**Enviar un mensaje de texto** 1. Escribir «Prueba funcional» en el campo de mensaje 2. Pulsar enviar	La app envía el mensaje (messages) con el contenido escrito	POST messages {"conversation_id":"718559b9-7ab7-400b-b885-9552fa48b2d6","sender_id":"ae465756-8673-43a4- (Escritura interceptada)	Aprobado

ID	Módulo	Caso y pasos	Resultado esperado	Resultado obtenido	Estado
CP-22 (Fig. 73)	Comunicación	**Menú de opciones del chat** 1. Pulsar los tres puntos	Aparecen Ver perfil, Ver publicación, Reportar usuario y Bloquear usuario	Opciones visibles: 4 de 4	Aprobado
CP-23 (Fig. 74)	Comunicación	**Reportar a un usuario** 1. En el menú, pulsar «Reportar usuario» 2. Elegir «No se presentó al encuentro» 3. Pulsar «Enviar reporte»	La app envía el reporte (reports) y confirma el envío	POST reports reason «No se presentó al encuentro»; aviso «Reporte enviado» (Escritura interceptada)	Aprobado
CP-24 (Fig. 75)	Comunicación	**Bloquear a un usuario** 1. Abrir el menú de opciones 2. Pulsar «Bloquear usuario» 3. Confirmar con «Bloquear»	La app registra el bloqueo (user_blocks) y confirma	POST user_blocks {"blocker_id":"ae465756-8673-43a4-aac7-7d9710877a7a","blocked_id":"3334b81d-81ed-486f-8e47-381e6e709c45"} (Escritura interceptada; en producción requiere la migración 021)	Aprobado
CP-25 (Fig. 76)	Notificaciones	**Lista de notificaciones** 1. Abrir la campana	Se listan los avisos de propuestas, aceptaciones y reseñas	5 notificaciones visibles	Aprobado
CP-26 (Fig. 77)	Perfil	**Pestañas del perfil** 1. Abrir el perfil 2. Pulsar «Historial» y «Reseñas»	Cada pestaña muestra su contenido	Historial: 2 trueques; Reseñas: comentarios de otros usuarios	Aprobado
CP-27 (Fig. 78)	Perfil	**Editar la biografía del perfil** 1. Abrir «Editar perfil» 2. Escribir una biografía 3. Pulsar «Guardar cambios»	La app actualiza el perfil (profiles) con la nueva biografía	PATCH profiles bio «Prueba funcional de edición» (Escritura interceptada)	Aprobado
CP-28 (Fig. 79)	Métricas e impacto	**Panel de impacto y logros registrados** 1. Abrir «Mi impacto ecológico» 2. Desplazarse hasta «Logros»	Se muestran EcoPoints, nivel y los 16 logros; los alcanzados aparecen marcados como desbloqueados	Panel con «286 EcoPoints» (nivel Brote); 3 logros desbloqueados y guardados en user_achievements: Primera Semilla, Nivel Brote, Primera Reseña	Aprobado
CP-29 (Fig. 80)	Adaptación visual	**Cambio a modo oscuro desde el perfil** 1. Abrir el perfil 2. Pulsar «Tema del sistema» hasta «Modo oscuro»	La interfaz cambia a fondo oscuro	Selector «Modo oscuro»; fondo rgb(26, 26, 24)	Aprobado
CP-30 (Fig. 81)	Autenticación	**Cerrar sesión** 1. En el perfil, pulsar «Cerrar sesión» 2. Confirmar	Vuelve a la pantalla de inicio de sesión	Se muestra «Bienvenido de vuelta» en /login	Aprobado
CP-31 (Fig. 82)	Administración	**Acceso al panel de administración** 1. Abrir el perfil con una cuenta administradora 2. Pulsar «Panel de administración»	Se abre /admin con el contador de reportes pendientes	Ruta /admin; «0 reportes pendientes»	Aprobado
CP-32 (Fig. 83)	Administración	**Filtro de reportes por estado** 1. En el panel, pulsar «Resueltos»	Se lista el reporte resuelto con su nota de revisión	«Nota: Reporte de prueba atendido: no se requieren medidas.»	Aprobado

ID	Módulo	Caso y pasos	Resultado esperado	Resultado obtenido	Estado
CP-33 (Fig. 84)	Administración	**Reabrir un reporte resuelto** 1. Abrir el reporte resuelto 2. Pulsar «Reabrir»	La app llama a admin_update_report con estado pending	POST rpc/admin_update_report {"p_report_id":"b65cd790-d893-4bec-b3d3-1fb2954db031","p_status":"pending","p_note":"Reporte de prueba atendido: no se requieren medidas."} (Escritura interceptada)	Aprobado
CP-34 (Fig. 85)	Administración	**Pausar la publicación reportada** 1. Abrir el reporte 2. Pulsar «Pausar publicación»	La app llama a admin_pause_listing con la publicación reportada	POST rpc/admin_pause_listing {"p_listing_id":"e36ff206-f473-4173-9a5d-43e201984be5"} (Escritura interceptada: la publicación real no se pausó)	Aprobado
CP-35 (Fig. 86)	Notificaciones	**Aviso «Tu reporte fue atendido» (regresión DEF-01)** 1. Abrir la campana de notificaciones	El título se muestra completo: «Tu reporte fue atendido»	Título completo visible (antes de la corrección se mostraba «porte fue atendido»)	Aprobado

Fuente: ejecución de las pruebas funcionales sobre la aplicación publicada; escrituras interceptadas.

10.8. Pruebas no funcionales

Las pruebas no funcionales se ejecutaron el 4 de noviembre de 2026 sobre la aplicación publicada; sus resultados completos están en evidencias/no_funcionales/resultados_no_funcionales.json.

10.8.1. Rendimiento

Se midieron, con la Performance API del navegador, el tiempo hasta el primer byte (TTFB), el primer contenido visible (FCP), el mayor contenido visible (LCP) y el evento de carga, como mediana de tres cargas sin caché en vista móvil. El LCP se compara con el umbral de 2,5 s que Google considera «bueno» en las Core Web Vitals.

Tabla 42. *Tiempos de carga en red normal (mediana de 3 cargas sin caché)*

Pantalla	TTFB	FCP	LCP	Evento load	JavaScript descargado	Resultado
Inicio de sesión	0,1 s	0,7 s	0,8 s	0,6 s	1032 KB	Aprobado
Pantalla principal	0,1 s	0,6 s	1,1 s	0,5 s	1032 KB	Aprobado
Detalle de publicación	0,1 s	0,7 s	1,1 s	0,6 s	1032 KB	Aprobado
Mapa	0,1 s	0,6 s	0,9 s	0,5 s	1032 KB	Aprobado
Panel de impacto	0,1 s	0,6 s	0,6 s	0,5 s	1032 KB	Aprobado

Fuente: Performance API de Google Chrome, aplicación publicada, 4 de noviembre de 2026.

Tabla 43. *Tiempos de carga con red móvil lenta (perfil «Slow 4G»: 150 ms de latencia y 1,6 Mb/s)*

Pantalla	FCP	LCP	Evento load	Resultado
Inicio de sesión	5,8 s	8,1 s	5,7 s	Necesita mejora (DEF-02)
Pantalla principal	5,8 s	8,0 s	5,7 s	Necesita mejora (DEF-02)

Fuente: emulación de red de Google Chrome (DevTools Protocol), 4 de noviembre de 2026.

Tabla 44. *Tiempo de respuesta del backend (20 solicitudes por operación)*

Operación	Media	Mediana	p95	Máximo	Resultado
RPC get_nearby_listings (20 km, Valera)	219 ms	205 ms	286 ms	506 ms	Aprobado
RPC get_nearby_safe_spots (10 km)	199 ms	204 ms	219 ms	236 ms	Aprobado
REST mensajes de una conversación	205 ms	205 ms	243 ms	284 ms	Aprobado
REST perfil propio	205 ms	203 ms	251 ms	269 ms	Aprobado

Fuente: solicitudes HTTP desde el equipo de pruebas, en Venezuela, hacia la API de Supabase (región us-west-2), 4 de noviembre de 2026.

En red normal todas las pantallas cargan con un LCP de alrededor de 1 s y el backend responde con un p95 inferior a 300 ms. Con red móvil lenta el LCP sube a unos 8 s, porque la aplicación descarga 1032 KB de JavaScript en un único archivo; se registra como defecto DEF-02.

10.8.2. Compatibilidad

Tabla 45. *Registro de pruebas de compatibilidad*

Navegador	Dispositivo	Resolución	Comprobaciones	Resultado
Google Chrome 156.0.8078.74	Teléfono pequeño	360 × 740	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Google Chrome 156.0.8078.74	Teléfono estándar	412 × 892	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Google Chrome 156.0.8078.74	Tableta	768 × 1024	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Google Chrome 156.0.8078.74	Portátil	1366 × 768	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Google Chrome 156.0.8078.74	Escritorio	1920 × 1080	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Microsoft Edge 156.0.4314.58	Teléfono pequeño	360 × 740	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Microsoft Edge 156.0.4314.58	Teléfono estándar	412 × 892	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Microsoft Edge 156.0.4314.58	Tableta	768 × 1024	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Microsoft Edge 156.0.4314.58	Portátil	1366 × 768	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado
Microsoft Edge 156.0.4314.58	Escritorio	1920 × 1080	Principal: contenido visible; Mapa: contenido visible; Chat: contenido visible	Aprobado

Fuente: Google Chrome y Microsoft Edge instalados en el equipo de pruebas (Windows 11), 4 de noviembre de 2026. Firefox y Safari no se probaron.

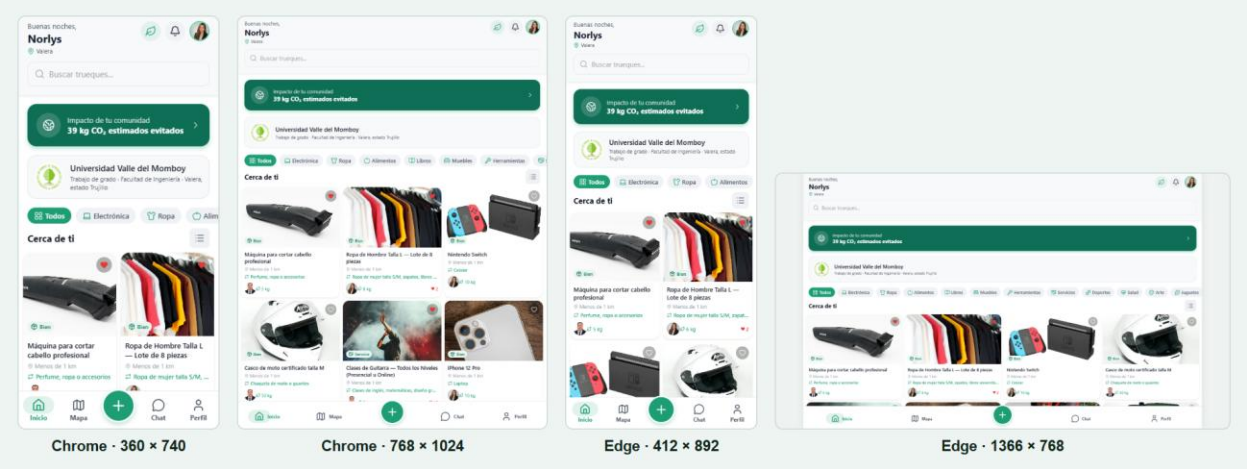


Figura 49. *Pantalla principal en distintos navegadores y resoluciones*

Fuente: capturas del sistema EcoTrueque Venezuela durante las pruebas de compatibilidad.

10.8.3. Accesibilidad

Se analizaron siete pantallas con axe-core 4.10 frente a las reglas WCAG 2.1 de nivel A y AA. La herramienta detecta de forma automática alrededor de la mitad de las barreras posibles; no sustituye una evaluación con usuarios de tecnologías de apoyo.

Tabla 46. *Registro de pruebas de accesibilidad*

Pantalla	Reglas aprobadas	Críticas	Graves	Moderadas	Hallazgos
Inicio de sesión	10	0	1	0	color-contrast (1)
Pantalla principal	22	1	1	0	aria-required-children (1), color-contrast (8)
Detalle de publicación	8	0	2	0	color-contrast (7), scrollable-region-focusable (1)
Chat	15	0	2	0	aria-prohibited-attr (1), color-contrast (6)
Perfil	20	1	1	0	aria-required-children (1), color-contrast (11)
Panel de impacto	16	0	2	0	color-contrast (5), scrollable-region-focusable (1)
Panel de administración	11	0	2	0	aria-prohibited-attr (1), color-contrast (1)

Fuente: axe-core 4.10.2 sobre la aplicación publicada, 4 de noviembre de 2026.

Los hallazgos más frecuentes son textos con contraste de color insuficiente (color-contrast), un contenedor de lista con un rol ARIA incompleto (aria-required-children), atributos ARIA no permitidos en un elemento (aria-prohibited-attr) y zonas desplazables sin acceso por teclado (scrollable-region-focusable). Se registran como defectos DEF-03 y DEF-04.

10.8.4. Seguridad

Tabla 47. *Registro de pruebas de seguridad*

ID	Prueba	Resultado esperado	Resultado obtenido	Estado
SEG-01	Lectura de mensajes sin sesión	Ningún mensaje (RLS)	HTTP 200, 0 filas	Aprobado

ID	Prueba	Resultado esperado	Resultado obtenido	Estado
SEG-02	Lectura de conversaciones sin sesión	Ninguna conversación	HTTP 200, 0 filas	Aprobado
SEG-03	Lectura de notificaciones sin sesión	Ninguna notificación	HTTP 200, 0 filas	Aprobado
SEG-04	Lectura de reportes sin sesión	Ningún reporte	HTTP 200, 0 filas	Aprobado
SEG-05	Correos en perfiles públicos	Ningún correo visible (migración 020)	10 perfiles, 0 con correo	Aprobado
SEG-06	Rol de administrador sin sesión	is_admin() = false	HTTP 200, false	Aprobado
SEG-07	Solicitud con token inválido	Rechazo 401	HTTP 401	Aprobado
SEG-08	Notificaciones de otros usuarios con sesión	Ninguna (solo las propias)	0 filas ajenas	Aprobado
SEG-09	Bloqueos hechos por otros usuarios	Ninguno visible	0 filas ajenas	Aprobado
SEG-10	Foto del chat por URL pública	Rechazo (bucket privado)	HTTP 400	Aprobado
SEG-11	Acceso por HTTP sin cifrar	Redirección a HTTPS	HTTP 308 → https://ecotrueque-venezuela-ten.vercel.app/	Aprobado
SEG-12	Cabecera HSTS en el sitio	Presente	max-age=63072000; includeSubDomains; preload	Aprobado
SEG-13	Credenciales fuera del repositorio	.env excluido en .gitignore	.env incluido en .gitignore	Aprobado

Fuente: solicitudes HTTP a la API de Supabase y al sitio publicado, 4 de noviembre de 2026.

Las pruebas se limitaron a lecturas y a accesos sin sesión para no modificar la base de datos real. Los intentos de escritura no autorizada (por ejemplo, asignarse el rol de administrador) quedan protegidos por las políticas RLS y el trigger protect_is_admin descritos en la sección 7, y deben probarse en un entorno de pruebas si se crea en el futuro.

10.8.5. Usabilidad

La usabilidad se evaluará con el System Usability Scale (SUS), un cuestionario estandarizado de diez afirmaciones con escala de 1 a 5 cuyo resultado va de 0 a 100; un puntaje igual o superior a 68 se considera por encima del promedio. El instrumento, adaptado al español, y la forma de calcular el puntaje están en el Anexo F. Su aplicación a usuarios de la comunidad está pendiente y no se reportan resultados.

10.8.6. Mantenibilidad y portabilidad

El código está escrito en TypeScript con verificación de tipos sin errores, organizado por capas (sección 10.3) y con 49 pruebas unitarias. La portabilidad se apoya en un único código React Native con Expo para Android, iOS y web; en este trabajo se verificó la ejecución web (secciones 8 y 10.7), y la ejecución nativa queda representada de forma ilustrativa (sección 8.3).

10.9. Registro de defectos

Tabla 48. *Defectos encontrados durante las pruebas*

ID	Descripción	Severidad	Detectado en	Estado y acción
DEF-01	El título de algunas notificaciones aparecía recortado («porte fue atendido» en lugar de «Tu reporte fue atendido») por una expresión regular sin las barras de \p{L} y \p{N}	Media	Flujo del panel de administración (sección 8.2)	Corregido en el commit 14df813; verificado con la prueba de regresión CP-35
DEF-02	Carga lenta con red móvil lenta (LCP ≈ 8 s) por un único archivo de JavaScript de 1032 KB	Media	Prueba de rendimiento (Tabla 43)	Abierto. Recomendación: dividir el código por rutas (carga diferida) y comprimir imágenes
DEF-03	Textos con contraste de color insuficiente en varias pantallas	Grave (WCAG AA)	Prueba de accesibilidad (Tabla 46)	Abierto. Recomendación: oscurecer los textos secundarios y grises claros hasta una relación de contraste de 4,5:1
DEF-04	Rol ARIA de lista incompleto, atributos ARIA no permitidos y zonas desplazables sin acceso por teclado	Crítica/grave (WCAG A)	Prueba de accesibilidad (Tabla 46)	Abierto. Recomendación: revisar los componentes de lista, chips y desplazamiento horizontal

Fuente: elaboración propia a partir de los registros de prueba.

10.10. Resumen de la ejecución de pruebas

Tabla 49. *Resumen de las pruebas del sistema*

Tipo de prueba	Casos o mediciones	Aprobados	Observaciones
Unitarias	49	49	Verificación de tipos sin errores
Funcionales	35	35	Todas aprobadas
Rendimiento (red normal)	9	9	Con red móvil lenta: necesita mejora (DEF-02)
Compatibilidad	10	10	Chrome y Edge, 5 resoluciones
Accesibilidad	7 pantallas	—	2 hallazgos críticos y 11 graves (DEF-03, DEF-04)
Seguridad	13	13	Solo lecturas y accesos sin sesión
Usabilidad (SUS)	—	—	Instrumento preparado (Anexo F); aplicación pendiente

Fuente: registros de las secciones 10.6 a 10.9.

11. Matriz de trazabilidad de evidencias

La Tabla 50 relaciona cada evidencia con el elemento técnico que demuestra, su ubicación y su estado. «Verificado» indica que se comprobó contra la fuente primaria; «Parcial», que solo una parte pudo comprobarse; «Pendiente», que requiere información o acceso adicional.

Tabla 50. *Matriz de trazabilidad de evidencias*

N.º	Evidencia	Elemento técnico demostrado	Ubicación o fuente	Estado
1	Diagrama entidad-relación (Figuras 2–6)	Estructura de datos	Sección 5; diagramas/*.mmd y *.png	Verificado
2	Código Mermaid del DER	DER editable	Anexo A; diagramas/der-ecotrueque.mmd	Verificado
3	Verificación de columnas en producción (Tabla 5)	Correspondencia entre scripts y base de datos	Consulta REST a Supabase	Verificado
4	Diccionario de datos (13 tablas)	Campos, tipos y restricciones	Sección 6.2; 001_initial_schema.sql	Verificado
5	Inventario de migraciones (Tabla 24)	Historial de cambios del esquema	supabase/migrations/	Verificado
6	Índices (Tabla 25)	Optimización de consultas	002_indexes.sql	Parcial (no consultable con clave pública)
7	Funciones y triggers (Tabla 26)	Lógica de negocio en la base de datos	003_functions.sql y scripts	Parcial (RPC de lectura verificadas)
8	Políticas RLS (Tabla 28)	Control de acceso	004, 020 y 021	Parcial (efecto observado)
8b	Refuerzos de seguridad (Tabla 30)	Privacidad del correo, notificaciones, logros, fotos y bloqueo	020 y 021	Verificado
9	Buckets de almacenamiento (Tabla 29)	Gestión de archivos	005_storage.sql	Verificado (fotos firmadas del chat)
10	Capturas de pantalla (Figuras 8–37)	Interfaz y funcionalidades	Aplicación publicada; carpeta evidencias/	Verificado (versión web)
10b	Logros registrados en user_achievements	Persistencia de la gamificación	Base de datos	Verificado (3 logros)
11	Representación en Android e iOS (Figuras 46 y 47)	Interfaz en formato de teléfono	Sección 8.3	Ilustrativa (capturas reales en marco de dispositivo)
11b	Capturas en dispositivo móvil físico	Ejecución nativa Android/iOS	Expo Go	Pendiente (opcional)
12	Ficha de versión 1.0.0 (Tabla 32)	Identificación del producto	package.json, app.config.ts, Acerca de	Verificado
13	Etiqueta v1.0.0 en Git	Versión formal en el repositorio	GitHub	Verificado (→ 5224639)
14	Repositorio y commits (Tablas 35 y 36)	Código fuente y trazabilidad (repositorio público)	https://github.com/Norlysc/ecotrueque-venezuela	Verificado
15	Despliegue web	Disponibilidad del producto	https://ecotrueque-venezuela-ten.vercel.app	Verificado (HTTP 200)

N.º	Evidencia	Elemento técnico demostrado	Ubicación o fuente	Estado
16	Arquitectura (Figura 1)	Componentes y comunicación	Sección 4	Verificado
17	Pruebas automatizadas (Tabla 39)	Calidad del software	src/tests/	Verificado (49 aprobadas)
18	Corte de indicadores M1–M7 (Tabla 34)	Métricas de consumo responsable	Consulta 06 en Supabase (toda la plataforma)	Verificado
19	Pruebas funcionales (Tabla 41)	Funcionamiento de la interfaz e integración	Sección 10.7; Anexo G	Verificado (35 de 35 aprobadas)
20	Capturas complementarias (registro paso 2, aceptar y rechazar, confirmación doble, reseña)	Flujos de intercambio completos	Anexo G (CP-03, CP-04, CP-16 a CP-19)	Verificado
21	Sitios seguros de Valera	Puntos de encuentro sugeridos	Migración 022; prueba CP-13	Verificado
19b	Pruebas no funcionales (Tablas 42 a 47)	Rendimiento, compatibilidad, accesibilidad y seguridad	Sección 10.8	Verificado (con defectos registrados en la sección 10.9)
19c	Registro de defectos (Tabla 48)	Gestión de calidad	Sección 10.9	Verificado (1 corregido, 3 abiertos)
19d	Panel de administración (sección 8.2)	Gestión de reportes	Aplicación publicada; migración 024	Verificado
19e	Usabilidad (cuestionario SUS)	Facilidad de uso percibida	Anexo F	Pendiente (aplicación a usuarios)
22	Exportación del esquema desde Supabase (Tabla 6, Anexo E)	Tablas, columnas, tipos, claves y CHECK en el servidor	evidencias/esquema/	Verificado
23	Índices, triggers y políticas desde el servidor	Objetos de base de datos aplicados	supabase/diagnostics/05_inventario_esquema.sql	Pendiente (requiere acceso de administrador)

Fuente: elaboración propia.

12. Limitaciones técnicas y mejoras aplicadas

12.1. Mejoras aplicadas

La primera versión de este anexo identificó doce limitaciones. Las que estaban al alcance del equipo se corrigieron el 4 de noviembre de 2026; la Tabla 51 relaciona cada una con su solución y el commit que la contiene.

Tabla 51. *Limitaciones corregidas y su evidencia*

Limitación original	Solución	Commit	Estado
Cifra fija «2,400 kg CO_2»	Impacto real de la comunidad: suma de los perfiles sin contar dos veces cada trueque	2e1cec0	Publicada
Script 015 incorrecto y no aplicado	Usa participant_1_id y participant_2_id y solo cuenta los mensajes del propio usuario	8331437	Aplicada · verificada en producción
Selector de tema sin efecto en la web	Hook useColorScheme propio y guardado de la preferencia	b174f03	Publicada
Logros no persistidos	Registro en user_achievements y panel con los 16 logros del catálogo	2e1cec0	Publicada
Defecto visual del registro web	El selector de estado ya no estira los botones	fb2ff54	Publicada
Sin pruebas automatizadas	49 pruebas con Jest (sección 10.6)	c240c5c	Publicada
4 errores de tipado	tsc --noEmit sin errores	fb2ff54, 0ad3ec1	Publicada
Migraciones dispersas	Todas en supabase/migrations con numeración 001–021	8116f81	Publicada
Bloqueo de usuarios no implementado	Bloquear y desbloquear desde el chat; políticas que impiden mensajes y propuestas	b06d0d8, 8331437	Aplicada · verificada en producción
Ubicación exacta en el mapa	Cuadrícula de unos 550 m y distancias redondeadas	0ad3ec1	Publicada
Observaciones de seguridad (sección 7.4)	Migración 020	8331437	Aplicada · verificada en producción
Panel de administración no implementado	Pantalla /admin con lista de reportes, notas, cambio de estado y pausa de publicaciones; rol de administrador protegido (migración 024)	fca2a46	Publicada · migración aplicada
Sitios seguros fuera de Trujillo (y marcador simulado en el mapa móvil)	El mapa web y móvil muestra los sitios de safe_spots; la migración 022 carga 8 lugares públicos de Valera (OpenStreetMap). Son sugerencias: el punto lo acuerdan las personas	a60bb61	Aplicada · verificada en producción

Fuente: historial Git de la rama main y verificación en producción.

«Publicada» indica que el cambio está en GitHub y en la versión web de Vercel. Los cambios de base de datos requieren además ejecutar supabase/aplicar_mejoras_2026-11.sql en Supabase; el código de la aplicación funciona igual mientras tanto y avisa si el bloqueo aún no está activado.

12.2. Limitaciones vigentes

- **Notificaciones push.** Solo hay avisos dentro de la aplicación; el envío push requiere compilar con EAS y probar en dispositivos.

- **Suspensión de cuentas.** El panel de administración atiende reportes y pausa publicaciones; suspender cuentas queda para una etapa posterior.
- **Publicación en tiendas.** Requiere cuentas de desarrollador de Google Play y App Store.
- **Indicadores M1–M7.** El corte de la sección 9.2 cubre toda la plataforma, pero se calculó sobre los datos de prueba del MVP; sus valores operacionales requieren uso real o una prueba piloto.
- **Rendimiento y accesibilidad.** Los defectos DEF-02 a DEF-04 (sección 10.9) siguen abiertos con su recomendación.
- **Ejecución nativa.** Las capturas y pruebas corresponden a la versión web en vista móvil; las representaciones de Android e iOS de la sección 8.3 son ilustrativas y no sustituyen capturas en un dispositivo físico.
- **Estimaciones ambientales.** El CO_2 y los residuos son factores promedio por categoría; no son mediciones ni equivalencias certificadas.
- **Alcance de las pruebas.** Las pruebas funcionales interceptan las escrituras para no alterar los datos: verifican la solicitud que envía la aplicación, no su efecto en la base de datos, que depende de las funciones y políticas descritas en las secciones 6 y 7.
- **Recompensas de logros en el cliente.** La suma de la recompensa de un logro se hace desde la aplicación; trasladarla a una función de la base de datos la haría inmodificable por el usuario.

13. Conclusiones

- EcoTrueque Venezuela dispone de evidencia técnica verificable: un repositorio en GitHub con 45 commits, una base de datos cuya estructura coincide con los scripts SQL, 49 pruebas automatizadas aprobadas y una versión web operativa en https://ecotrueque-venezuela-ten.vercel.app.
- La arquitectura cliente – backend como servicio, basada en React Native con Expo y Supabase, permite un único código para Android, iOS y web, y traslada la lógica crítica (cierre de trueques, EcoPoints, niveles y reputación) a funciones y triggers de PostgreSQL protegidos por RLS.
- Las 30 capturas reales demuestran el ciclo completo del trueque: publicación, búsqueda geolocalizada, propuesta, chat con fotos, cierre, reseña y métricas de impacto.
- El producto se identifica como versión 1.0.0 del MVP, asociada al commit 14df813, con funcionalidades parciales y pendientes declaradas explícitamente.
- Las limitaciones detectadas inicialmente se atendieron en el código, incluido el panel de administración; los sitios seguros se mantienen como sugerencias, porque el punto de encuentro lo acuerdan las dos personas. Las limitaciones vigentes son funcionalidades nuevas que exceden el alcance del MVP.
- Las pruebas del sistema quedaron registradas por nivel: 49 pruebas unitarias, 35 casos funcionales (35 aprobados) y pruebas no funcionales de rendimiento, compatibilidad (10/10), accesibilidad y seguridad (13/13). Los defectos encontrados se documentan con su estado y recomendación.
- La versión queda fijada en el repositorio con la etiqueta v1.0.0.

14. Anexos técnicos

Anexo A. Código Mermaid del diagrama entidad-relación

El código completo, con todos los atributos, está en diagramas/der-ecotrueque.mmd. Puede editarse en https://mermaid.live o en cualquier editor compatible con Mermaid. Se reproduce la sección de relaciones:

```
erDiagram
  AUTH_USERS ||--|| PROFILES : "id"
  PROFILES ||--o{ LISTINGS : "publica"
  PROFILES ||--o{ FAVORITES : "marca"
  LISTINGS ||--o{ FAVORITES : "recibe"
  PROFILES ||--o{ TRADE_REQUESTS : "solicita (requester_id)"
  PROFILES ||--o{ TRADE_REQUESTS : "recibe (owner_id)"
  LISTINGS ||--o{ TRADE_REQUESTS : "solicitada / ofrecida"
  TRADE_REQUESTS ||--o{ TRADES : "se completa en"
  PROFILES ||--o{ TRADES : "participa"
  LISTINGS ||--o{ TRADES : "intercambiada"
  TRADE_REQUESTS |o--o{ CONVERSATIONS : "origina"
  PROFILES ||--o{ CONVERSATIONS : "participante 1 / 2"
  LISTINGS |o--o{ CONVERSATIONS : "listing_id"
  CONVERSATIONS ||--o{ MESSAGES : "contiene"
  PROFILES ||--o{ MESSAGES : "envía"
  TRADE_REQUESTS |o--o{ MESSAGES : "propuesta"
  TRADES ||--o{ REVIEWS : "se valora en"
  PROFILES ||--o{ REVIEWS : "escribe / recibe"
  PROFILES ||--o{ USER_ACHIEVEMENTS : "obtiene"
  ACHIEVEMENTS ||--o{ USER_ACHIEVEMENTS : "se otorga"
  PROFILES ||--o{ NOTIFICATIONS : "recibe"
  PROFILES ||--o{ REPORTS : "reporta / es reportado / revisa"
  LISTINGS |o--o{ REPORTS : "reportada"
  PROFILES ||--o{ USER_BLOCKS : "bloquea / es bloqueado"
```

Fuente: diagramas/der-ecotrueque.mmd.

Anexo B. Función complete_trade (código real)

```
CREATE OR REPLACE FUNCTION public.complete_trade(p_request_id UUID)
RETURNS void AS $$
DECLARE
  v_request RECORD;
  v_req_listing RECORD;
  v_off_listing RECORD;
  v_total_co2 NUMERIC;
  v_total_waste NUMERIC;
  v_eco_points_req INTEGER;
  v_eco_points_own INTEGER;
BEGIN
  SELECT * INTO v_request FROM public.trade_requests WHERE id = p_request_id;
  IF NOT FOUND THEN RAISE EXCEPTION 'Trade request not found'; END IF;

  SELECT eco_impact INTO v_req_listing FROM public.listings WHERE id = v_request.requested_listing_id;
  SELECT eco_impact INTO v_off_listing FROM public.listings WHERE id = v_request.offered_listing_id;

  v_total_co2 := COALESCE((v_req_listing.eco_impact->>'co2_saved_kg')::NUMERIC, 0) +
                 COALESCE((v_off_listing.eco_impact->>'co2_saved_kg')::NUMERIC, 0);
  v_total_waste := COALESCE((v_req_listing.eco_impact->>'waste_reduced_kg')::NUMERIC, 0) +
                   COALESCE((v_off_listing.eco_impact->>'waste_reduced_kg')::NUMERIC, 0);
  v_eco_points_req := FLOOR(v_total_co2 * 2 + v_total_waste * 3 + 20);
  v_eco_points_own := FLOOR(v_total_co2 * 2 + v_total_waste * 3 + 20);

  INSERT INTO public.trades (
    trade_request_id, requester_id, owner_id,
    requested_listing_id, offered_listing_id,
    eco_impact_total
  ) VALUES (
    p_request_id, v_request.requester_id, v_request.owner_id,
```

```
    v_request.requested_listing_id, v_request.offered_listing_id,
    jsonb_build_object('co2_saved_kg', v_total_co2, 'waste_reduced_kg', v_total_waste)
  );

  UPDATE public.trade_requests
  SET status = 'completed', completed_at = NOW()
  WHERE id = p_request_id;

  UPDATE public.listings
  SET status = 'traded' WHERE id IN (v_request.requested_listing_id, v_request.offered_listing_id);

  UPDATE public.profiles
  SET
    total_trades = total_trades + 1,
    eco_points = eco_points + v_eco_points_req,
    co2_saved_kg = co2_saved_kg + v_total_co2,
    waste_reduced_kg = waste_reduced_kg + v_total_waste
  WHERE id = v_request.requester_id;

  UPDATE public.profiles
  SET
    total_trades = total_trades + 1,
    eco_points = eco_points + v_eco_points_own,
    co2_saved_kg = co2_saved_kg + v_total_co2,
    waste_reduced_kg = waste_reduced_kg + v_total_waste
  WHERE id = v_request.owner_id;

  PERFORM public.update_eco_level(v_request.requester_id);
  PERFORM public.update_eco_level(v_request.owner_id);

  UPDATE public.profiles SET active_listings_count = (
    SELECT COUNT(*) FROM public.listings WHERE user_id = profiles.id AND status = 'active'
  ) WHERE id IN (v_request.requester_id, v_request.owner_id);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Fuente: supabase/migrations/003_functions.sql, líneas 81 a 145.

Anexo C. Función get_nearby_listings (consulta geoespacial)

```
  RETURN QUERY
  SELECT
    l.id, l.user_id, l.title, l.description,
    l.type, l.category, l.subcategory, l.condition,
    l.images, l.tags, l.looking_for,
    l.estimated_value_usd, l.latitude, l.longitude,
    l.city, l.state, l.status, l.views_count, l.favorites_count,
    l.eco_impact, l.created_at, l.updated_at,
    ROUND((ST_Distance(l.location, v_point) / 1000.0)::NUMERIC, 2)::DOUBLE PRECISION AS distance_km
  FROM public.listings l
  WHERE
    l.status = 'active'
    AND (l.location IS NULL OR ST_DWithin(l.location, v_point, p_radius_km * 1000))
    AND (p_category IS NULL OR l.category = p_category)
    AND (p_listing_type IS NULL OR l.type = p_listing_type)
    AND (
      p_search_query IS NULL
      OR l.search_vector @@ plainto_tsquery('spanish', unaccent(p_search_query))
      OR l.title ILIKE '%' || p_search_query || '%'
    )
  ORDER BY
    CASE WHEN l.location IS NOT NULL THEN ST_Distance(l.location, v_point) ELSE 999999999 END ASC,
    l.created_at DESC
  LIMIT p_limit
  OFFSET p_offset;
```

Fuente: supabase/migrations/003_functions.sql, función get_nearby_listings.

Anexo D. Archivos que acompañan este documento

Archivo	Contenido
evidencias/*.png	30 capturas originales en resolución completa (824 × 1784 px)
diagramas/der-ecotrueque.mmd	DER completo en Mermaid (editable)
diagramas/der-general, der-1 … der-4 (.mmd y .png)	DER general y por dominio
diagramas/arquitectura.mmd y .png	Diagrama de arquitectura
evidencias/pruebas/*.png y resultados.json	Capturas señaladas y registro de los 35 casos de prueba funcional
supabase/diagnostics/05 y 06 (repositorio)	Consultas para el inventario del esquema y el corte de M1–M7
evidencias/esquema/	Exportación del esquema y visualizador de Supabase
evidencias/no_funcionales/	Registro JSON y capturas de las pruebas de rendimiento, compatibilidad, accesibilidad y seguridad
evidencias/admin/ y evidencias/indicadores/	Flujo del panel de administración y resultado de la consulta de M1–M7
evidencias/logros/ y evidencias/dispositivos/	Capturas de los logros registrados y representaciones ilustrativas en Android e iOS

Anexo E. Exportación del esquema desde Supabase

Definición de tablas exportada por los autores desde el panel de Supabase el 4 de noviembre de 2026, reproducida sin cambios. La advertencia inicial es de Supabase: el texto describe el esquema y no está pensado para ejecutarse.

```
-- WARNING: This schema is for context only and is not meant to be run.
-- Table order and constraints may not be valid for execution.

CREATE TABLE public.spatial_ref_sys (
  srid integer NOT NULL CHECK (srid > 0 AND srid <= 998999),
  auth_name character varying,
  auth_srid integer,
  srtext character varying,
  proj4text character varying,
  CONSTRAINT spatial_ref_sys_pkey PRIMARY KEY (srid)
);
CREATE TABLE public.profiles (
  id uuid NOT NULL,
  email text NOT NULL,
  full_name text NOT NULL,
  avatar_url text,
  phone text,
  city text,
  state text,
  bio text,
  is_verified boolean DEFAULT false,
  eco_points integer DEFAULT 0,
  eco_level text DEFAULT 'seedling'::text CHECK (eco_level = ANY (ARRAY['seedling'::text, 'sprout'::text,
'guardian'::text, 'protector'::text, 'hero'::text, 'legend'::text])),
  reputation_score numeric DEFAULT 0.00,
  total_trades integer DEFAULT 0,
  active_listings_count integer DEFAULT 0,
  co2_saved_kg numeric DEFAULT 0,
  waste_reduced_kg numeric DEFAULT 0,
  push_token text,
  created_at timestamp with time zone DEFAULT now(),
  updated_at timestamp with time zone DEFAULT now(),
```

```
  CONSTRAINT profiles_pkey PRIMARY KEY (id),
  CONSTRAINT profiles_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id)
);
CREATE TABLE public.listings (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  user_id uuid NOT NULL,
  title text NOT NULL,
  description text DEFAULT ''::text,
  type text NOT NULL CHECK (type = ANY (ARRAY['good'::text, 'service'::text])),
  category text NOT NULL,
  subcategory text,
  condition text CHECK (condition = ANY (ARRAY['new'::text, 'like_new'::text, 'good'::text, 'fair'::text,
'poor'::text])),
  images jsonb DEFAULT '[]'::jsonb,
  tags ARRAY DEFAULT '{}'::text[],
  looking_for text NOT NULL,
  estimated_value_usd numeric,
  latitude double precision,
  longitude double precision,
  location USER-DEFINED,
  city text,
  state text,
  status text DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'paused'::text,
'traded'::text, 'expired'::text, 'deleted'::text])),
  views_count integer DEFAULT 0,
  favorites_count integer DEFAULT 0,
  eco_impact jsonb DEFAULT '{"co2_saved_kg": 0, "waste_reduced_kg": 0, "estimated_value_usd": 0}'::jsonb,
  search_vector tsvector,
  created_at timestamp with time zone DEFAULT now(),
  updated_at timestamp with time zone DEFAULT now(),
  CONSTRAINT listings_pkey PRIMARY KEY (id),
  CONSTRAINT listings_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.profiles(id)
);
CREATE TABLE public.favorites (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  user_id uuid NOT NULL,
  listing_id uuid NOT NULL,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT favorites_pkey PRIMARY KEY (id),
  CONSTRAINT favorites_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.profiles(id),
  CONSTRAINT favorites_listing_id_fkey FOREIGN KEY (listing_id) REFERENCES public.listings(id)
);
CREATE TABLE public.trade_requests (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  requester_id uuid NOT NULL,
  owner_id uuid NOT NULL,
  requested_listing_id uuid NOT NULL,
  offered_listing_id uuid NOT NULL,
  message text,
  status text DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'accepted'::text,
'rejected'::text, 'cancelled'::text, 'completed'::text])),
  requester_confirmed boolean DEFAULT false,
  owner_confirmed boolean DEFAULT false,
  completed_at timestamp with time zone,
  created_at timestamp with time zone DEFAULT now(),
  updated_at timestamp with time zone DEFAULT now(),
  CONSTRAINT trade_requests_pkey PRIMARY KEY (id),
  CONSTRAINT trade_requests_requester_id_fkey FOREIGN KEY (requester_id) REFERENCES public.profiles(id),
  CONSTRAINT trade_requests_owner_id_fkey FOREIGN KEY (owner_id) REFERENCES public.profiles(id),
  CONSTRAINT trade_requests_requested_listing_id_fkey FOREIGN KEY (requested_listing_id) REFERENCES
public.listings(id),
  CONSTRAINT trade_requests_offered_listing_id_fkey FOREIGN KEY (offered_listing_id) REFERENCES
public.listings(id)
);
CREATE TABLE public.trades (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  trade_request_id uuid NOT NULL,
  requester_id uuid NOT NULL,
  owner_id uuid NOT NULL,
  requested_listing_id uuid NOT NULL,
  offered_listing_id uuid NOT NULL,
  completed_at timestamp with time zone DEFAULT now(),
  eco_impact_total jsonb DEFAULT '{}'::jsonb,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT trades_pkey PRIMARY KEY (id),
  CONSTRAINT trades_trade_request_id_fkey FOREIGN KEY (trade_request_id) REFERENCES public.trade_requests(id),
```

```
  CONSTRAINT trades_requester_id_fkey FOREIGN KEY (requester_id) REFERENCES public.profiles(id),
  CONSTRAINT trades_owner_id_fkey FOREIGN KEY (owner_id) REFERENCES public.profiles(id),
  CONSTRAINT trades_requested_listing_id_fkey FOREIGN KEY (requested_listing_id) REFERENCES
public.listings(id),
  CONSTRAINT trades_offered_listing_id_fkey FOREIGN KEY (offered_listing_id) REFERENCES public.listings(id)
);
CREATE TABLE public.conversations (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  trade_request_id uuid,
  participant_1_id uuid NOT NULL,
  participant_2_id uuid NOT NULL,
  last_message_at timestamp with time zone,
  created_at timestamp with time zone DEFAULT now(),
  listing_id uuid,
  CONSTRAINT conversations_pkey PRIMARY KEY (id),
  CONSTRAINT conversations_trade_request_id_fkey FOREIGN KEY (trade_request_id) REFERENCES
public.trade_requests(id),
  CONSTRAINT conversations_participant_1_id_fkey FOREIGN KEY (participant_1_id) REFERENCES
public.profiles(id),
  CONSTRAINT conversations_participant_2_id_fkey FOREIGN KEY (participant_2_id) REFERENCES
public.profiles(id),
  CONSTRAINT conversations_listing_id_fkey FOREIGN KEY (listing_id) REFERENCES public.listings(id)
);
CREATE TABLE public.messages (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  conversation_id uuid NOT NULL,
  sender_id uuid NOT NULL,
  type text DEFAULT 'text'::text CHECK (type = ANY (ARRAY['text'::text, 'image'::text, 'trade_proposal'::text,
'system'::text])),
  content text,
  image_url text,
  trade_request_id uuid,
  is_read boolean DEFAULT false,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT messages_pkey PRIMARY KEY (id),
  CONSTRAINT messages_conversation_id_fkey FOREIGN KEY (conversation_id) REFERENCES public.conversations(id),
  CONSTRAINT messages_sender_id_fkey FOREIGN KEY (sender_id) REFERENCES public.profiles(id),
  CONSTRAINT messages_trade_request_id_fkey FOREIGN KEY (trade_request_id) REFERENCES
public.trade_requests(id)
);
CREATE TABLE public.reviews (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  trade_id uuid NOT NULL,
  reviewer_id uuid NOT NULL,
  reviewed_id uuid NOT NULL,
  rating smallint NOT NULL CHECK (rating >= 1 AND rating <= 5),
  comment text,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT reviews_pkey PRIMARY KEY (id),
  CONSTRAINT reviews_trade_id_fkey FOREIGN KEY (trade_id) REFERENCES public.trades(id),
  CONSTRAINT reviews_reviewer_id_fkey FOREIGN KEY (reviewer_id) REFERENCES public.profiles(id),
  CONSTRAINT reviews_reviewed_id_fkey FOREIGN KEY (reviewed_id) REFERENCES public.profiles(id)
);
CREATE TABLE public.achievements (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  key text NOT NULL UNIQUE,
  title text NOT NULL,
  description text NOT NULL,
  emoji text NOT NULL,
  points_reward integer DEFAULT 0,
  requirement_type text NOT NULL,
  requirement_value integer NOT NULL,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT achievements_pkey PRIMARY KEY (id)
);
CREATE TABLE public.user_achievements (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  user_id uuid NOT NULL,
  achievement_id uuid NOT NULL,
  unlocked_at timestamp with time zone DEFAULT now(),
  CONSTRAINT user_achievements_pkey PRIMARY KEY (id),
  CONSTRAINT user_achievements_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.profiles(id),
  CONSTRAINT user_achievements_achievement_id_fkey FOREIGN KEY (achievement_id) REFERENCES
public.achievements(id)
);
CREATE TABLE public.safe_spots (
```

```
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  name text NOT NULL,
  type text NOT NULL CHECK (type = ANY (ARRAY['police'::text, 'mall'::text, 'metro'::text, 'bank'::text,
'hospital'::text, 'public'::text])),
  address text NOT NULL,
  city text NOT NULL,
  state text NOT NULL,
  latitude double precision NOT NULL,
  longitude double precision NOT NULL,
  location USER-DEFINED,
  is_24h boolean DEFAULT false,
  phone text,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT safe_spots_pkey PRIMARY KEY (id)
);
CREATE TABLE public.notifications (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  user_id uuid NOT NULL,
  type text NOT NULL,
  title text NOT NULL,
  body text NOT NULL,
  data jsonb DEFAULT '{}'::jsonb,
  is_read boolean DEFAULT false,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT notifications_pkey PRIMARY KEY (id),
  CONSTRAINT notifications_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.profiles(id)
);
CREATE TABLE public.reports (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  reporter_id uuid NOT NULL,
  reported_user_id uuid,
  reported_listing_id uuid,
  reason text NOT NULL,
  description text,
  status text DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'reviewed'::text,
'resolved'::text])),
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT reports_pkey PRIMARY KEY (id),
  CONSTRAINT reports_reporter_id_fkey FOREIGN KEY (reporter_id) REFERENCES public.profiles(id),
  CONSTRAINT reports_reported_user_id_fkey FOREIGN KEY (reported_user_id) REFERENCES public.profiles(id),
  CONSTRAINT reports_reported_listing_id_fkey FOREIGN KEY (reported_listing_id) REFERENCES public.listings(id)
);
CREATE TABLE public.v_user_id (
  id uuid
);
CREATE TABLE public.user_blocks (
  id uuid NOT NULL DEFAULT uuid_generate_v4(),
  blocker_id uuid NOT NULL,
  blocked_id uuid NOT NULL,
  created_at timestamp with time zone DEFAULT now(),
  CONSTRAINT user_blocks_pkey PRIMARY KEY (id),
  CONSTRAINT user_blocks_blocker_id_fkey FOREIGN KEY (blocker_id) REFERENCES public.profiles(id),
  CONSTRAINT user_blocks_blocked_id_fkey FOREIGN KEY (blocked_id) REFERENCES public.profiles(id)
);
```

Fuente: evidencias/esquema/esquema_supabase_2026-11-04.sql.

Anexo F. Instrumento de usabilidad (System Usability Scale)

Cuestionario SUS (Brooke, 1996) adaptado al español para EcoTrueque Venezuela. Cada persona, después de usar la aplicación, indica su grado de acuerdo con cada afirmación de 1 (totalmente en desacuerdo) a 5 (totalmente de acuerdo).

N.º	Afirmación	1	2	3	4	5
1	Creo que me gustaría usar EcoTrueque con frecuencia.	☐	☐	☐	☐	☐
2	Encontré la aplicación innecesariamente compleja.	☐	☐	☐	☐	☐
3	Pensé que la aplicación era fácil de usar.	☐	☐	☐	☐	☐

N.º	Afirmación	1	2	3	4	5
4	Creo que necesitaría ayuda de una persona con conocimientos técnicos para usarla.	☐	☐	☐	☐	☐
5	Las funciones de la aplicación (publicar, buscar, proponer, chatear) están bien integradas.	☐	☐	☐	☐	☐
6	Pensé que había demasiada inconsistencia en la aplicación.	☐	☐	☐	☐	☐
7	Imagino que la mayoría de las personas aprendería a usarla muy rápido.	☐	☐	☐	☐	☐
8	Encontré la aplicación muy difícil de usar.	☐	☐	☐	☐	☐
9	Me sentí con confianza al usar la aplicación.	☐	☐	☐	☐	☐
10	Necesité aprender muchas cosas antes de poder usarla.	☐	☐	☐	☐	☐

Fuente: Brooke, J. (1996). SUS: A quick and dirty usability scale; adaptación propia.

Cálculo. En las afirmaciones impares se resta 1 a la respuesta; en las pares se resta la respuesta a 5. Se suman los diez valores y se multiplica por 2,5. El puntaje final va de 0 a 100; 68 es el promedio de referencia.

Participante	Fecha	Tarea realizada	Puntaje SUS	Observaciones
P1				
P2				
P3				
P4				
P5				

Fuente: plantilla de registro para la aplicación del cuestionario.

Anexo G. Evidencias de las pruebas funcionales

Cada figura corresponde a un caso de la Tabla 41. El recuadro rojo señala el elemento verificado y la etiqueta indica el caso y su resultado (✓ aprobado, ⏸ bloqueado). En los casos CP-16, CP-18 y CP-19 se incluye además la pantalla anterior a la acción.



Figura 50. *CP-01: Inicio de sesión con credenciales válidas*

Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 51. *CP-02: Validación de correo inválido al iniciar sesión*

Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

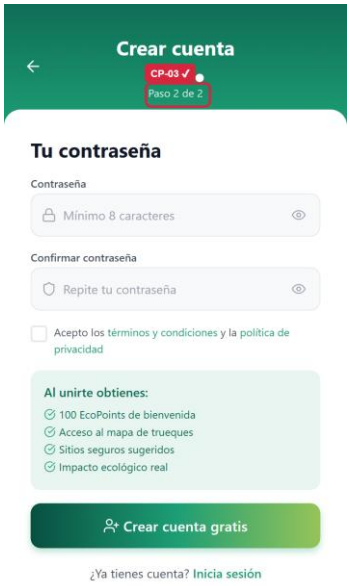


Figura 52. *CP-03: Registro: paso 1 a paso 2*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

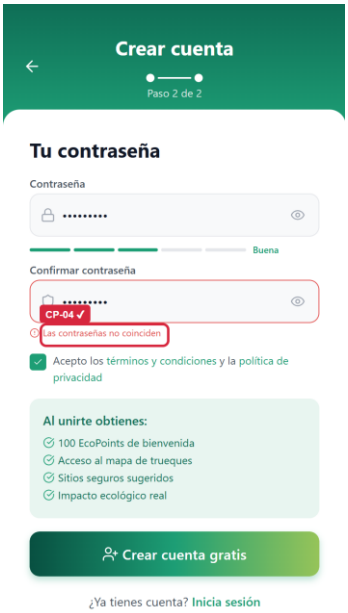


Figura 53. *CP-04: Registro: contraseñas que no coinciden*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 54. *CP-05: Recuperación de contraseña: validación del correo*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

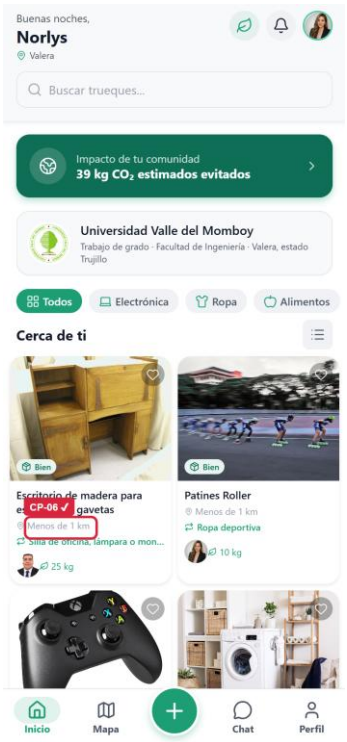


Figura 55. *CP-06: Listado de ofertas cercanas*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

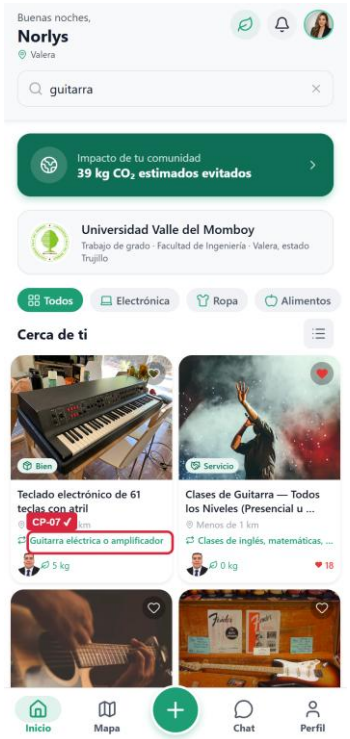


Figura 56. *CP-07: Búsqueda por texto*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

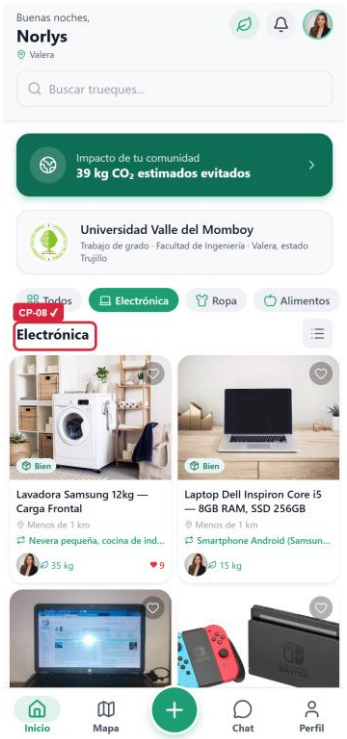


Figura 57. *CP-08: Filtro por categoría*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 58. *CP-09: Detalle de publicación*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 59. *CP-10: Marcar «Me gusta»*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 60. *CP-11: Asistente para publicar (paso 1 a paso 2)*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

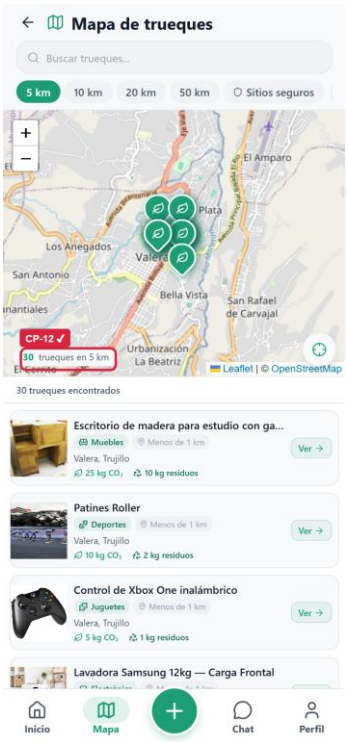


Figura 61. *CP-12: Cambio del radio de búsqueda*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 62. *CP-13: Sitios seguros sugeridos en el mapa*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 63. *CP-14: Enviar una propuesta de trueque*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

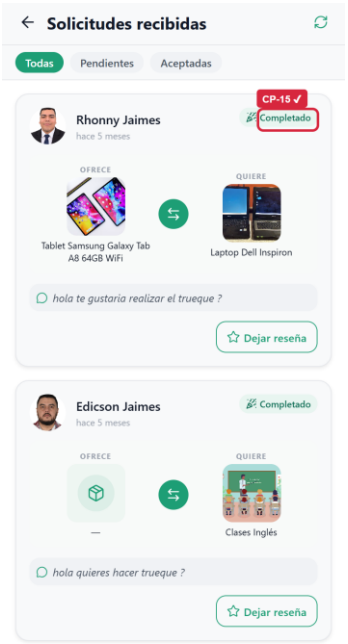


Figura 64. *CP-15: Solicitudes recibidas y filtros*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

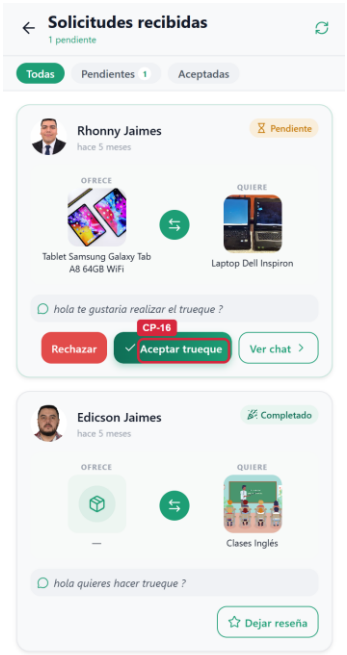


Figura 65. *CP-16: Aceptar una solicitud recibida (antes de la acción)*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

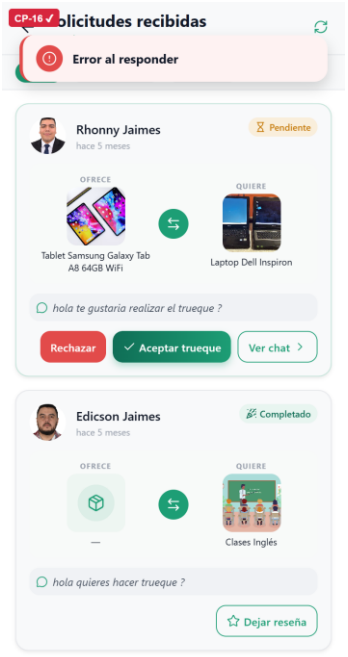


Figura 66. *CP-16: Aceptar una solicitud recibida*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

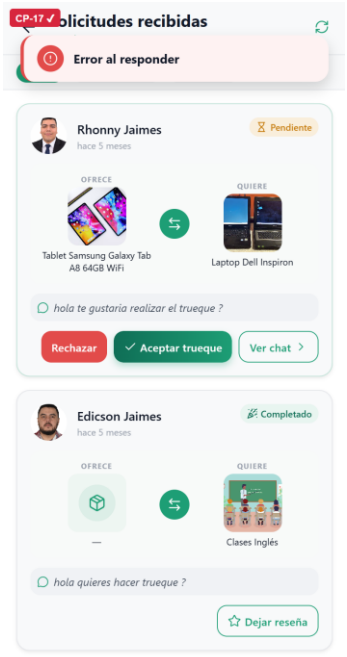


Figura 67. *CP-17: Rechazar una solicitud recibida*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 68. *CP-18: Confirmación doble del encuentro*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

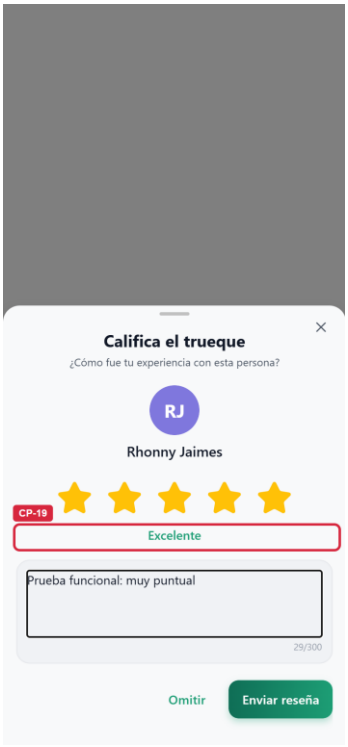


Figura 69. *CP-19: Reseña con estrellas (antes de la acción)*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

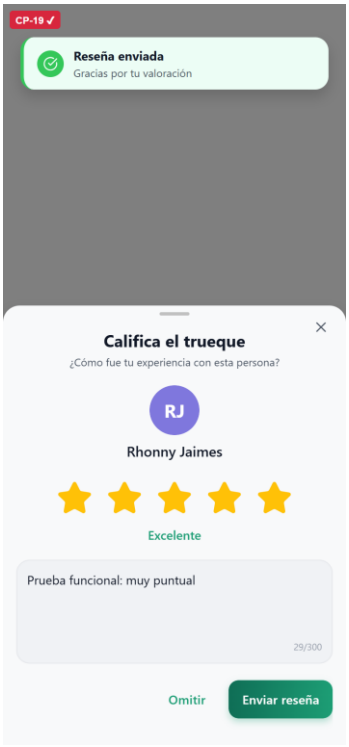


Figura 70. *CP-19: Reseña con estrellas*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 71. *CP-20: Conversación con mensajes*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

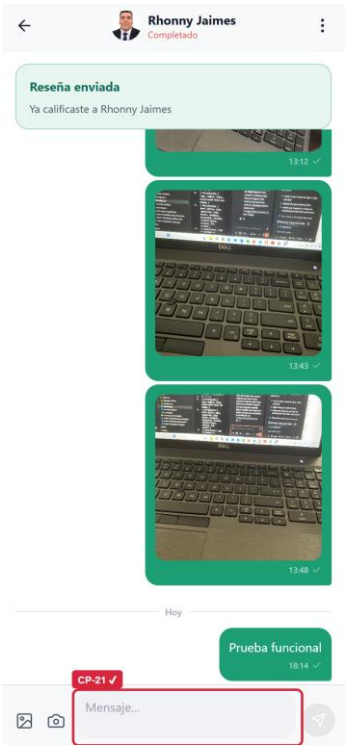


Figura 72. *CP-21: Enviar un mensaje de texto*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 73. *CP-22: Menú de opciones del chat*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

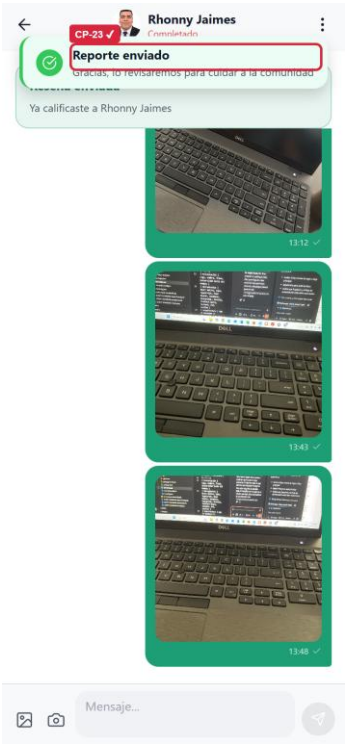


Figura 74. *CP-23: Reportar a un usuario*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

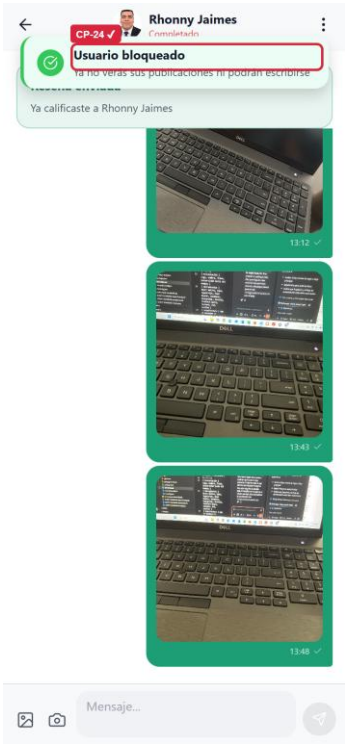


Figura 75. *CP-24: Bloquear a un usuario*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

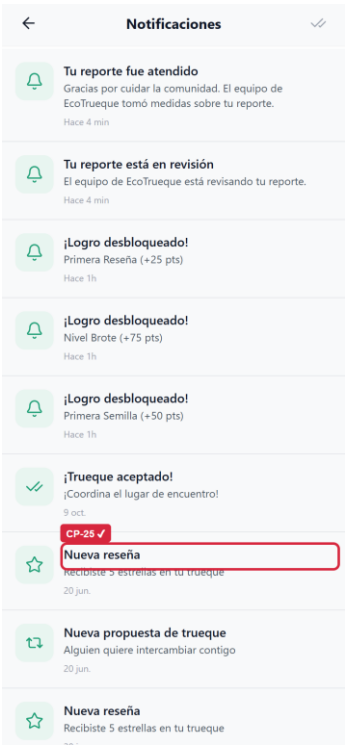


Figura 76. *CP-25: Lista de notificaciones*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

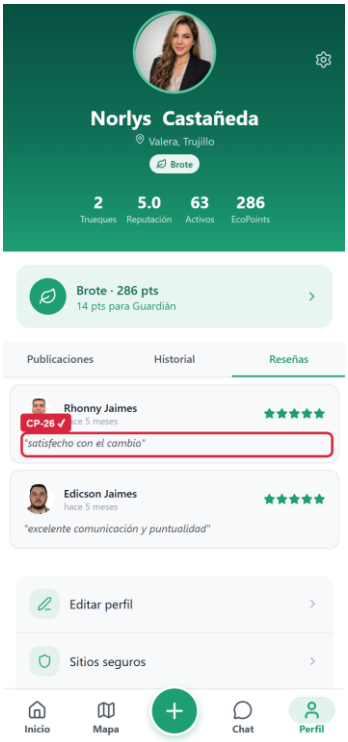


Figura 77. *CP-26: Pestañas del perfil*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

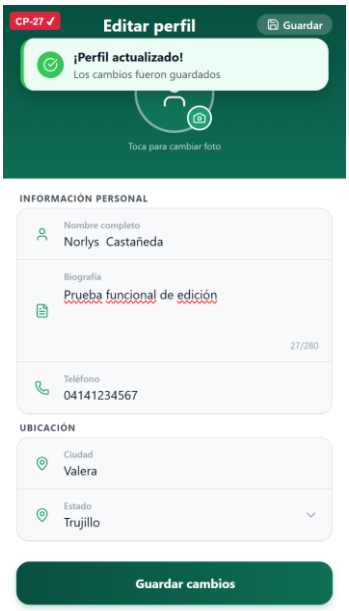


Figura 78. *CP-27: Editar la biografía del perfil*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 79. *CP-28: Panel de impacto y logros registrados*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

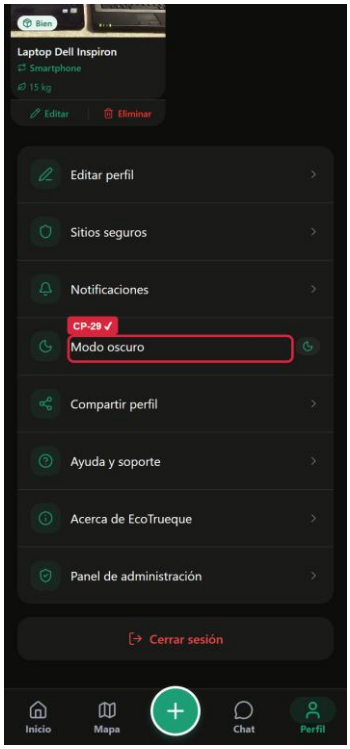


Figura 80. *CP-29: Cambio a modo oscuro desde el perfil*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 81. *CP-30: Cerrar sesión*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 82. *CP-31: Acceso al panel de administración*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 83. *CP-32: Filtro de reportes por estado*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 84. *CP-33: Reabrir un reporte resuelto*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.



Figura 85. *CP-34: Pausar la publicación reportada*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

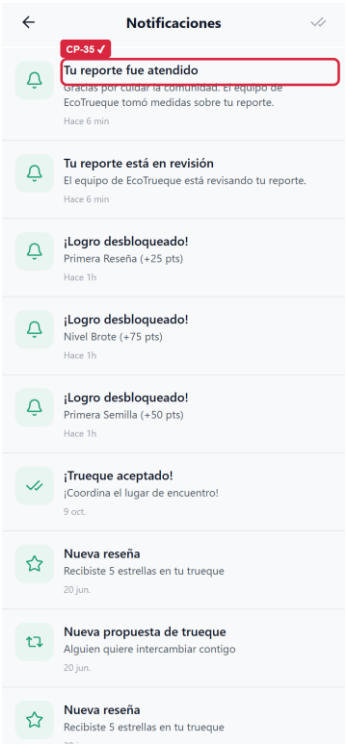


Figura 86. *CP-35: Aviso «Tu reporte fue atendido» (regresión DEF-01)*
Fuente: prueba funcional sobre el sistema EcoTrueque Venezuela.

15. Evidencias pendientes de incorporar o verificar

La Tabla 52 recoge cada evidencia solicitada en la revisión anterior, su estado y dónde se atiende. Quedan pendientes las que requieren acceso de administrador a Supabase y la aplicación del cuestionario de usabilidad.

Tabla 52. *Estado de las evidencias solicitadas*

N.º	Evidencia solicitada	Estado	Dónde se atiende o qué falta
1	Título oficial del trabajo de grado	Atendido	Portada e introducción
2	Exportación del esquema desde Supabase	Atendido (estructura)	Exportación de los autores (Anexo E y Tabla 6): confirma las 14 tablas, columnas, tipos, claves y CHECK. Para índices, triggers y políticas, ejecutar supabase/diagnostics/05_inventario_esquema.sql y adjuntar los CSV
3	Script 015	Corregido · Aplicada · verificada en producción	Usa participant_1_id y participant_2_id (Tabla 24); aplicada con supabase/aplicar_mejoras_2026-11.sql
4	Cambios locales y etiqueta v1.0.0	Atendido	Todo el código está confirmado y publicado; etiqueta v1.0.0 → commit 5224639 (Tabla 35)
5	Visibilidad del repositorio	Atendido	Repositorio público, verificado sin iniciar sesión (Tabla 35)
6	Capturas en dispositivo Android o iOS	Atendido (ilustrativo)	Representaciones en Android e iOS con capturas reales en marco de dispositivo (Figuras 46 y 47), identificadas como ilustrativas. Si se requiere evidencia de ejecución nativa: Expo Go y npx expo start
7	Capturas complementarias	Atendido	Registro paso 2 (CP-03, CP-04), aceptar y rechazar (CP-16, CP-17), confirmación doble (CP-18) y reseña (CP-19), en el Anexo G
8	Registros de pruebas funcionales y no funcionales	Atendido	Plan y entorno (10.5), 49 pruebas unitarias (10.6), 35 casos funcionales (10.7 y Anexo G), pruebas no funcionales (10.8) y registro de defectos (10.9)
9	Corte de datos de M1–M7	Atendido	Sección 9.2: valores de toda la plataforma con la consulta 06 ejecutada en Supabase (M4 = 50,0 %)
10	Sitios seguros de Trujillo	Atendido en código · Aplicada · verificada en producción	Migración 022 con 8 lugares públicos de Valera (OpenStreetMap), mostrados como sugerencia en el mapa
11	Captura de los logros registrados	Atendido	3 logros registrados en user_achievements y 286 EcoPoints (sección 8.1, Figuras 38 a 40)
12	Eliminar la tabla accidental v_user_id	Atendido	Migración 023 ejecutada; la tabla ya no existe (la API responde 404)
13	Base de datos de pruebas separada	No requerida (decisión de los autores)	Las pruebas se ejecutaron en producción interceptando las escrituras (sección 10.5)
14	Aplicación del cuestionario de usabilidad	Pendiente (autores)	Aplicar el SUS del Anexo F a unos 5 usuarios y registrar los puntajes

Fuente: elaboración propia.